AWS Certified Generative AI Developer - Professional

AIP-C01 Exam Study Guide

Comprehensive study material covering all five exam domains,
the AWS Well-Architected Generative AI Lens,
and key AWS services for GenAI development.

Domain	Weight
1. FM Integration, Data Management, and Compliance	31%
2. Implementation and Integration	26%
3. AI Safety, Security, and Governance	20%
4. Operational Efficiency and Optimization	12%
5. Testing, Validation, and Troubleshooting	11%

Table of Contents

AWS Certified Generative AI Developer - Professional (AIP-C01) Exam Guide . . . 5

Overview . . . 5

Exam Details . . . 5

Target Candidate . . . 5

Content Domains and Weightings . . . 6

Domain 1: Foundation Model Integration, Data Management, and Compliance (31%) . . . 6

Domain 2: Implementation and Integration (26%) . . . 8

Domain 3: AI Safety, Security, and Governance (20%) . . . 10

Domain 4: Operational Efficiency and Optimization for GenAI Applications (12%) . . . 12

Domain 5: Testing, Validation, and Troubleshooting (11%) . . . 13

Appendix A: Technologies and Concepts . . . 14

Appendix B: In-Scope AWS Services and Features . . . 15

Appendix C: Service Mentions on the Exam . . . 18

Domain 1: Foundation Model Integration, Data Management, and Compliance . . . 19

Task 1.1 — Analyze Requirements and Design GenAI Solutions . . . 19

Task 1.2 — Select and Configure Foundation Models . . . 20

Task 1.3 — Implement Data Validation and Processing Pipelines . . . 21

Task 1.4 — Design and Implement Vector Store Solutions . . . 23

Task 1.5 — Design Retrieval Mechanisms for FM Augmentation . . . 24

Task 1.6 — Implement Prompt Engineering Strategies and Governance . . . 26

Important Services Reference for Domain 1 . . . 28

Exam Patterns and Common Distractors . . . 29

Quick-Reference: AWS Well-Architected Generative AI Lens . . . 30

Quick-Reference: Retrieval-Augmented Generation (RAG) Pipeline . . . 31

Domain 1 Key Numbers . . . 31

Domain 2: Implementation and Integration — Study Guide . . . 32

Task 2.1: Implement Agentic AI Solutions and Tool Integrations . . . 32

Task 2.2: Implement Model Deployment Strategies . . . 34

Task 2.3: Design and Implement Enterprise Integration Architectures . . . 36

Task 2.4: Implement Foundation Model API Integrations . . . 38

Task 2.5: Implement Application Integration Patterns and Development Tools . . . 40

Key Services Quick Reference for Domain 2 . . . 42
Common Exam Scenarios and Correct Answers . . . 44
Important Notes and Exam Tips . . . 45
Domain 3: AI Safety, Security, and Governance . . . 46
Task 3.1 — Implement Input and Output Safety Controls . . . 46
Task 3.2 — Implement Data Security and Privacy Controls . . . 51
Task 3.3 — Implement AI Governance and Compliance Mechanisms . . . 55
Task 3.4 — Implement Responsible AI Principles . . . 59
Domain 3 Quick-Reference: Service-to-Task Mapping . . . 63
Domain 4: Operational Efficiency and Optimization for GenAI Applications . . . 65
Task 4.1 -- Implement Cost Optimization and Resource Efficiency Strategies . . . 65
Task 4.2 -- Optimize Application Performance . . . 70
Task 4.3 -- Implement Monitoring Systems for GenAI Applications . . . 75
Monitoring Service Quick Reference . . . 80
Common Exam Pitfalls for Domain 4 . . . 81
Domain 5: Testing, Validation, and Troubleshooting . . . 82
Task 5.1 — Implement Evaluation Systems for GenAI . . . 82
Task 5.2 — Troubleshoot GenAI Applications . . . 88
Important Services Reference for Domain 5 . . . 93
Exam Patterns and Common Distractors . . . 95
Quick-Reference: GenAI Evaluation Framework . . . 96
Quick-Reference: Troubleshooting Decision Tree . . . 97
Domain 5 Key Numbers . . . 97
AWS Well-Architected Framework: Generative AI Lens . . . 99
Exam Relevance . . . 99
Overview . . . 99
Design Principles . . . 99
Key Definitions . . . 100
Responsible AI . . . 101
Generative AI Lifecycle . . . 101
Data Architecture . . . 103
Agentic AI . . . 103
Pillar 1: Operational Excellence . . . 104

Pillar 2: Security 108
Pillar 3: Reliability 112
Pillar 4: Performance Efficiency 115
Pillar 5: Cost Optimization 118
Pillar 6: Sustainability 120
Quick Reference: All Best Practices by Risk Level 122
AWS Services Cross-Reference 124

AWS Certified Generative AI Developer - Professional (AIP-C01) Exam Guide

Overview

The AWS Certified Generative AI Developer - Professional (AIP-C01) exam validates a candidate's ability to effectively integrate Foundation Models into applications and business workflows. This certification demonstrates practical knowledge of how to implement GenAI solutions into production environments by using AWS technologies.

Validated Abilities

- Design and implement solutions using vector stores, RAG, knowledge bases, and other GenAI architectures
- Integrate Foundation Models into applications and business workflows
- Apply prompt engineering and management techniques
- Implement agentic AI solutions
- Optimize GenAI applications for cost, performance, and business value
- Implement security, governance, and Responsible AI practices
- Troubleshoot, monitor, and optimize GenAI applications
- Evaluate Foundation Models for quality and responsibility

Exam Details

Attribute	Detail
Exam Code	AIP-C01
Question Types	Multiple choice (1 correct / 3 distractors), Multiple response (2+ correct / 5+ options)
Scored Questions	65
Unscored Questions	10 (not identified on the exam)
Total Questions	75
Score Range	100 - 1,000 (scaled)
Passing Score	750
Scoring Model	Compensatory (no per-section minimum; only overall score matters)

Target Candidate

Experience: 2+ years building production-grade applications on AWS or with open-source technologies, general AI/ML or data engineering experience, and 1 year of hands-on experience implementing GenAI solutions.

Recommended AWS Knowledge

- AWS compute, storage, and networking services
- AWS security best practices and identity management
- AWS deployment and infrastructure as code (IaC) tools
- AWS monitoring and observability services

- AWS cost optimization principles

Out of Scope for the Target Candidate

- Model development and training
- Advanced ML techniques
- Data engineering and feature engineering

Content Domains and Weightings

Domain	Weight
1. Foundation Model Integration, Data Management, and Compliance	31%
2. Implementation and Integration	26%
3. AI Safety, Security, and Governance	20%
4. Operational Efficiency and Optimization for GenAI Applications	12%
5. Testing, Validation, and Troubleshooting	11%

Domain 1: Foundation Model Integration, Data Management, and Compliance (31%)

Task 1.1: Analyze requirements and design GenAI solutions

- **1.1.1** Create comprehensive architectural designs that align with specific business needs and technical constraints (appropriate Foundation Models, integration patterns, deployment strategies).
- **1.1.2** Develop technical proof-of-concept implementations to validate feasibility, performance characteristics, and business value before full-scale deployment (e.g., Amazon Bedrock).
- **1.1.3** Create standardized technical components to ensure consistent implementation across multiple deployment scenarios (AWS Well-Architected Framework, WA Tool Generative AI Lens).

Task 1.2: Select and configure Foundation Models

- **1.2.1** Assess and choose Foundation Models to ensure optimal alignment with specific business use cases and technical requirements (performance benchmarks, capability analysis, limitation evaluation).
- **1.2.2** Create flexible architecture patterns to enable dynamic model selection and provider switching without code modifications (Lambda, API Gateway, AWS AppConfig).
- **1.2.3** Design resilient AI systems to ensure continuous operation during service disruptions (Step Functions circuit breaker patterns, Bedrock Cross-Region Inference, cross-Region model deployment, graceful degradation strategies).
- **1.2.4** Implement Foundation Model customization deployment and lifecycle management (SageMaker AI for

domain-specific fine-tuned models, parameter-efficient adaptation techniques such as LoRA and adapters, SageMaker Model Registry for versioning, automated deployment pipelines, rollback strategies, lifecycle management to retire and replace models).

Task 1.3: Implement data validation and processing pipelines for Foundation Model consumption

- **1.3.1** Create comprehensive data validation workflows to ensure data meets quality standards

for Foundation Model consumption (AWS Glue Data Quality, SageMaker Data Wrangler, custom Lambda functions, CloudWatch metrics).

- **1.3.2** Create data processing workflows to handle complex data types including text, image,

audio, and tabular data with specialized processing requirements for Foundation Model consumption (Bedrock multimodal models, SageMaker Processing, AWS Transcribe, advanced multimodal pipeline architectures).

- **1.3.3** Format input data for Foundation Model inference according to model-specific requirements (JSON

formatting for Bedrock API requests, structured data preparation for SageMaker AI endpoints, conversation formatting for dialog-based applications).

- **1.3.4** Enhance input data quality to improve Foundation Model response quality and consistency (Bedrock to

reformat text, Amazon Comprehend to extract entities, Lambda functions to normalize data).

Task 1.4: Design and implement vector store solutions

- **1.4.1** Create advanced vector database architectures for Foundation Model augmentation to enable efficient

semantic retrieval (Bedrock Knowledge Bases for hierarchical organization, OpenSearch Service with Neural plugin, RDS with S3 document repositories, DynamoDB with vector databases for metadata and embeddings).

- **1.4.2** Develop comprehensive metadata frameworks to improve search precision and context

awareness for Foundation Model interactions (S3 object metadata for document timestamps, custom attributes for authorship, tagging systems for domain classification).

- **1.4.3** Implement high-performance vector database architectures to optimize semantic search

performance at scale (OpenSearch sharding strategies, multi-index approaches for specialized domains, hierarchical indexing techniques).

- **1.4.4** Use AWS services to create integration components to connect with resources (document

management systems, knowledge bases, internal wikis for comprehensive data integration in GenAI applications).

- **1.4.5** Design and deploy data maintenance systems to ensure vector stores contain current and

accurate information (incremental update mechanisms, real-time change detection systems, automated synchronization workflows, scheduled refresh pipelines).

Task 1.5: Design retrieval mechanisms for Foundation Model augmentation

- **1.5.1** Develop effective document segmentation approaches to optimize retrieval performance

(Bedrock chunking capabilities, Lambda functions for fixed-size chunking, custom processing for hierarchical chunking based on content structure).

- **1.5.2** Select and configure optimal embedding solutions to create efficient vector

representations for semantic search (Amazon Titan embeddings, evaluating performance characteristics of Bedrock embedding models, Lambda functions to batch generate embeddings).

- **1.5.3** Deploy and configure vector search solutions to enable semantic search capabilities

(OpenSearch Service with vector search, Aurora with pgvector extension, Bedrock Knowledge Bases with managed vector store functionality).

- **1.5.4** Create advanced search architectures to improve relevance and accuracy of retrieved information (OpenSearch for semantic search, hybrid search combining keywords and vectors, Bedrock reranker models).
- **1.5.5** Develop sophisticated query handling systems to improve retrieval effectiveness and result quality (Bedrock for query expansion, Lambda functions for query decomposition, Step Functions for query transformation).
- **1.5.6** Create consistent access mechanisms to enable seamless integration with Foundation Models (function calling interfaces for vector search, Model Context Protocol [MCP] clients for vector queries, standardized API patterns for retrieval augmentation).

Task 1.6: Implement prompt engineering strategies and governance for Foundation Model interactions

- **1.6.1** Create effective model instruction frameworks to control Foundation Model behavior and outputs (Bedrock Prompt Management to enforce role definitions, Bedrock Guardrails to enforce responsible AI guidelines, template configurations to format responses).
- **1.6.2** Build interactive AI systems to maintain context and improve user interactions with Foundation Models (Step Functions for clarification workflows, Amazon Comprehend for intent recognition, DynamoDB for conversation history storage).
- **1.6.3** Implement comprehensive prompt management and governance systems to ensure consistency and oversight (Bedrock Prompt Management for parameterized templates and approval workflows, S3 for template repositories, CloudTrail for usage tracking, CloudWatch Logs for access logging).
- **1.6.4** Develop quality assurance systems to ensure prompt effectiveness and reliability (Lambda functions to verify expected output, Step Functions to test edge cases, CloudWatch to test prompt regression).
- **1.6.5** Enhance Foundation Model performance to refine prompts iteratively and improve response quality beyond basic prompting (structured input components, output format specifications, chain-of-thought instruction patterns, feedback loops).
- **1.6.6** Design complex prompt systems to handle sophisticated tasks with Foundation Models (Bedrock Prompt Flows for sequential prompt chains, conditional branching based on model responses, reusable prompt components, integrated pre-processing and post-processing steps).

Domain 2: Implementation and Integration (26%)

Task 2.1: Implement agentic AI solutions and tool integrations

- **2.1.1** Develop intelligent autonomous systems with appropriate memory and state management capabilities (Strands Agents and AWS Agent Squad for multi-agent systems, MCP for agent-tool interactions).
- **2.1.2** Create advanced problem-solving systems to give Foundation Models the ability to break down and solve complex problems by following structured reasoning steps (Step Functions to implement ReAct patterns and chain-of-thought reasoning approaches).
- **2.1.3** Develop safeguarded AI workflows to ensure controlled Foundation Model behavior (Step Functions for

stopping conditions, Lambda functions for timeout mechanisms, IAM policies for resource boundaries, circuit breakers to mitigate failures).

- **2.1.4** Create sophisticated model coordination systems to optimize performance across multiple

capabilities (specialized Foundation Models for complex tasks, custom aggregation logic for model ensembles, model selection frameworks).

- **2.1.5** Develop collaborative AI systems to enhance Foundation Model capabilities with human expertise (Step

Functions for review and approval processes, API Gateway for feedback collection mechanisms, human augmentation patterns).

- **2.1.6** Implement intelligent tool integrations to extend Foundation Model capabilities and ensure reliable

tool operations (Strands API for custom behaviors, standardized function definitions, Lambda functions for error handling and parameter validation).

- **2.1.7** Develop model extension frameworks to enhance Foundation Model capabilities (Lambda functions for

stateless MCP servers providing lightweight tool access, ECS for MCP servers providing complex tools, MCP client libraries for consistent access patterns).

Task 2.2: Implement model deployment strategies

- **2.2.1** Deploy Foundation Models based on specific application needs and performance requirements (Lambda

functions for on-demand invocation, Bedrock provisioned throughput configurations, SageMaker AI endpoints for hybrid solutions).

- **2.2.2** Deploy Foundation Model solutions by addressing unique challenges of LLMs that differ from

traditional ML deployments (container-based deployment patterns optimized for memory requirements, GPU utilization, and token processing capacity, specialized model loading strategies).

- **2.2.3** Develop optimized Foundation Model deployment approaches to balance performance and resource

requirements (selecting appropriate models, using smaller pre-trained models for specific tasks, API-based model cascading for routine queries).

Task 2.3: Design and implement enterprise integration architectures

- **2.3.1** Create enterprise connectivity solutions to seamlessly incorporate Foundation Model capabilities into

existing enterprise environments (API-based integrations with legacy systems, event-driven architectures for loose coupling, data synchronization patterns).

- **2.3.2** Develop integrated AI capabilities to enhance existing applications with GenAI

functionality (API Gateway for microservice integrations, Lambda functions for webhook handlers, EventBridge for event-driven integrations).

- **2.3.3** Create secure access frameworks to ensure appropriate security controls (identity

federation between Foundation Model services and enterprise systems, role-based access control, least privilege API access to Foundation Models).

- **2.3.4** Develop cross-environment AI solutions to ensure data compliance across jurisdictions

while enabling Foundation Model access (AWS Outposts for on-premises data integration, AWS Wavelength for edge deployments, secure routing between cloud and on-premises resources).

- **2.3.5** Implement CI/CD pipelines and GenAI gateway architectures to implement secure and

compliant consumption patterns (CodePipeline, CodeBuild, automated testing frameworks for continuous deployment and testing with security scans and rollback support, centralized abstraction layers, observability and control mechanisms).

Task 2.4: Implement Foundation Model API integrations

- **2.4.1** Create flexible model interaction systems (Bedrock APIs for synchronous requests, language-specific AWS SDKs and SQS for asynchronous processing, API Gateway for request validation).
- **2.4.2** Develop real-time AI interaction systems to provide immediate feedback from Foundation Model (Bedrock streaming APIs for incremental response delivery, WebSockets or server-sent events for real-time text generation, API Gateway for chunked transfer encoding).
- **2.4.3** Create resilient Foundation Model systems to ensure reliable operations (AWS SDK for exponential backoff, API Gateway for rate limiting, fallback mechanisms for graceful degradation, X-Ray for observability across service boundaries).
- **2.4.4** Develop intelligent model routing systems to optimize model selection (application code for static routing configurations, Step Functions for dynamic content-based routing, intelligent model routing based on metrics, API Gateway with request transformations for routing logic).

Task 2.5: Implement application integration patterns and development tools

- **2.5.1** Create Foundation Model API interfaces to address the specific requirements of GenAI workloads (API Gateway for streaming responses, token limit management, retry strategies for model timeouts).
- **2.5.2** Develop accessible AI interfaces to accelerate adoption and integration (AWS Amplify for declarative UI components, OpenAPI specifications for API-first development, Bedrock Prompt Flows for no-code workflow builders).
- **2.5.3** Create business system enhancements (Lambda functions for CRM enhancements, Step Functions for document processing systems, Amazon Q Business data sources for internal knowledge tools, Bedrock Data Automation for automated data processing workflows).
- **2.5.4** Enhance developer productivity to accelerate development workflows for GenAI applications (Amazon Q Developer for code generation and refactoring, code suggestions for API assistance, AI component testing, performance optimization).
- **2.5.5** Develop advanced GenAI applications to implement sophisticated AI capabilities (Strands Agents and AWS Agent Squad for AWS native orchestration, Step Functions for agent design patterns, Bedrock for prompt chaining patterns).
- **2.5.6** Improve troubleshooting efficiency for Foundation Model applications (CloudWatch Logs Insights for prompt/response analysis, X-Ray for Foundation Model API call tracing, Amazon Q Developer for GenAI-specific error pattern recognition).

Domain 3: AI Safety, Security, and Governance (20%)

Task 3.1: Implement input and output safety controls

- **3.1.1** Develop comprehensive content safety systems to protect against harmful user inputs to Foundation Models (Bedrock guardrails for content filtering, Step Functions and Lambda functions for custom moderation workflows, real-time validation mechanisms).
- **3.1.2** Create content safety frameworks to prevent harmful outputs (Bedrock guardrails to filter responses, specialized Foundation Model evaluations for content moderation and toxicity detection, text-to-SQL transformations to ensure deterministic results).

- **3.1.3** Develop accuracy verification systems to reduce hallucinations in Foundation Model responses (Bedrock Knowledge Base for grounding responses and fact-checking, confidence scoring and semantic similarity search for verification, JSON Schema to enforce structured outputs).
- **3.1.4** Create defense-in-depth safety systems to provide comprehensive protection against Foundation Model misuse (Amazon Comprehend for pre-processing filters, Bedrock for model-based guardrails, Lambda functions for post-processing validation, API Gateway for API response filtering).
- **3.1.5** Implement advanced threat detection to protect against adversarial inputs and security vulnerabilities (prompt injection and jailbreak detection mechanisms, input sanitization and content filters, safety classifiers, automated adversarial testing workflows).

Task 3.2: Implement data security and privacy controls

- **3.2.1** Develop protected AI environments to ensure comprehensive security for Foundation Model deployments (VPC endpoints for network isolation, IAM policies for secure data access patterns, Lake Formation for granular data access, CloudWatch for data access monitoring).
- **3.2.2** Develop privacy-preserving systems to protect sensitive information during Foundation Model interactions (Amazon Comprehend and Macie for PII detection, Bedrock native data privacy features, Bedrock guardrails for output filtering, S3 Lifecycle configurations for data retention policies).
- **3.2.3** Create privacy-focused AI systems to protect user privacy while maintaining Foundation Model utility (data masking techniques, Amazon Comprehend PII detection, anonymization strategies, Bedrock guardrails).

Task 3.3: Implement AI governance and compliance mechanisms

- **3.3.1** Develop compliance frameworks to ensure regulatory compliance for Foundation Model deployments (SageMaker AI for programmatic model cards, AWS Glue for data lineage tracking, metadata tagging for data source attribution, CloudWatch Logs for decision logs).
- **3.3.2** Implement data source tracking to maintain traceability in GenAI applications (Glue Data Catalog for data source registration, metadata tagging for source attribution in Foundation Model-generated content, CloudTrail for audit logging).
- **3.3.3** Create organizational governance systems to ensure consistent oversight of Foundation Model implementations (frameworks aligned with organizational policies, regulatory requirements, and responsible AI principles).
- **3.3.4** Implement continuous monitoring and advanced governance controls to support safety audits and regulatory readiness (automated detection for misuse, drift, and policy violations, bias drift monitoring, automated alerting and remediation workflows, token-level redaction, response logging, AI output policy filters).

Task 3.4: Implement responsible AI principles

- **3.4.1** Develop transparent AI systems in Foundation Model outputs (reasoning displays for user-facing explanations, CloudWatch for confidence metrics and uncertainty quantification, evidence presentation for source attribution, Bedrock agent tracing for reasoning traces).
- **3.4.2** Apply fairness evaluations to ensure unbiased Foundation Model outputs (pre-defined fairness metrics in CloudWatch, Bedrock Prompt Management and Prompt Flows for systematic A/B testing, Bedrock with LLM-as-a-Judge for automated model evaluations).
- **3.4.3** Develop policy-compliant AI systems to ensure adherence to responsible AI practices

(Bedrock guardrails based on policy requirements, model cards to document Foundation Model limitations, Lambda functions for automated compliance checks).

Domain 4: Operational Efficiency and Optimization for GenAI Applications (12%)

Task 4.1: Implement cost optimization and resource efficiency strategies

- **4.1.1** Develop token efficiency systems to reduce Foundation Model costs while maintaining effectiveness

(token estimation and tracking, context window optimization, response size controls, prompt compression, context pruning, response limiting).

- **4.1.2** Create cost-effective model selection frameworks (cost-capability tradeoff evaluation,

tiered Foundation Model usage based on query complexity, inference cost balancing against response quality, price-to-performance ratio measurement, efficient inference patterns).

- **4.1.3** Develop high-performance Foundation Model systems to maximize resource utilization and throughput

(batching strategies, capacity planning, utilization monitoring, auto-scaling configurations, provisioned throughput optimization).

- **4.1.4** Create intelligent caching systems to reduce costs and improve response times (semantic

caching, result fingerprinting, edge caching, deterministic request hashing, prompt caching).

Task 4.2: Optimize application performance

- **4.2.1** Create responsive AI systems to address latency-cost tradeoffs and improve UX

(pre-computation for predictable queries, latency-optimized Bedrock models for time-sensitive applications, parallel requests for complex workflows, response streaming, performance benchmarking).

- **4.2.2** Enhance retrieval performance to improve relevance and speed of retrieved information

for Foundation Model context augmentation (index optimization, query preprocessing, hybrid search with custom scoring).

- **4.2.3** Implement Foundation Model throughput optimization to address throughput challenges of GenAI

workloads (token processing optimization, batch inference strategies, concurrent model invocation management).

- **4.2.4** Enhance Foundation Model performance to achieve optimal results for specific GenAI use cases

(model-specific parameter configurations, A/B testing, appropriate temperature and top-k/top-p selection).

- **4.2.5** Create efficient resource allocation systems for Foundation Model workloads (capacity planning for

token processing requirements, utilization monitoring for prompt and completion patterns, auto-scaling configurations for GenAI traffic patterns).

- **4.2.6** Optimize Foundation Model system performance for GenAI workflows (API call profiling for

prompt-completion patterns, vector database query optimization for retrieval augmentation, latency reduction techniques specific to LLM inference, efficient service communication patterns).

Task 4.3: Implement monitoring systems for GenAI applications

- **4.3.1** Create holistic observability systems to provide complete visibility into Foundation Model

application performance (operational metrics, performance tracing, Foundation Model interaction tracing, business impact metrics with custom dashboards).

- **4.3.2** Implement comprehensive GenAI monitoring systems to proactively identify issues and

evaluate key performance indicators specific to Foundation Model implementations (CloudWatch to track token usage, prompt effectiveness, hallucination rates and response quality, anomaly detection for token burst patterns and response drift, Bedrock Model Invocation Logs for request/response analysis, performance benchmarks, cost anomaly detection).

- **4.3.3** Develop integrated observability solutions to provide actionable insights for Foundation Model

applications (operational metric dashboards, business impact visualizations, compliance monitoring, forensic traceability and audit logging, user interaction tracking, model behavior pattern tracking).

- **4.3.4** Create tool performance frameworks to ensure optimal tool operation and utilization for

Foundation Models (call pattern tracking, performance metric collection, tool calling observability and multi-agent coordination tracking, usage baselines for anomaly detection).

- **4.3.5** Create vector store operational management systems to ensure optimal vector store

operation and reliability (performance monitoring for vector databases, automated index optimization routines, data quality validation processes).

- **4.3.6** Develop Foundation Model-specific troubleshooting frameworks to identify unique GenAI failure modes

(golden datasets for hallucination detection, output diffing for response consistency analysis, reasoning path tracing for logical errors, specialized observability pipelines).

Domain 5: Testing, Validation, and Troubleshooting (11%)

Task 5.1: Implement evaluation systems for GenAI

- **5.1.1** Develop comprehensive assessment frameworks to evaluate quality and effectiveness of Foundation Model

outputs beyond traditional ML evaluation (metrics for relevance, factual accuracy, consistency, and fluency).

- **5.1.2** Create systematic model evaluation systems to identify optimal configurations (Bedrock

Model Evaluations, A/B testing and canary testing of Foundation Models, multi-model evaluation, cost-performance analysis for token efficiency, latency-to-quality ratios, and business outcomes).

- **5.1.3** Develop user-centered evaluation mechanisms to continuously improve Foundation Model performance

based on user experience (feedback interfaces, rating systems for model outputs, annotation workflows to assess response quality).

- **5.1.4** Create systematic quality assurance processes to maintain consistent performance

standards (continuous evaluation workflows, regression testing for model outputs, automated quality gates for deployments).

- **5.1.5** Develop comprehensive assessment systems to ensure thorough evaluation from multiple

perspectives (RAG evaluation, automated quality assessment with LLM-as-a-Judge techniques, human feedback collection interfaces).

- **5.1.6** Implement retrieval quality testing to evaluate and optimize information retrieval

components (relevance scoring, context matching verification, retrieval latency measurements).

- **5.1.7** Develop agent performance frameworks to ensure agents perform tasks correctly and

efficiently (task completion rate measurements, tool usage effectiveness evaluations, Bedrock Agent evaluations, reasoning quality assessment in multi-step workflows).

- **5.1.8** Create comprehensive reporting systems to communicate performance metrics and insights

to stakeholders (visualization tools, automated reporting mechanisms, model comparison visualizations).

- **5.1.9** Create deployment validation systems to maintain reliability during Foundation Model updates

(synthetic user workflows, AI-specific output validation for hallucination rates and semantic drift, automated quality checks for response consistency).

Task 5.2: Troubleshoot GenAI applications

- **5.2.1** Resolve content handling issues to ensure necessary information is processed completely

in Foundation Model interactions (context window overflow diagnostics, dynamic chunking strategies, prompt design optimization, truncation-related error analysis).

- **5.2.2** Diagnose and resolve Foundation Model integration issues to identify and fix API integration problems

specific to GenAI services (error logging, request validation, response analysis).

- **5.2.3** Troubleshoot prompt engineering problems to improve Foundation Model response quality and consistency

beyond basic prompt adjustments (prompt testing frameworks, version comparison, systematic refinement).

- **5.2.4** Troubleshoot retrieval system issues to identify and resolve problems affecting

information retrieval effectiveness for Foundation Model augmentation (model response relevance analysis, embedding quality diagnostics, drift monitoring, vectorization issue resolution, chunking and preprocessing remediation, vector search performance optimization).

- **5.2.5** Troubleshoot prompt maintenance issues to continuously improve Foundation Model interaction

performance (template testing and CloudWatch Logs for prompt confusion diagnosis, X-Ray for prompt observability pipelines, schema validation for format inconsistencies, systematic prompt refinement workflows).

Appendix A: Technologies and Concepts

The following technologies and concepts might appear on the exam (non-exhaustive, subject to change):

- Retrieval Augmented Generation (RAG)
- Vector databases and embeddings
- Prompt engineering and management
- Foundation Model integration
- Agentic AI systems
- Responsible AI practices
- Content safety and moderation
- Model evaluation and validation
- Cost optimization for AI workloads
- Performance tuning for AI applications
- Monitoring and observability for AI systems
- Security and governance for AI applications
- API design and integration patterns
- Event-driven architectures
- Serverless computing
- Container orchestration
- Infrastructure as code (IaC)
- CI/CD for AI applications
- Hybrid cloud architectures
- Enterprise system integration

Appendix B: In-Scope AWS Services and Features

Analytics

- Amazon Athena
- Amazon EMR
- AWS Glue
- Amazon Kinesis
- Amazon OpenSearch Service
- Amazon QuickSight
- Amazon Managed Streaming for Apache Kafka (Amazon MSK)

Application Integration

- Amazon AppFlow
- AWS AppConfig
- Amazon EventBridge
- Amazon SNS
- Amazon SQS
- AWS Step Functions

Compute

- AWS App Runner
- Amazon EC2
- AWS Lambda
- AWS Lambda@Edge
- AWS Outposts
- AWS Wavelength

Containers

- Amazon ECR
- Amazon ECS
- Amazon EKS
- AWS Fargate

Customer Engagement

- Amazon Connect

Database

- Amazon Aurora
- Amazon DocumentDB
- Amazon DynamoDB
- Amazon DynamoDB Streams
- Amazon ElastiCache
- Amazon Neptune

- Amazon RDS

Developer Tools

- AWS Amplify
- AWS CDK
- AWS CLI
- AWS CloudFormation
- AWS CodeArtifact
- AWS CodeBuild
- AWS CodeDeploy
- AWS CodePipeline
- AWS Tools and SDKs
- AWS X-Ray

Machine Learning

- Amazon Augmented AI
- Amazon Bedrock
- Amazon Bedrock AgentCore
- Amazon Bedrock Knowledge Bases
- Amazon Bedrock Prompt Management
- Amazon Bedrock Prompt Flows
- Amazon Comprehend
- Amazon Kendra
- Amazon Lex
- Amazon Q Business
- Amazon Q Business Apps
- Amazon Q Developer
- Amazon Rekognition
- Amazon SageMaker AI
- Amazon SageMaker Clarify
- Amazon SageMaker Data Wrangler
- Amazon SageMaker Ground Truth
- Amazon SageMaker JumpStart
- Amazon SageMaker Model Monitor
- Amazon SageMaker Model Registry
- Amazon SageMaker Neo
- Amazon SageMaker Processing
- Amazon SageMaker Unified Studio
- Amazon Textract
- Amazon Titan
- Amazon Transcribe

Management and Governance

- AWS Auto Scaling

- AWS Chatbot
- AWS CloudTrail
- Amazon CloudWatch
- Amazon CloudWatch Logs
- Amazon CloudWatch Synthetics
- AWS Cost Anomaly Detection
- AWS Cost Explorer
- Amazon Managed Grafana
- AWS Service Catalog
- AWS Systems Manager
- AWS Well-Architected Tool

Migration and Transfer

- AWS DataSync
- AWS Transfer Family

Networking and Content Delivery

- Amazon API Gateway
- AWS AppSync
- Amazon CloudFront
- Elastic Load Balancing (ELB)
- AWS Global Accelerator
- AWS PrivateLink
- Amazon Route 53
- Amazon VPC

Security, Identity, and Compliance

- Amazon Cognito
- AWS Encryption SDK
- IAM
- IAM Access Analyzer
- IAM Identity Center
- AWS KMS
- Amazon Macie
- AWS Secrets Manager
- AWS WAF

Storage

- Amazon EBS
- Amazon EFS
- Amazon S3
- Amazon S3 Intelligent-Tiering
- Amazon S3 Lifecycle policies
- Amazon S3 Cross-Region Replication

Appendix C: Service Mentions on the Exam

AWS Certification uses official short names for well-known AWS services on this exam to reduce reading load. For example, "Amazon Simple Notification Service (Amazon SNS)" appears as "Amazon SNS". The exam's Help feature contains the full mapping of short names to full names. Not every abbreviation is fully spelled out; some AWS services have abbreviations that are never expanded (e.g., Amazon API Gateway, Amazon EMR).

Domain 1: Foundation Model Integration, Data Management, and Compliance

Exam Weight: 31% — the largest domain on the exam

This domain covers six task areas: designing GenAI solutions, selecting and configuring Foundation Models (FMs), building data pipelines, implementing vector stores, designing retrieval mechanisms, and implementing prompt engineering with governance.

Task 1.1 — Analyze Requirements and Design GenAI Solutions

Key Concepts

Architectural Design Alignment (Skill 1.1.1)

When designing a GenAI solution, match every architectural choice to a stated business need or technical constraint:

- Choose FMs based on task type (text generation, summarization, classification, code generation, multimodal), latency SLA, and cost envelope.
- Select integration patterns: synchronous API calls for real-time responses; asynchronous via SQS for batch or bursty workloads; streaming for progressive UI rendering.
- Choose deployment strategies: on-demand (pay-per-token) vs. Provisioned Throughput (predictable high-volume) vs. self-hosted on SageMaker (data residency, custom models).

Proof-of-Concept with Amazon Bedrock (Skill 1.1.2)

Amazon Bedrock is the primary PoC vehicle on the exam. Key PoC steps:

1. Use the Bedrock console Playground to run ad-hoc prompts against multiple models.
2. Use Bedrock Model Evaluation to compare models on accuracy, robustness, and toxicity using your own dataset before committing to a model.
3. Measure token costs, latency, and quality metrics. Establish baselines before full deployment.

Standardized Components via Well-Architected (Skill 1.1.3)

The **AWS Well-Architected Generative AI Lens** extends the six Well-Architected pillars to GenAI:

Pillar	GenAI Focus
Operational Excellence	Prompt template management, versioning, MLOps pipelines
Security	IAM least privilege, VPC isolation, Guardrails, PII protection
Reliability	Cross-Region inference, circuit breakers, graceful degradation
Performance Efficiency	Model selection, caching, batching, provisioned throughput
Cost Optimization	Token efficiency, tiered routing, right-sized models
Sustainability	Efficient inference, smaller models where appropriate

The Generative AI Lens covers six lifecycle phases: **scoping, model selection, customization, integration, deployment, and iteration**. Use the **AWS WA Tool** (accessible from the console) to run assessments against the Generative AI Lens catalog.

> ***Exam tip:*** *If a question asks how to ensure "consistent implementation across multiple deployment scenarios," the answer involves the Well-Architected Framework and/or the WA Tool Generative AI Lens.*

Task 1.2 — Select and Configure Foundation Models

Key Concepts

FM Evaluation Framework (Skill 1.2.1)

Evaluate FMs across four dimensions:

1. **Task Performance** — task-specific accuracy, few-shot learning, instruction-following fidelity, output consistency, domain knowledge, reasoning capability.
2. **Architectural Characteristics** — parameter count, context window size, modality (text, image, video, audio, embedding), tokenization methodology.
3. **Operational Considerations** — throughput/latency profiles, cost-per-token, scalability, customization options, ease of integration, security.
4. **Responsible AI Attributes** — hallucination propensity, bias measurements, safety guardrail effectiveness, explainability, legal implications.

For **agentic AI** use cases, also evaluate: planning/reasoning capability, tool and API integration quality, agent-to-agent communication, and multi-agent collaboration.

Bedrock Model Evaluation supports:

- **Automatic evaluations** — predefined metrics (accuracy, robustness, toxicity) against curated datasets.
- **Human evaluations** — bring your own dataset, define custom metrics (relevance, brand alignment, style).
- **LLM-as-a-Judge** — use a judge model (e.g., Claude, Nova Pro) to score generator model outputs on correctness, completeness, harmfulness, and refusal.

Dynamic Model Selection (Skill 1.2.2)

Pattern: decouple model identity from application code so you can swap providers without code changes.

- **AWS AppConfig** — store the current model ID (e.g., `anthropic.claude-3-5-sonnet-20241022-v2:0`) as a configuration value; Lambda reads it at runtime. Changing the model is a config update, not a code deployment.
- **Amazon API Gateway** — front-end the model invocation; route requests to different backend Lambda functions or Bedrock endpoints via request transformation.
- **AWS Lambda** — stateless handler that reads the model identifier from AppConfig and calls `bedrock-runtime:InvokeModel`.

Resilient AI Systems (Skill 1.2.3)

Patterns for continuity during service disruptions:

Pattern	Implementation
Cross-Region Inference (Geographic)	Keeps data within a geographic boundary; higher throughput than single-region; best for data residency requirements

Pattern	Implementation
Cross-Region Inference (Global)	Routes to any supported commercial region worldwide; ~10% cost savings; highest throughput; best without geographic restrictions
Circuit Breaker	Step Functions state machine: if model invocation fails N times, open the circuit and return a cached/fallback response
Graceful Degradation	Fallback to a smaller, cheaper model or a deterministic rule-based response when the primary FM is unavailable
Fallback Model	Configure alternate model IDs in AppConfig; Lambda switches to fallback on 5xx or throttle errors

Cross-Region inference key facts:

- No additional cost for cross-region routing.
- All data stays encrypted on the AWS network during transit.
- CloudTrail logs are recorded in the **source region only**.
- Geographic profiles maintain data residency; Global profiles do not constrain geography.

FM Customization Lifecycle (Skill 1.2.4)

Technique	When to Use	AWS Service
Fine-tuning (SFT)	Labeled examples available; adapt behavior/style	Amazon Bedrock or SageMaker AI
Continued Pre-training	Large unlabeled domain corpus; expand knowledge	Bedrock or SageMaker AI
LoRA (Low-Rank Adaptation)	Parameter-efficient; fast; lower compute cost	SageMaker AI adapter inference components
Full Fine-tuning (FFT)	Large dataset; maximum adaptation; higher compute	SageMaker AI
DPO / PPO (Alignment)	Steer model outputs toward preferences	SageMaker AI (Nova customization)
Knowledge Distillation	Transfer from large teacher to small student model	Bedrock Model Distillation

SageMaker Model Registry — centralized catalog for model versioning:

- Register each trained model as a new version under a Model Group.
- Associate metadata: training metrics, dataset pointers, approval status.
- Control deployment through approval workflows (pending/approved/rejected).
- Integrate with CI/CD pipelines (CodePipeline + CodeBuild) for automated promotion.
- Rollback by re-pointing the endpoint to a previous approved model version.

> ***Exam tip:*** *LoRA is the correct answer for "parameter-efficient adaptation" questions. SageMaker Model Registry is the correct answer for "versioning and lifecycle management of customized models."*

Task 1.3 — Implement Data Validation and Processing Pipelines

Key Concepts

Data Validation Workflows (Skill 1.3.1)

Service	Role in Validation
AWS Glue Data Quality	Rule-based validation (DQDL — Data Quality Definition Language), ML anomaly detection, serverless, integrates with Glue ETL jobs and the Glue Data Catalog
SageMaker Data Wrangler	Visual data preparation, profiling, transformation; generates code for SageMaker Pipelines
AWS Lambda	Custom validation logic (field format checks, business rule enforcement) before data enters the FM pipeline
Amazon CloudWatch	Publish custom metrics for data quality scores; trigger alarms on validation failure rates

AWS Glue Data Quality (DQDL) key rules types: `Completeness`, `Uniqueness`, `Referential Integrity`, `RowCount`, `ColumnValues`. Records that fail rules can be written to a separate rejected S3 path for remediation.

Multimodal Data Processing (Skill 1.3.2)

FMs consume multiple data types. Appropriate processing per modality:

Modality	Processing Service
Text	AWS Glue ETL, Lambda, SageMaker Processing
Audio	Amazon Transcribe → text → embed or ingest
Images	Amazon Rekognition (labels/text extraction), Bedrock Nova multimodal embeddings
Video	Bedrock Data Automation, Amazon Rekognition Video, Nova multimodal embeddings
Tabular	SageMaker Data Wrangler, Glue ETL, Bedrock structured data KB (Redshift / Glue Data Catalog)

Bedrock multimodal models (e.g., Anthropic Claude, Amazon Nova) can natively consume images and text in a single API request. For video and audio, you typically convert to text (Transcribe) or use Bedrock Data Automation before passing to the FM.

Input Formatting (Skill 1.3.3)

Every model has its own prompt format. Know the key patterns:

- **Bedrock Converse API** — model-agnostic abstraction; automatically handles model-specific formatting; recommended for new integrations.
- **Bedrock InvokeModel API** — model-specific JSON body (e.g., `anthropic_version`, `messages`, `max_tokens` for Claude; `inputText`, `textGenerationConfig` for Titan).
- **SageMaker Endpoints** — send the request body in the format expected by the model container.
- **Conversation formatting** — for multi-turn dialogs, maintain a `messages` array with alternating `user` and `assistant` roles. Store history in DynamoDB; retrieve and prepend on each invocation.

Input Quality Enhancement (Skill 1.3.4)

Enhancement	Service
Text reformatting / normalization	Bedrock (prompt the FM to clean/normalize), Lambda
Entity extraction	Amazon Comprehend (entities, key phrases, PII, sentiment)
Data normalization	Lambda (regex, format standardization), Glue
Noise removal / deduplication	SageMaker Processing, Glue ETL

Amazon Comprehend provides pre-built NLP models for entity recognition, key phrase extraction, language detection, sentiment analysis, and PII detection — no training required.

Task 1.4 — Design and Implement Vector Store Solutions

Key Concepts

Vector Database Architectures (Skill 1.4.1)

A vector database stores dense numerical representations (embeddings) that encode semantic meaning. Similarity search finds semantically related content even without keyword overlap.

AWS Vector Store Option	Best For
Amazon OpenSearch Service (Serverless or managed)	Full-text + vector hybrid search, large scale, real-time updates, Neural plugin for Bedrock integration
Amazon Aurora PostgreSQL with pgvector	Existing relational workloads that need vector capability; SQL-compatible; transactional consistency
Amazon Neptune Analytics	Graph + vector hybrid; knowledge graphs with vector search
Amazon DynamoDB + external vector store	Metadata and scalar lookups in DynamoDB; embeddings in a dedicated vector DB
Amazon Bedrock Knowledge Bases	Fully managed; abstracts ingestion, vectorization, and retrieval; quickest time-to-value
Pinecone / Redis Enterprise Cloud / MongoDB Atlas	Third-party options supported by Bedrock Knowledge Bases

Amazon Bedrock Knowledge Bases supports these vector stores: OpenSearch Serverless, Aurora PostgreSQL, Neptune Analytics, Pinecone, Redis Enterprise Cloud, MongoDB Atlas, and Amazon S3 Vectors (cost-optimized, durable, sub-second queries).

Metadata Frameworks (Skill 1.4.2)

Attach metadata to vector documents to enable filtered retrieval:

- **S3 object metadata** — document timestamps, version, source URL stored as S3 metadata tags.
- **Custom attributes** — author, department, classification level, domain.
- **Tagging systems** — domain taxonomy for filtering by topic (e.g., `domain: legal`).

Metadata filtering in Bedrock Knowledge Bases: pass filter expressions in the `RetrieveAndGenerate` or `Retrieve` API calls to scope results (e.g., `{"equals": {"key": "domain", "value": "legal"}}`).

High-Performance Vector Architecture (Skill 1.4.3)

OpenSearch Service performance levers:

- **Sharding** — distribute index across multiple nodes; increase shard count for large indexes.
- **HNSW (Hierarchical Navigable Small World)** — approximate nearest-neighbor algorithm; tunable `m` (connections per node) and `ef_construction` (build-time quality) parameters.
- **IVF (Inverted File Index)** — alternative ANN algorithm; good for very large indexes with faster build times.
- **Multi-index approach** — separate indexes per domain or content type; reduces search space and improves relevance.
- **Quantization (product quantization, scalar quantization)** — compress vectors to reduce RAM usage; trades slight accuracy loss for cost savings.
- **Disk-based vector search** — keep compressed vectors in memory for candidate selection; retrieve full-precision vectors from disk for final scoring.
- **GPU-accelerated indexing** — serverless GPU attachment for billion-scale index builds.

Data Maintenance for Vector Stores (Skill 1.4.5)

Vector stores must reflect current source data:

Mechanism	Description
Incremental sync	Re-index only changed/new documents (Bedrock KB sync job detects additions and deletions)
Real-time change detection	DynamoDB Streams → Lambda → re-embed and upsert into vector store
EventBridge rules	Trigger re-indexing on S3 object creation/update events
Scheduled refresh	CloudWatch Events / EventBridge Scheduler → Step Functions pipeline → full or partial re-sync
TTL-based expiry	Remove stale embeddings using vector store TTL features or scheduled Lambda cleanup

---

Task 1.5 — Design Retrieval Mechanisms for FM Augmentation

Key Concepts

Chunking Strategies (Skill 1.5.1)

Chunking splits documents into pieces that fit within embedding model token limits and maximize retrieval precision.

Strategy	How It Works	Best For
Fixed-size chunking	Divide by token count with configurable overlap	Simple text; uniform structure
Default Bedrock chunking	~300 tokens; preserves sentence boundaries	General-purpose
Hierarchical chunking	Parent chunks + child chunks; retrieve child, return parent for context	Long documents where context matters
Semantic chunking	NLP-based; split at topic/semantic boundaries; configurable breakpoint percentile	High-quality retrieval; nuanced content
No chunking	Each document is one chunk	Short documents (PDFs, emails)

Hierarchical chunking detail: child chunks are retrieved for precision; parent chunks (larger context window) are returned to the FM for broader context. Configurable parent/child sizes and overlap tokens.

> ***Exam tip:*** *Hierarchical chunking is the best answer for "improve context quality while maintaining retrieval precision."*

Embedding Models (Skill 1.5.2)

An embedding model converts text (or multimodal content) into a dense vector. Key properties:

- **Dimensionality** — higher dimensions capture more semantic nuance but cost more storage/compute.
- **Domain fit** — a model trained on legal text will produce better embeddings for legal queries.
- **Max token input** — must be >= your chunk size.

AWS embedding options:

Model	Key Characteristics
Amazon Titan Text Embeddings V2	Supports binary vector embeddings; variable dimensions (256/512/1024); available in all Bedrock Knowledge Bases regions
Cohere Embed	High-quality multilingual; binary embeddings supported; optimized for retrieval
Amazon Nova Multimodal Embeddings	Cross-modal retrieval (text, image, video, audio) in a unified vector space

Binary vector embeddings (Titan V2, Cohere Embed): represent each dimension as 0 or 1. Benefits: lower storage cost, faster similarity computation. Tradeoff: slightly less precise than float32. Currently supported with OpenSearch Serverless as the vector store.

Batch embedding generation: use Lambda to call `bedrock:InvokeModel` for the embedding model in batches to reduce API calls and cost.

Vector Search Solutions (Skill 1.5.3)

Solution	Notes
OpenSearch Service with vector search	k-NN and Approximate k-NN (HNSW/IVF); supports `knn_vector` field; scalable to billions of vectors
Aurora PostgreSQL + pgvector	SQL-based vector operations (`<->` cosine distance operator); transactional; good for hybrid relational + vector queries
Bedrock Knowledge Bases	Fully managed; handles ingestion, chunking, embedding, indexing, retrieval

Advanced Search Architectures (Skill 1.5.4)

Search Type	Description	When to Use
Semantic search	Embed query → vector similarity search	When keyword matching fails; synonyms, paraphrasing
Keyword search (BM25/lexical)	Exact term matching, TF-IDF scoring	Exact product codes, names, IDs
Hybrid search	Combine semantic + keyword scores (Reciprocal Rank Fusion or custom scoring)	Most production RAG systems; best overall relevance
Reranking	Post-retrieval model scores candidates by relevance to original query	When top-K results need reordering for quality

Bedrock reranker models: after initial retrieval, send retrieved chunks + original query to a reranker model in Bedrock. The reranker scores and re-orders results before passing context to the FM. This is a two-stage retrieval pattern.

OpenSearch hybrid search combines lexical (BM25) and semantic (k-NN) scores. Conditional scoring logic, parallel query processing, and post-filtering are supported.

Query Handling Systems (Skill 1.5.5)

Advanced techniques to improve what gets retrieved:

Technique	Implementation
Query expansion	Bedrock: prompt an FM to generate synonyms/alternate phrasings; re-run search with expanded terms
Query decomposition	Lambda: split a complex multi-part question into atomic sub-queries; retrieve for each; merge results
Query transformation	Step Functions: orchestrate multi-step query refinement (expand → search → evaluate → refine)

Technique	Implementation
HyDE (Hypothetical Document Embedding)	Generate a hypothetical answer, embed it, search with the answer embedding instead of the question embedding

Access Mechanisms (Skill 1.5.6)

Standardize how FMs and agents access vector search:

- **Function calling / tool use** — define a `search_knowledge_base` tool in the FM's tool specification; the FM calls it autonomously when it needs retrieved context.
- **Model Context Protocol (MCP)** — open protocol for agent-tool integration; MCP clients provide standardized access to vector stores, APIs, databases; lightweight Lambda MCP servers for simple tools, ECS for complex ones.
- **Standardized API patterns** — wrap vector search behind a consistent internal API (API Gateway + Lambda); consumers never interact with the vector store directly.

Task 1.6 — Implement Prompt Engineering Strategies and Governance

Key Concepts

Model Instruction Frameworks (Skill 1.6.1)

Prompts have four components:

1. **Instruction** — what the model must do.
2. **Context** — background information, retrieved documents, system persona.
3. **Demonstration examples (few-shot)** — input/output examples that calibrate behavior.
4. **Input text** — the actual user query or data to process.

System prompts establish the model's role, tone, constraints, and output format. Enforce these via:

- **Bedrock Prompt Management** — store role definitions as versioned, reusable prompt templates.
- **Bedrock Guardrails** — enforce Responsible AI policies (denied topics, content filters, PII redaction) independently of the prompt.
- **Template configurations** — use variables (`{{user_input}}`, `{{context}}`) for reusable parameterized prompts.

Interactive AI Systems / Conversation Management (Skill 1.6.2)

Managing multi-turn context:

Service	Role
Amazon DynamoDB	Store conversation history (session ID → message history); retrieve and prepend on each turn
Amazon Comprehend	Detect user intent, extract entities, classify domain before invoking the FM
Step Functions	Implement clarification workflows: ask follow-up questions when intent is ambiguous; route to correct handler based on intent

Prompt Management and Governance (Skill 1.6.3)

Amazon Bedrock Prompt Management:

- Create, version, test, and deploy parameterized prompt templates.
- Support approval workflows before a new prompt version goes to production.
- Share prompts across teams with IAM-controlled access.
- Use the Prompt builder console UI for iterative testing.

Governance supporting services:

Service	Governance Role
Amazon S3	Central repository for prompt template files and version history
AWS CloudTrail	Audit log of all Bedrock API calls (who invoked which prompt, which model, when)
Amazon CloudWatch Logs	Log every prompt/response pair for debugging and compliance review
AWS IAM	Control who can create, modify, or deploy prompt versions

Quality Assurance for Prompts (Skill 1.6.4)

- **Lambda functions** — verify that model output matches expected format (JSON schema validation, regex checks).
- **Step Functions** — orchestrate test suites that exercise edge cases; assert on outputs.
- **CloudWatch alarms** — monitor regression metrics (e.g., failure rate, unexpected output rate) over time; alert on degradation after a prompt change.
- **A/B testing** — route a percentage of traffic to a new prompt variant via Bedrock Prompt Flows or AppConfig feature flags; compare metrics.

Advanced Prompt Engineering Techniques (Skill 1.6.5)

Technique	Description
Zero-shot	No examples; relies on instruction clarity alone
Few-shot (in-context learning)	Provide 2-5 input/output examples in the prompt
Chain-of-thought (CoT)	Instruct the model to reason step-by-step before answering; improves accuracy on complex reasoning
Structured input/output	Define exact input structure and JSON output schema; use tool use for guaranteed JSON
Critique prompting	Ask the model to evaluate and improve its own initial response
Self-consistency	Generate multiple responses with different random seeds; vote on the majority answer
Prompt compression	Remove redundant context to reduce token usage and cost
Context pruning	Trim conversation history to the most recent/relevant turns
Feedback loops	Collect user ratings; use low-rated examples as negative training data or prompt refinement signals

Complex Prompt Systems (Skill 1.6.6)

Amazon Bedrock Prompt Flows — visual no-code/low-code builder for multi-step prompt orchestration:

- **Sequential prompt chains** — output of prompt A becomes input to prompt B.
- **Conditional branching** — route to different prompt nodes based on model output classification.
- **Reusable prompt components** — reference Bedrock Prompt Management templates as nodes in a flow.
- **Pre/post processing** — add Lambda nodes before or after FM invocations for data

transformation or validation.

Use cases for prompt flows: document processing pipelines, intent classification → response generation chains, multi-step summarization, automated compliance checks before returning responses.

Important Services Reference for Domain 1

Amazon Bedrock

Feature	Description
InvokeModel API	Synchronous model invocation; model-specific request body
Converse API	Model-agnostic conversation abstraction; handles system prompts, tool use, and multi-turn history
Streaming APIs	InvokeModelWithResponseStream; returns tokens incrementally for better UX
Cross-Region Inference	Geographic (data residency) or Global (max throughput) inference profile routing
Provisioned Throughput	Reserved capacity for predictable high-volume workloads; eliminates throttling
Knowledge Bases	Fully managed RAG: ingestion, chunking, embedding, vector store, retrieval, augmented generation
Prompt Management	Versioned, parameterized prompt templates with approval workflows
Prompt Flows	Visual multi-step prompt orchestration with conditional branching
Guardrails	Input/output content filtering, PII protection, denied topics, contextual grounding, automated reasoning checks
Model Evaluation	Automatic (LLM-as-a-Judge) and human evaluation; supports RAG evaluation
Bedrock Data Automation	Converts multimodal content (video, audio, images, documents) to structured text for ingestion

Amazon OpenSearch Service

- Supports `knn_vector` field type for dense vector storage (up to 16,000 dimensions).
- HNSW (Hierarchical Navigable Small World) and IVF algorithms for approximate nearest-neighbor.
- Distance metrics: cosine similarity, Euclidean distance, dot product.
- Hybrid search: combines BM25 (lexical) + k-NN (semantic) with Reciprocal Rank Fusion.
- Neural search plugin integrates with Bedrock embedding models for pipeline-based indexing.
- Serverless or managed cluster deployment.
- Sharding and replication for horizontal scalability.

Amazon SageMaker AI

Feature	Role in Domain 1
SageMaker Processing	Run batch data transformation and validation jobs
SageMaker Data Wrangler	Visual data preparation, profiling, and feature engineering
SageMaker Model Registry	Catalog, version, approve, and deploy customized FM models
SageMaker Pipelines	Automated MLOps workflows for fine-tuning → evaluation → deployment

Feature	Role in Domain 1
Adapter Inference Components	Deploy LoRA adapters alongside base models for parameter-efficient serving

AWS Glue

- **Glue Data Quality (DQDL)** — rule-based data validation; ML anomaly detection; runs within

Glue ETL jobs; integrates with EventBridge for alerts.

- **Glue Data Catalog** — metadata repository for data lineage and source attribution.
- **Glue ETL** — serverless Spark for large-scale data transformation before FM ingestion.

Supporting Services

Service	Domain 1 Use Case
AWS Step Functions	Orchestrate multi-step workflows: circuit breakers, clarification flows, query transformation, prompt testing
AWS AppConfig	Store and dynamically retrieve model IDs and feature flags without code deployment
Amazon Comprehend	Entity extraction, intent detection, PII identification, sentiment analysis
Amazon Transcribe	Audio-to-text conversion for multimodal data pipelines
AWS Lambda	Custom validation, data normalization, batch embedding generation, query decomposition
Amazon DynamoDB	Conversation history storage; metadata storage alongside vector stores
AWS CloudTrail	Audit logging for all Bedrock API calls
Amazon CloudWatch	Metrics, alarms, logs for prompt regression testing and data quality monitoring

---

Exam Patterns and Common Distractors

Pattern: "How do you switch models without code changes?"

Answer: AWS AppConfig to store the model ID; Lambda reads it at runtime. API Gateway can also route to different model endpoints via request transformations.

Pattern: "How do you ensure continuous operation when a Bedrock model is unavailable?"

Answer: Cross-Region inference profiles (geographic or global) for regional failover, plus Step Functions circuit breaker for service-level failures, plus graceful degradation to a fallback model.

Pattern: "Which chunking strategy improves both precision and context quality?"

Answer: Hierarchical chunking. Child chunks for precision retrieval; parent chunks returned to the FM for broader context.

Pattern: "How do you validate data quality before FM ingestion?"

Answer: AWS Glue Data Quality with DQDL rules, supplemented by custom Lambda functions for business logic validation, and CloudWatch metrics for ongoing monitoring.

Pattern: "How do you improve RAG retrieval relevance beyond basic semantic search?"

Answer: Hybrid search (semantic + keyword with RRF scoring) in OpenSearch, combined with Bedrock reranker models for post-retrieval reordering.

Pattern: "How do you govern and audit prompt usage?"

Answer: Bedrock Prompt Management for versioning and approval workflows; S3 for template repositories; CloudTrail for API-level audit; CloudWatch Logs for request/response logging.

Pattern: "What is the parameter-efficient customization technique?"

Answer: LoRA (Low-Rank Adaptation). It adds small trainable adapter layers to the base model; much cheaper than full fine-tuning.

Pattern: "How do you handle multi-modal data in a RAG pipeline?"

Answer: Amazon Transcribe for audio, Rekognition/Nova multimodal embeddings for images and video, Bedrock Data Automation for document and multimedia parsing, then ingest into Bedrock Knowledge Bases with a multimodal embedding model (Amazon Nova Multimodal Embeddings).

Pattern: "What is the correct approach to store and retrieve conversation history?"

Answer: DynamoDB keyed by session ID; retrieve history and prepend to the `messages` array on each Converse API call.

Quick-Reference: AWS Well-Architected Generative AI Lens

Six lifecycle phases the lens covers:

1. Scoping — define business problem, success metrics, use case fit for GenAI.
2. Model Selection — evaluate FMs using the four-dimension framework.
3. Customization — fine-tuning, continued pretraining, LoRA adapters.
4. Integration — RAG, APIs, enterprise system connectivity.
5. Deployment — on-demand, provisioned throughput, SageMaker endpoints.
6. Iteration — continuous evaluation, drift monitoring, prompt optimization.

Best practice IDs to know:

- `GENOPS03-BP01` — Implement prompt template management (Bedrock Prompt Management + Flows).
- `GENSEC04-BP01` — Implement a secure prompt catalog (versioned, IAM-controlled).

Quick-Reference: Retrieval-Augmented Generation (RAG) Pipeline

```
Source Data (S3, Confluence, SharePoint, Salesforce, Web)
    |
    v
Parsing (Bedrock Data Automation / Foundation Model parser / default)
    |
    v
Chunking (Fixed-size / Hierarchical / Semantic / No chunking)
    |
    v
Embedding (Titan Text V2 / Cohere Embed / Nova Multimodal Embeddings)
    |
    v
Vector Store (OpenSearch Serverless / Aurora pgvector / Neptune / Pinecone / S3 Vectors)
    |
    v
Retrieval (Semantic / Hybrid / Reranking)
    |
    v
Augmented Prompt (System prompt + retrieved chunks + user query)
    |
    v
Foundation Model (Bedrock InvokeModel / Converse API)
    |
    v
Response (optionally post-processed by Guardrails / Lambda)
```

Bedrock Knowledge Bases APIs:

- `RetrieveAndGenerate` — end-to-end RAG: retrieves context and generates a response.
- `Retrieve` — retrieval only; returns ranked chunks; you construct the FM call yourself.

Domain 1 Key Numbers

Item	Value
Domain weight	31%
Bedrock default chunk size	~300 tokens
OpenSearch max vector dimensions	16,000
Titan Text Embeddings V2 dimensions	256, 512, or 1024 (configurable)
Cross-region inference cost savings (Global)	~10% vs. geographic profiles
LLM-as-a-Judge response accuracy (Automated Reasoning)	Up to 99% for factual validation
Bedrock Model Evaluation max prompts per job	1,000

Domain 2: Implementation and Integration — Study Guide

Exam Weight: 26% Source: AWS Certified Generative AI Developer - Professional (AIP-C01)

This domain covers the practical implementation of GenAI solutions on AWS, including agentic systems, model deployment, enterprise integrations, API patterns, and developer tooling. It carries the second-highest exam weight and requires deep familiarity with how AWS services wire together in production GenAI architectures.

Task 2.1: Implement Agentic AI Solutions and Tool Integrations

What is Agentic AI?

An AI agent is an autonomous system that uses a foundation model (FM) as its reasoning engine, coordinates with external tools and APIs, maintains state, and executes multi-step workflows to accomplish a goal. Agentic AI unifies:

- Multi-agent system architecture (distributed AI, actor model)
- The cognitive loop: perceive → reason → act
- The generative power of LLMs
- Cloud-native and serverless compute flexibility

Core AWS Frameworks for Agents

Amazon Bedrock Agents

- Fully managed service for building goal-driven, tool-using agents
- Uses the ReAct (Reason + Act) framework internally: the agent breaks user input into logical

steps, selects tools (action groups), executes them, and synthesizes a response

- Supports multi-agent collaboration (supervisor + collaborator agents)
- Integrates with Knowledge Bases for RAG, Lambda action groups, code interpreter, and memory

Strands Agents (Open Source SDK)

- Open-sourced by AWS in May 2025; provider-independent and model-agnostic
- Model-first design philosophy: the LLM drives orchestration decisions, not rigid state machines
- Core agent = Model + Tools + System Prompt
- Supports multiple model providers: Amazon Bedrock, Anthropic, Ollama, Meta, OpenAI, Cohere
- Four multi-agent coordination patterns:
  - **Agents-as-Tools**: an agent wraps another agent as a callable tool, creating hierarchical delegation
  - **Swarms**: self-organizing peer agents that hand off tasks to each other autonomously
  - **Graphs**: directed workflows with conditional routing and deterministic execution paths
  - **Meta Agents**: a single agent with tools that dynamically create and orchestrate other agents
- Built-in OpenTelemetry (OTEL) instrumentation for observability
- Deploy via Lambda (serverless), ECS/EKS (containers), or Amazon Bedrock AgentCore Runtime

AWS Agent Squad

- AWS-native orchestration for multi-agent systems
- Works alongside Strands Agents for coordinating agent ensembles
- Used in Task 2.5.5 for advanced GenAI orchestration

Amazon Bedrock AgentCore

- Platform for building, deploying, and managing agents securely at scale using any framework/model
- Modular services: Runtime, Memory, Gateway, Identity, Code Interpreter, Browser, Observability, Evaluations, Policy
- AgentCore Runtime: serverless hosting for agents; framework-agnostic (supports LangGraph, Strands, CrewAI, custom); each session runs in a dedicated microVM for isolation
- Supports long-running workloads (up to 8 hours), bidirectional streaming (WebSocket), and 100 MB payloads
- AgentCore Gateway: converts APIs and services into MCP-compatible tools
- Consumption-based pricing with no upfront commitments

Model Context Protocol (MCP)

MCP is an open standard from Anthropic that enables LLMs to interact with external tools, environments, and memory in a standardized way.

- Defines how agents discover and call tools (MCP servers)
- Enterprise-ready features (March 2025): OAuth 2.1 security, expanded resource types, Streamable HTTP transport
- AWS joined the MCP steering committee in May 2025
- **MCP Server hosting options:**
  - AWS Lambda: stateless MCP servers for lightweight, low-latency tool access (skill 2.1.7)
  - Amazon ECS: MCP servers for complex, long-running, or stateful tools (skill 2.1.7)
- **MCP Client libraries**: provide consistent, standardized access patterns for agents to call MCP servers
- Amazon Q Developer, Cursor, and other AI coding assistants integrated MCP for standardized development workflows

ReAct Pattern and Chain-of-Thought Reasoning

ReAct (Reason + Act) is the core agent reasoning loop:

1. Receive user request
2. Reason about what action to take (chain-of-thought)
3. Select and invoke a tool
4. Observe the result
5. Repeat until the goal is achieved

AWS Implementation:

- AWS Step Functions implements ReAct patterns as state machines
- Each state in Step Functions can represent a reasoning/acting step
- Lambda functions handle individual tool calls within the workflow
- Amazon Bedrock's custom orchestration Lambda gives full control over the ReAct loop

Safeguarded AI Workflows (Guardrails on Agent Behavior)

Controlling agent behavior in production requires multiple safety mechanisms:

Control	AWS Service	Purpose
Stopping conditions	Step Functions	Halt workflow after max iterations or time
Timeout mechanisms	Lambda	Force-stop long-running tool calls
Resource boundaries	IAM policies	Prevent agents from accessing unauthorized resources
Circuit breakers	Step Functions / custom	Stop cascading failures from tool errors
Content safety	Bedrock Guardrails	Block harmful inputs/outputs at model layer

Circuit Breaker Pattern: When a tool fails repeatedly, a circuit breaker "opens" and stops calling the failing service, returning a fallback response instead of allowing cascading failures.

Model Coordination and Ensembles

- **Specialized FMs for subtasks**: Route different parts of a complex task to the FM best suited for that step (e.g., a coding model for code generation, a summarization model for document synthesis)
- **Custom aggregation logic for ensembles**: Multiple models independently process the same input; outputs are aggregated (e.g., majority vote, weighted average)
- **Model selection frameworks**: Dynamically pick the right FM based on task type, expected token count, latency requirements, or cost budget

Human-in-the-Loop (HITL)

Amazon Bedrock Agents offers two HITL patterns:

- **User Confirmation**: Agent presents an action for binary (yes/no) approval before executing
- **Return of Control (ROC)**: Agent prepares the action but hands control back to the application; the user can review and modify parameters before the application executes

AWS Services for HITL orchestration:

- AWS Step Functions: orchestrate review and approval workflows with wait states and task tokens
- API Gateway: collect user feedback via webhooks or REST endpoints
- Amazon Augmented AI (A2I): managed human review workflows for ML predictions (confidence thresholds trigger human review automatically for built-in task types)
- Amazon SageMaker Ground Truth: labeled data via private, vendor, or Mechanical Turk workforces

Tool Integrations

- **Strands API**: define custom tool behaviors with standardized function signatures; tools are Python functions decorated with `@tool`
- **Standardized function definitions**: JSON schema descriptions of tool inputs/outputs that the FM uses to decide when and how to call each tool
- **Lambda for tool hosting**: handles error catching, parameter validation, and response formatting before returning results to the agent loop

---

Task 2.2: Implement Model Deployment Strategies

Deployment Options Comparison

Deployment Mode	Service	Best For	Key Characteristics
On-demand (serverless)	Amazon Bedrock	Variable/unpredictable traffic	Pay-per-token; no pre-provisioning
Provisioned Throughput	Amazon Bedrock	Consistent/high-volume traffic	Fixed hourly cost; guaranteed Model Units (MUs)
Custom endpoint	SageMaker AI	Fine-tuned or custom models	Full control; GPU/CPU instance selection
Serverless invocation	Lambda → Bedrock	Event-driven, low-frequency	No idle costs; cold-start considerations
Hybrid	Bedrock + SageMaker	Complex routing needs	Mix on-demand + provisioned

Amazon Bedrock Provisioned Throughput

- Purchase **Model Units (MUs)** that guarantee a fixed throughput level (input + output tokens per minute)
- Commitment options: no-commitment (hourly), 1-month, 6-month (increasing discounts)
- Supports base models and custom (fine-tuned) models
- Required for: consistent low-latency applications, high-throughput batch processing, production SLAs that require guaranteed capacity
- Use the provisioned model ARN as the `modelId` in InvokeModel / Converse API calls

LLM-Specific Deployment Challenges

LLMs differ from traditional ML models in several critical ways:

- **Memory requirements**: LLMs require large amounts of GPU VRAM; container images must be sized accordingly
- **GPU utilization patterns**: token generation is inherently sequential (autoregressive); batching strategies must account for variable-length sequences
- **Token processing capacity**: measured in tokens per minute (TPM), not requests per minute (RPM)
- **Model loading strategies**: lazy loading, weight streaming, model sharding across multiple GPUs
- **Container-based deployment patterns**: containers must be optimized for GPU drivers, CUDA versions, and model-specific inference servers (TGI, vLLM, Triton)

Optimized Deployment Approaches

- **Right-sizing**: match model size to task complexity; avoid using a 100B-parameter model for simple classification tasks
- **Smaller pre-trained models**: distilled or smaller models (e.g., Haiku vs. Opus, Nova Micro vs. Nova Pro) for routine queries that do not require maximum capability
- **API-based model cascading**: attempt the simplest/cheapest model first; escalate to a more capable model only if quality is insufficient
- **Cross-Region Inference (CRIS)**: Amazon Bedrock automatically routes to other regions within

a geography (US, EU, APAC) for higher throughput and resilience during regional capacity constraints

---

Task 2.3: Design and Implement Enterprise Integration Architectures

Enterprise Connectivity Patterns

API-based integrations with legacy systems:

- REST or SOAP adapters via API Gateway or Lambda functions
- Transform legacy data formats to JSON before sending to Bedrock APIs
- Use SQS as a buffer to handle throughput mismatches between legacy systems and Bedrock limits

Event-driven architectures (loose coupling):

- Amazon EventBridge: route events from enterprise applications to Lambda functions that invoke FMs
- Amazon SQS: decouple producers from FM consumers; enables retries and dead-letter queues (DLQs)
- Amazon SNS: fan-out pattern for broadcasting AI-generated content to multiple downstream consumers
- DynamoDB Streams / Kinesis: trigger FM processing in response to real-time data changes

Data synchronization patterns:

- AWS AppFlow: managed data sync between SaaS apps (Salesforce, ServiceNow) and AWS
- AWS DataSync: sync on-premises or cross-cloud file data to S3 for FM consumption
- AWS Glue: ETL pipelines to prepare and synchronize structured data for knowledge bases

Enhancing Existing Applications with GenAI

- **API Gateway + Lambda**: add a GenAI endpoint to an existing microservice architecture without modifying the core application
- **Lambda webhook handlers**: receive events from third-party systems (Slack, Jira, Salesforce) and trigger FM processing
- **EventBridge event-driven integrations**: applications emit events; EventBridge rules route them to FM-powered processors asynchronously

Amazon Q Business for internal knowledge tools:

- Pre-built connectors to 40+ data sources (S3, SharePoint, Salesforce, Confluence, Jira)
- Built-in semantic search across enterprise data
- Fine-grained access control via IAM Identity Center
- Amazon Q Apps: lightweight purpose-built apps employees create from Q Business conversations

Secure Access Frameworks

- **Identity federation**: federate enterprise identity providers (Okta, Azure AD) with AWS IAM Identity Center; use SAML 2.0 or OIDC for single sign-on to FM services
- **Role-based access control (RBAC)**: separate IAM roles for read-only RAG access vs. full model invocation vs. fine-tuning operations
- **Least privilege API access**: scope IAM policies to specific Bedrock model ARNs, specific actions (bedrock:InvokeModel, bedrock:Retrieve), and specific resources

- **VPC endpoints (PrivateLink)**: keep FM API traffic on the AWS network; prevents data from traversing the public internet
- **Amazon Cognito**: user pool authentication for customer-facing GenAI applications

Cross-Environment AI Solutions (Hybrid and Edge)

AWS Outposts:

- Brings AWS infrastructure (compute, storage, networking) into on-premises data centers
- Use case: organizations with strict data residency or sovereignty requirements who must keep sensitive data on-premises while still using FM APIs
- Supports EC2, ECS, EKS, Lambda, and RDS on-premises with the same AWS APIs
- Ideal for regulated industries (healthcare, financial services, government)

AWS Wavelength:

- AWS infrastructure embedded inside telecom provider 5G networks
- Enables ultra-low latency GenAI inference at the mobile network edge (sub-10ms)
- Use cases: real-time voice AI, AR/VR with AI assistance, connected vehicle AI

Secure routing patterns:

- AWS Direct Connect: dedicated network link from on-premises to AWS (bypasses public internet)
- Site-to-Site VPN: encrypted tunnel for secure cloud-to-on-premises communication
- AWS PrivateLink: private connectivity between VPCs and AWS services without public IPs

CI/CD Pipelines for GenAI

A GenAI-specific CI/CD pipeline must handle both code changes AND model/prompt changes.

Standard pipeline stages with GenAI additions:

1. **Source**: CodeCommit, GitHub, or CodeStar Connections
2. **Build**: CodeBuild compiles code, runs linting, unit tests
3. **Security scan**: CodeBuild with SAST tools; scan for prompt injection vulnerabilities, hardcoded credentials, and IAM over-permissions
4. **GenAI-specific tests**: automated prompt regression tests; validate FM responses against golden datasets; check hallucination rates
5. **Deploy (dev/staging)**: CodeDeploy or CDK deploy with canary/blue-green strategies
6. **Approval gate**: manual or automated approval based on GenAI evaluation metrics
7. **Deploy (prod)**: rollback support; automated rollback triggered by CloudWatch alarms

AWS Services:

- CodePipeline: orchestrates the end-to-end pipeline
- CodeBuild: executes build, test, and scan stages
- CodeDeploy: manages deployment strategies (blue/green, canary, rolling)
- AWS CDK: infrastructure as code for pipeline and GenAI resources
- CloudFormation: deploy and version infrastructure stacks

GenAI Gateway Architecture:

- A centralized abstraction layer that sits between applications and FM APIs
- Provides: rate limiting, cost allocation, observability, request logging, model routing, security policy enforcement, and A/B testing

- Implemented using: API Gateway (entry point) → Lambda (business logic / routing) → Bedrock
- Enables model swapping without changing downstream application code

---

Task 2.4: Implement Foundation Model API Integrations

Bedrock API Families

API Family	Use Case	Endpoint
Invoke API	Single transactions; large payloads; direct model access	bedrock-runtime
Converse API	Multi-turn chat; unified interface across all models	bedrock-runtime
Responses API	Stateful conversations; built-in tool use; OpenAI-compatible	bedrock-mantle
Chat Completions	OpenAI migration; stateless multi-turn	bedrock-mantle
StartAsyncInvoke	Long-running batch inference; video/image generation	bedrock-runtime

Key distinction: The Converse API provides a normalized request/response format that works across all Bedrock-supported models — you can switch models without changing application code. The InvokeModel API requires model-specific JSON payloads.

Synchronous vs. Asynchronous Patterns

Synchronous (InvokeModel / Converse):

- Application sends request and blocks until the full response is returned
- Suitable for: interactive chatbots, low-latency queries
- Risk: timeouts for long-running generation tasks

Streaming (InvokeModelWithResponseStream / ConverseStream):

- Response is returned as a sequence of events (token chunks) as the model generates them
- Reduces perceived latency significantly; user sees text appear progressively
- Implementation options:
  - **API Gateway WebSocket API**: bidirectional real-time connection (persistent)
  - **Server-Sent Events (SSE)**: unidirectional server-to-client streaming over HTTP
  - **API Gateway chunked transfer encoding**: for REST-based streaming responses
- Bedrock streaming event types: messageStart, contentBlockStart, contentBlockDelta, contentBlockStop, messageStop, metadata

Asynchronous (SQS + Lambda):

- Application sends request to SQS; Lambda polls the queue and calls Bedrock; result stored in DynamoDB or S3; application polls or subscribes for results
- Suitable for: batch processing, long-running tasks, fire-and-forget workflows
- Provides built-in retry, DLQ support, and decoupling

StartAsyncInvoke:

- Submit model invocation and receive an `invocationArn` immediately

- Poll for results or receive notification when complete
- Used for: video generation (Nova Reel), large document processing, image generation

Resilient FM Systems

Throttling and Error Handling:

Amazon Bedrock returns two primary error types that require resilience handling:

- **429 ThrottlingException**: account quota exceeded (RPM or TPM limits)
  - Mitigation: exponential backoff with jitter, request rate limiting, quota increase requests
- **503 ServiceUnavailableException**: temporary service unavailability
  - Mitigation: exponential backoff, circuit breaker pattern, cross-region failover

Exponential Backoff with Jitter (AWS SDK built-in):

- On each retry, wait `min(cap, base * 2^attempt) + random(0, jitter)`
- Jitter prevents "thundering herd" problems where all retries hit simultaneously
- AWS SDKs include built-in retry logic; configure `max_attempts` and `retry_mode`

Circuit Breaker Pattern:

- Monitor failure rates for FM API calls
- When failures exceed a threshold, "open" the circuit: stop calling the failing endpoint

and return a fallback response immediately

- After a cool-down period, allow a single test request; if it succeeds, "close" the circuit

Graceful Degradation:

- If primary model is unavailable, fall back to a simpler/smaller model
- If all FM APIs are unavailable, return a cached response or a static fallback message
- Use Amazon Bedrock Cross-Region Inference for automatic regional failover

AWS X-Ray for Observability:

- Distributed tracing across Lambda → API Gateway → Bedrock invocations
- Identifies latency hotspots and error rates across service boundaries
- Bedrock supports X-Ray active tracing on Lambda functions invoking it

Intelligent Model Routing

Organizations often need to route requests to different models based on context:

Static Routing:

- Configuration-based rules in application code or AWS AppConfig
- Example: route all SQL generation requests to model A; route all creative writing to model B
- Simple, predictable, low overhead

Dynamic Content-Based Routing:

- Step Functions state machine evaluates request properties to select the target model
- Lambda function classifies input (using a lightweight classifier model or rule engine)
- API Gateway request transformations apply routing headers based on request parameters

Intelligent Prompt Routing (Amazon Bedrock feature):

- Routes requests within a model family (e.g., Haiku vs. Sonnet) based on predicted response

quality for each specific prompt

- Uses advanced prompt matching and model understanding to predict which model will perform best

- Can reduce costs by up to 30% without sacrificing accuracy
- Supports: Anthropic Claude family, Meta Llama family, Amazon Nova family
- Each request is fully traceable for debugging
- Default prompt routers: pre-configured by Bedrock; ready to use out of the box
- Configured prompt routers: user-defined routing criteria and fallback model

Metric-Based Routing:

- CloudWatch metrics drive routing decisions (e.g., route to cheaper model when cost budget is exceeded for the hour)

Task 2.5: Implement Application Integration Patterns and Development Tools

GenAI-Specific API Gateway Patterns

API Gateway requires special configuration for GenAI workloads:

- **Streaming responses**: configure integration response timeout to 29 seconds maximum; use WebSocket APIs for truly real-time streaming; use chunked transfer encoding for HTTP streaming
- **Token limit management**: validate request payload sizes at the API Gateway level; return 413 errors before the request reaches Lambda/Bedrock for oversized requests
- **Retry strategies**: implement retry logic in the client or use SQS for automatic retries; do not retry at the API Gateway level for non-idempotent FM calls
- **Request validation**: API Gateway request validators enforce required parameters and JSON schema compliance before forwarding to Lambda

Accessible AI Interfaces

AWS Amplify AI Kit:

- Frontend SDK for building AI-powered web/mobile apps in TypeScript
- Pre-built `<AIConversation>` component: handles real-time subscriptions, multi-turn chat history, UI state management
- `conversation` API: maintains persistent conversation history tied to user accounts
- Connects to Amazon Bedrock models via Amplify backend
- Supports React, Next.js, Vue, Angular, React Native, Flutter, Swift, Android

OpenAPI Specifications for API-First Development:

- Define GenAI API contracts in OpenAPI 3.0 before implementation
- API Gateway imports OpenAPI specs to auto-generate REST APIs
- Enables SDK generation for client teams; establishes clear interface boundaries
- Supports versioning strategies for prompt/model changes

Amazon Bedrock Prompt Flows (No-Code Builder):

- Visual drag-and-drop builder in the AWS Console and Bedrock Studio
- Connect prompts, FMs, Lambda functions, knowledge bases, and other nodes visually
- Supports: conditional branching, loops, parallel execution, code hooks

- Version flows for rollback and A/B testing
- Deploy flows via aliases that point to immutable versions
- Use cases: multi-step document processing, sequential prompt chains, automated workflows

Business System Enhancements

CRM Enhancements (Lambda):

- Lambda functions triggered by CRM events (Salesforce webhooks) invoke Bedrock to generate personalized responses, summaries, or recommendations
- Results written back to CRM via API calls within the Lambda function

Document Processing (Step Functions):

- Step Functions orchestrate multi-step document workflows:
  1. S3 upload → EventBridge trigger → Step Functions execution
  2. Textract / Bedrock Data Automation extracts structured data
  3. Confidence score evaluation: high-confidence → store; low-confidence → A2I human review
  4. Validated data stored in downstream systems

Amazon Bedrock Data Automation:

- Extracts insights from unstructured multimodal content (documents, images, audio, video)
- Returns structured output with confidence scores and visual grounding
- Use: invoice processing, claims processing, contract analysis, media metadata extraction
- Invoke via `InvokeDataAutomationAsync` API with a project ARN
- Projects group standard and custom output configurations; support LIVE and DEVELOPMENT stages

Amazon Q Business for Internal Knowledge Tools:

- Data sources: S3, SharePoint, Confluence, Salesforce, Jira, ServiceNow (40+ connectors)
- Built-in semantic retrieval (Agentic RAG), conversation history, access controls
- Topic controls: restrict responses to specific domains; block sensitive topics
- Amazon Q Apps: no-code apps built from Q Business conversations; automate repetitive tasks

Developer Productivity

Amazon Q Developer:

- AI-powered assistant for developers embedded in IDEs (VS Code, JetBrains, CLI)
- Capabilities:
  - Code generation and completion (inline suggestions)
  - Code explanation and documentation
  - Unit test generation
  - Bug finding and fixing
  - Code transformation (e.g., Java version upgrades)
  - Security vulnerability scanning
  - API call suggestions for AWS services
  - GenAI-specific error pattern recognition (skill 2.5.6)
- Integrated MCP support for standardized tool access in development workflows
- CloudWatch Logs Insights queries: Q Developer can help write and explain complex queries for analyzing FM prompt/response logs

Advanced GenAI Application Patterns

Prompt Chaining (Amazon Bedrock):

- The output of one FM call becomes the input to the next
- Enables complex, multi-step reasoning that exceeds a single context window
- Implemented via: Step Functions (orchestration), Lambda (glue code), Bedrock Prompt Flows

Agent Design Patterns (Step Functions):

- Step Functions orchestrates agent loops as state machines
- States: Choose (routing), Task (tool call), Wait (human review), Parallel (multi-model), Map (batch)
- Error handling with Catch and Retry blocks

Strands and AWS Agent Squad for Orchestration:

- Strands: model-driven, framework-level orchestration; agents define their own tool use sequences
- AWS Agent Squad: higher-level multi-agent coordination; manage agent pools and delegation
- Combined: Strands for individual agent logic; Agent Squad for fleet-level coordination

Troubleshooting GenAI Applications

Key Tools:

Problem	Tool	Approach
Poor response quality	CloudWatch Logs Insights	Query stored prompt/response pairs; identify failure patterns
Latency issues	AWS X-Ray	Trace end-to-end; identify which service boundary adds the most latency
Error pattern recognition	Amazon Q Developer	Ask Q Developer to analyze error logs and suggest fixes
Prompt regressions	Bedrock Prompt Management	Version prompts; compare A/B test results
Tool call failures	CloudWatch + X-Ray	Trace Lambda invocations from agent action groups

CloudWatch Logs Insights queries for GenAI:

- Filter by model ID to analyze per-model performance
- Parse JSON log fields for inputTokenCount, outputTokenCount, latency
- Identify high-latency invocations with `stats avg(latency) by modelId`

Bedrock Model Invocation Logs:

- Enable on the Bedrock console; logs all request/response data to CloudWatch and/or S3
- Critical for: debugging hallucinations, auditing prompt content, cost allocation by use case

---

Key Services Quick Reference for Domain 2

Agentic AI

Service	Role
Amazon Bedrock Agents	Managed ReAct-based agent platform

Service	Role
Amazon Bedrock AgentCore	Deploy/host agents at scale; modular services
Strands Agents (SDK)	Open-source model-driven agent framework
AWS Agent Squad	Multi-agent coordination and fleet management
MCP (protocol)	Standardized agent-tool communication
AWS Step Functions	Orchestrate agent loops and multi-step workflows
AWS Lambda	Host tools, MCP servers (stateless), action groups
Amazon ECS	Host MCP servers (complex/stateful tools)
Amazon Augmented AI	Human review workflows for agent decisions

Model Deployment

Service	Role
Amazon Bedrock (on-demand)	Serverless, pay-per-token FM inference
Amazon Bedrock Provisioned Throughput	Guaranteed capacity via Model Units
Amazon SageMaker AI endpoints	Custom model hosting; full infrastructure control
Amazon Bedrock Cross-Region Inference	Automatic regional failover and load balancing

Enterprise Integration

Service	Role
Amazon API Gateway	Entry point for GenAI APIs; request validation; routing
AWS Lambda	Glue code; webhook handlers; tool execution
Amazon EventBridge	Event-driven routing from enterprise apps to GenAI
Amazon SQS	Async decoupling; retry and DLQ for FM calls
Amazon SNS	Fan-out distribution of AI-generated content
AWS AppFlow	SaaS-to-AWS data synchronization
AWS Outposts	On-premises data residency with AWS APIs
AWS Wavelength	5G edge deployments for ultra-low latency
AWS PrivateLink / VPC Endpoints	Private FM API access without internet exposure
AWS CodePipeline / CodeBuild	CI/CD for GenAI applications
Amazon Q Business	Enterprise knowledge assistant with 40+ connectors

API and Application Patterns

Service	Role
Amazon Bedrock Converse API	Unified multi-model chat interface
Amazon Bedrock Invoke API	Direct model access with full payload control

Service	Role
Amazon Bedrock Prompt Flows	Visual no-code workflow builder for FM pipelines
Amazon Bedrock Data Automation	Structured extraction from multimodal content
AWS Amplify AI Kit	Frontend components for AI-powered apps
Amazon Q Developer	AI coding assistant; error analysis; code generation
AWS X-Ray	Distributed tracing for GenAI API call chains
Amazon CloudWatch Logs Insights	Query and analyze FM prompt/response logs

---

Common Exam Scenarios and Correct Answers

Q: A chatbot must stream responses token-by-token to the user. What is the correct approach? Use Amazon Bedrock's ConverseStream or InvokeModelWithResponseStream API. Front it with API Gateway using a WebSocket API for bidirectional real-time communication, or use server-sent events (SSE) for one-way streaming.

Q: An FM API call is failing intermittently with 429 errors. How should the application handle this? Implement exponential backoff with jitter using the AWS SDK retry configuration. Consider purchasing Provisioned Throughput if the workload is consistent, or use Amazon Bedrock Intelligent Prompt Routing to shift load to smaller models during peak periods.

Q: A company needs to run GenAI inference on data that cannot leave their on-premises data center. What architecture should they use? AWS Outposts brings AWS infrastructure (including EC2 for self-hosted model inference, or EKS/ECS for containerized models) on-premises. For managed FM access, use Direct Connect to reach Bedrock APIs over a private network, combined with VPC endpoints and IAM policies that restrict access to specific data.

Q: An organization wants to route simple queries to a cheap model and complex queries to a powerful model. What is the best approach? Use Amazon Bedrock Intelligent Prompt Routing (for within-family routing with up to 30% cost savings) or implement a custom Step Functions state machine that classifies the query and routes to the appropriate model endpoint.

Q: A developer wants to add a GenAI chat interface to an existing React web application quickly. What is the fastest approach? Use the AWS Amplify AI Kit with the pre-built `<AIConversation>` component. It handles real-time subscriptions, conversation history, and UI state management out of the box with minimal configuration.

Q: An enterprise needs to build a CI/CD pipeline for a GenAI application that includes prompt testing and security scanning. Which services are involved? CodePipeline (orchestration), CodeBuild (build, test, security scan), CodeDeploy or CDK (deployment), CloudWatch alarms (rollback triggers), and automated prompt regression tests against golden datasets as a quality gate stage.

Q: An agent must stop after 10 tool calls to prevent runaway execution. How is this implemented? Use Step Functions with a counter in the state machine context. Each iteration increments the counter; a Choice state checks if the counter exceeds 10 and routes to a termination state if so. Lambda functions can implement timeout mechanisms for individual tool calls.

Q: An application needs to call three specialized agents in parallel and merge their results. What architecture achieves this? AWS Step Functions Parallel state: fan out to three Lambda functions (each calling a specialized Bedrock agent), then aggregate results in a subsequent state using custom aggregation logic. This matches the model coordination / ensemble pattern from skill 2.1.4.

Q: A company wants to give their employees a chat interface over internal SharePoint, Confluence, and Salesforce data. What is the fastest path? Amazon Q Business with pre-built connectors for SharePoint,

Confluence, and Salesforce. Configure IAM Identity Center for SSO. Use Amazon Q Apps for task-specific automations.

Important Notes and Exam Tips

- **Strands Agents** is the preferred AWS-native open-source agent SDK as of 2025. Expect questions about its model-first approach, MCP integration, and deployment on Lambda/ECS/AgentCore.
- **MCP = Model Context Protocol**: Always used for standardizing agent-tool communication. Lambda hosts stateless MCP servers; ECS hosts complex/long-running MCP servers.
- **Bedrock Converse API** is the preferred unified API for multi-turn conversations. It normalizes request/response format across all Bedrock models.
- **Provisioned Throughput** = guaranteed capacity via Model Units (MUs). Required for predictable high-volume workloads. Billed hourly even if not used.
- **Intelligent Prompt Routing** routes within a model family (e.g., Haiku ↔ Sonnet) based on predicted per-prompt quality; can reduce cost up to 30%.
- **Step Functions** is the orchestration backbone for agentic workflows, CI/CD gates, document processing, and human-in-the-loop patterns.
- **AWS Outposts** = on-premises data residency. **AWS Wavelength** = 5G/telecom edge. Know when to use each based on the data sovereignty vs. latency constraint.
- **Bedrock Data Automation** processes multimodal content (documents, images, audio, video) into structured output with confidence scores. Pairs with SageMaker A2I for human review of low-confidence extractions.
- **Amazon Q Business** targets enterprise employees (internal knowledge assistant). **Amazon Q Developer** targets software developers (code generation, debugging). Know which is correct for each scenario.
- **GenAI Gateway pattern**: centralized API Gateway + Lambda layer that provides rate limiting, routing, observability, and security policy enforcement across all FM API calls.
- **Exponential backoff with jitter** is the correct answer for handling 429 and 503 errors from Bedrock. AWS SDKs have built-in retry logic — configure `max_attempts` and `retry_mode`.
- **ReAct pattern** = the agent reasoning loop. Bedrock Agents uses ReAct internally; Step Functions implements custom ReAct loops for non-managed agents.
- **Human-in-the-Loop**: Step Functions task tokens enable pause-and-wait-for-human patterns; Amazon A2I provides built-in UI and workforce management for review tasks.
- For **streaming real-time responses**, the answer is almost always ConverseStream/InvokeModelWithResponseStream + WebSocket API Gateway or SSE.
- **Bedrock Prompt Flows** = visual no-code workflow builder for chaining prompts and AWS services. Versions are immutable snapshots; deploy via aliases.

Domain 3: AI Safety, Security, and Governance

Exam Weight: 20% — the third-largest domain on the exam

This domain covers four task areas: implementing input/output safety controls, data security and privacy controls, AI governance and compliance mechanisms, and responsible AI principles. Bedrock Guardrails is the single most important service across all four tasks.

Task 3.1 — Implement Input and Output Safety Controls

Key Concepts

Content Safety Systems for Harmful Inputs (Skill 3.1.1)

Amazon Bedrock Guardrails is the primary content safety mechanism on the exam. A guardrail is a combination of multiple policies that evaluate both user inputs and model responses.

Core architectural facts:

- An account can have multiple guardrails, each configured differently per use case.
- A guardrail requires at least one filter policy and blocked messaging for both prompts and responses (`blockedInputMessaging` and `blockedOutputsMessaging`).
- Guardrails work with any text or image Foundation Model on Bedrock, plus external/self-hosted models via the `ApplyGuardrail` API.
- Guardrails integrate natively with Bedrock Agents, Knowledge Bases, and Prompt Flows.
- Input evaluation happens in parallel across all configured policies for low latency.
- If the input is blocked, the Foundation Model is never invoked (no inference charge, only the guardrail evaluation charge).
- If the output is blocked, you are charged for both guardrail evaluation and model inference.

Seven guardrail policies:

#	Policy	What It Does	Applies To
1	Content Filters	Detects harmful content across 6 categories: Hate, Insults, Sexual, Violence, Misconduct, Prompt Attack	Input + Output
2	Denied Topics	Blocks user-defined undesirable topics (up to 30 per guardrail)	Input + Output
3	Word Filters	Exact-match blocking of custom words/phrases and built-in profanity list	Input + Output
4	Sensitive Information Filters	Detects/blocks/masks PII using ML-based detection + custom regex	Input + Output
5	Contextual Grounding Checks	Detects hallucinations based on grounding source and query relevance	Output only

#	Policy	What It Does	Applies To
6	Automated Reasoning Checks	Validates responses against logical rules using formal mathematical reasoning	Output only
7	Prompt Attacks (sub-category of Content Filters)	Detects jailbreaks, prompt injection, prompt leakage	Input only

Content filter strength levels (configured independently per category for input and output):

Filter Strength	Blocks Content Classified At	Allows Content Classified At
NONE	Nothing	NONE, LOW, MEDIUM, HIGH
LOW	HIGH confidence only	NONE, LOW, MEDIUM
MEDIUM	HIGH + MEDIUM	NONE, LOW
HIGH	HIGH + MEDIUM + LOW	NONE only

Content is classified with a confidence level (NONE/LOW/MEDIUM/HIGH). The filter strength determines the threshold for blocking. A higher strength means more aggressive filtering.

Safeguard tiers:

- **Classic Tier** — Original behavior. Supports English/French/Spanish. Optimized for lower latency.
- **Standard Tier** — 30% better recall for content filters, 32% better recall for denied topics.

Supports 60+ languages. Includes typo detection. Distinguishes between jailbreaks and prompt injection. Extends protection to code elements (comments, variable names, function names, string literals). Requires cross-Region inference opt-in.

Denied topics configuration:

- Up to 30 denied topics per guardrail.
- Each has a name (max 100 chars), a definition (max 200 chars), and optional sample phrases

(up to 5 phrases, max 100 chars each).

- Can independently enable/disable for input and output with separate actions (BLOCK or NONE).
- Best practice: use crisp definitions, not instructions. Do not use negative definitions. Do not

use denied topics for entity or word capture (use word filters or PII filters instead).

Word filters:

- Custom words/phrases for exact-match blocking (up to 10,000 items, up to 3 words each).
- Built-in profanity managed word list (toggle on/off).
- Can upload from .txt/.csv or S3 object.

Guardrail versioning:

- Creating a guardrail produces a working DRAFT.
- Create numbered, immutable versions via `CreateGuardrailVersion`.
- Versions do not have their own ARN. IAM policies on a guardrail apply to all its versions.
- Specify `guardrailVersion` as `"DRAFT"` or a numeric string (e.g., `"1"`, `"2"`).

The `ApplyGuardrail` API evaluates content against a guardrail without invoking any Foundation Model:

```
POST /guardrail/{guardrailIdentifier}/version/{guardrailVersion}/apply
```

- `source`: `"INPUT"` for user content or `"OUTPUT"` for model output.
- Returns `action` (`GUARDRAIL_INTERVENED` or `NONE`), `outputs`, and detailed `assessments`.
- Use cases: pre-retrieval validation in RAG, evaluate non-Bedrock model outputs, insert guardrail

checks at any point in custom application flow.

Input tagging for selective evaluation — when using `InvokeModel` / `InvokeModelWithResponseStream`, you must wrap user content with XML tags to enable prompt attack detection:

```
&lt;amazon-bedrock-guardrails-guardContent_xyz&gt;
  user input here
&lt;/amazon-bedrock-guardrails-guardContent_xyz&gt;
```

The tag suffix is specified in `guardrailConfig.tagSuffix`. Without tags, prompt attacks in user input will not be filtered. The Converse API uses `guardContent` blocks with qualifiers instead.

Step Functions and Lambda extend guardrails with custom moderation workflows: pre-check with Comprehend, invoke model with guardrails, post-process with Lambda, with branching logic, retries, and human review escalation.

IAM enforcement: use the `bedrock:GuardrailIdentifier` condition key in IAM policies to mandate specific guardrails for all inference calls, preventing bypass.

> ***Exam tip:*** *If a question asks how to prevent Foundation Model invocation without content safety checks, the answer involves IAM policies with* `bedrock:GuardrailIdentifier` *condition keys combined with explicit deny statements.*

Content Safety Frameworks for Harmful Outputs (Skill 3.1.2)

Input vs. output evaluation differences:

- Input blocked: Foundation Model never invoked. Returns `blockedInputMessaging`.
- Output blocked: Foundation Model already generated the response. Response replaced with `blockedOutputsMessaging` or masked.
- Prompt Attack filter is INPUT-ONLY — it has only `inputStrength`, no `outputStrength`.
- All other content filter categories (Hate, Insults, Sexual, Violence, Misconduct) can have different strengths for input vs. output.
- Contextual Grounding and Automated Reasoning checks are OUTPUT-ONLY.

Bedrock Model Evaluations for content moderation:

- **Automatic evaluations** — predefined metrics: accuracy, robustness, toxicity. Use curated or custom datasets.
- **LLM-as-a-Judge** — a second Foundation Model evaluates the first model's responses on correctness, completeness, harmfulness, answer refusal. Up to 98% cost savings vs. human evaluation.
- **Human evaluations** — custom metrics (relevance, style, safety) via internal employees or AWS-managed workers.
- **RAG evaluations** — context relevance, answer correctness, faithfulness for Knowledge Bases.

Text-to-SQL for deterministic results — Bedrock Knowledge Bases supports structured data stores with natural language to SQL conversion:

- Connects to Amazon Redshift as the query engine.
- Converts natural language questions into SQL queries deterministically.
- Three APIs: `RetrieveAndGenerate` (query + retrieve + generate), `Retrieve` (query + retrieve), `GenerateQuery` (natural language to SQL only).
- Provides deterministic retrieval from structured data, reducing hallucination risk.

Accuracy Verification to Reduce Hallucinations (Skill 3.1.3)

Bedrock Knowledge Bases provide RAG-based grounding:

- User query is embedded, semantic search retrieves relevant documents, retrieved context is injected into the Foundation Model prompt, Foundation Model generates a grounded response.
- Citations are automatically included in responses for verification.
- Supported vector stores: OpenSearch Serverless, Aurora PostgreSQL, Pinecone, Redis Enterprise Cloud, MongoDB Atlas, Neptune Analytics.

Contextual Grounding Checks in Guardrails — the primary hallucination detection mechanism:

Two check types:

1. **Grounding Check** — Is the response factually accurate based on the source? New information not in the source is considered ungrounded.
2. **Relevance Check** — Is the response relevant to the user's query? A factually correct response to the wrong question is irrelevant.

Configuration:

- Each check produces a score from 0 to 1 (higher = more grounded/relevant).
- Configure thresholds between 0 and 0.99.
- If the score falls below the threshold, the response is flagged as a hallucination.
- Action can be BLOCK or NONE (detect only).
- Character limits: grounding source max 100,000 chars, query max 1,000 chars, response max 5,000 chars.

Three required components:

1. **Grounding source** — reference/source material (retrieved documents).
2. **Query** — the user's question.
3. **Content to guard** — the model's response.

How to provide these depends on the API:

- Invoke APIs: use XML tags (`<amazon-bedrock-guardrails-groundingSource_xyz>` and `<amazon-bedrock-guardrails-query_xyz>`).
- Converse APIs: use `guardContent` blocks with qualifiers `["grounding_source"]` and `["query"]`.
- ApplyGuardrail API: provide content blocks with `guard_content` qualifier.

Grounding check scenario examples (exam-relevant):

Source	Query	Response	Grounding	Relevance
"Tokyo is capital of Japan"	"Capital of Japan?"	"Capital of Japan is Tokyo"	Grounded	Relevant
"Tokyo is capital of Japan"	"Capital of Japan?"	"Capital of Japan is London"	NOT grounded	Relevant
"London is capital of UK. Tokyo is capital of Japan."	"Capital of Japan?"	"Capital of UK is London"	Grounded	NOT relevant
Same source	"Capital of Japan?"	"It is raining outside"	NOT grounded	NOT relevant

Structured outputs (JSON Schema) via Converse API:

- Constrains model responses to follow a user-defined JSON schema, eliminating parsing errors.
- Specify schema in `outputConfig.textFormat` (Converse API).
- First-time schema compilation can take up to a few minutes. Compiled grammars cached 24 hours.
- Helps reduce hallucinations by constraining output format for downstream processing.

Automated Reasoning Checks:

- Uses formal logic (mathematical proofs) to verify response accuracy.
- Identifies correct responses with up to 99% accuracy.
- Provides provable explanations for why information is accurate or inaccurate.
- Operates in detect mode; the application decides how to act on findings.

> ***Exam tip:*** *For "reduce hallucinations" questions: Bedrock Knowledge Bases (RAG grounding) + Contextual Grounding Checks in Guardrails is the primary answer. JSON Schema structured outputs is the answer for "ensure deterministic output format." Automated Reasoning is the answer for "verify logical/policy compliance with provable explanations."*

Defense-in-Depth Safety Systems (Skill 3.1.4)

AWS recommends a multi-layered approach. Each layer addresses different threat vectors:

Layer 1 — Pre-Processing (Application Level):

- Amazon Comprehend Trust and Safety: Toxicity Detection (`DetectToxicContent` API, 7 categories:

sexual harassment, hate speech, threat, abuse, profanity, insult, graphic), Prompt Safety Classifier (safe/unsafe binary classification).

- Application-level input sanitization in Lambda: remove unusual Unicode, encoded content

(base64, hex, ROT13, Morse code), control characters.

- AWS WAF for API-level input validation, rate limiting, IP filtering.

Layer 2 — Model Level (Bedrock Guardrails):

- All seven filter types: content filters, denied topics, word filters, sensitive information

filters, prompt attack detection, contextual grounding, automated reasoning.

Layer 3 — Post-Processing:

- Lambda functions for custom validation, filtering, or transformation of model responses.
- Additional Comprehend toxicity checks on output.
- Business logic validation.

Layer 4 — API Level:

- API Gateway for rate limiting, throttling, authentication, authorization.
- IAM policies to enforce mandatory guardrails.
- CloudTrail for audit logging.
- CloudWatch for monitoring and anomaly detection.

The `ApplyGuardrail` API enables guardrail evaluation decoupled from model invocation. Insert at any point: before retrieval in RAG, after retrieval but before generation, after generation from non-Bedrock models, or between agent steps.

Advanced Threat Detection (Skill 3.1.5)

Three types of prompt attacks detected by Bedrock Guardrails:

1. **Jailbreaks** — bypass native safety capabilities (e.g., "Do Anything Now" prompts).
2. **Prompt Injection** — override developer instructions (e.g., "Ignore everything earlier").
3. **Prompt Leakage** (Standard tier only) — extract system prompts or confidential configuration.

Prompt Attack filter configuration:

- Part of `contentPolicyConfig` with type `PROMPT_ATTACK`.
- Has only `inputStrength` — no `outputStrength`. Prompt attacks are INPUT-ONLY detection.
- Action can be BLOCK or NONE (detect only).

Input tagging is critical for prompt attack detection when using `InvokeModel` / `InvokeModelWithResponseStream`. Without tags, prompt attacks will not be filtered. The Converse API handles this via message roles.

Input sanitization strategies (defense-in-depth):

1. Application-level: remove unnecessary Unicode, unusual patterns, encoded content.
2. AWS WAF: block known attack patterns at the API gateway level.
3. Prompt templates: structure prompts with designated slots for user variables.
4. System prompts: constrain model behavior with clear boundaries.
5. Format breakers: convert text through multiple formats to strip hidden malicious content.

Safety classifiers available:

- Bedrock Guardrails content filters (ML-based, 6 categories).
- Comprehend Toxicity Detection (7-category classifier).
- Comprehend Prompt Safety Classifier (binary safe/unsafe).
- Custom classifiers via SageMaker or fine-tuned Foundation Models deployed as Lambda functions.

Automated adversarial testing:

- Use tools like `promptfoo` to test prompt injection, jailbreaking, adversarial inputs.
- Test categories: role confusion, context manipulation, privilege escalation, multi-turn attacks, context poisoning, cross-agent prompt propagation.
- Include both benign scenarios and malicious attack vectors in test suites.
- Use Bedrock Evaluations with toxicity metrics and LLM-as-a-Judge with harmfulness metrics.

Monitoring:

- All blocked content appears as plain text in Model Invocation Logs (disable if unwanted).
- CloudWatch metrics: monitor `GuardrailPolicyType` for masking/blocking events.
- Guardrail tracing: enable via `trace` field (`"enabled"`, `"disabled"`, `"enabled_full"`).

> ***Exam tip:*** *For "protect against prompt injection" questions, the answer is Bedrock Guardrails Prompt Attack content filter (Standard tier for best detection). For "defense in depth," describe the four-layer architecture: Comprehend pre-processing, Guardrails at model level, Lambda post-processing, API Gateway at the API level.*

Task 3.2 — Implement Data Security and Privacy Controls

Key Concepts

Protected AI Environments (Skill 3.2.1)

VPC endpoints for network isolation:

- AWS PrivateLink creates a private connection between your VPC and Bedrock. Traffic never traverses the public internet.
- Creates an endpoint network interface (ENI) in each subnet you enable.
- No internet gateway, NAT device, VPN, or Direct Connect required.

Bedrock VPC endpoint service names:

Category	Service Name
Control Plane APIs	`com.amazonaws.region.bedrock`

Category	Service Name
Runtime APIs (InvokeModel, Converse)	`com.amazonaws.region.bedrock-runtime`
Agents Build-time APIs	`com.amazonaws.region.bedrock-agent`
Agents Runtime APIs	`com.amazonaws.region.bedrock-agent-runtime`

Private DNS: when enabled, API requests use the default Regional DNS name (e.g., `bedrock-runtime.us-east-1.amazonaws.com`). Existing SDK code works without modification.

VPC endpoint policies: attach a resource-based IAM policy to restrict which principals, actions, and resources are accessible through the endpoint. Default policy allows full access.

When using a VPC with no internet access for model customization, create an S3 VPC gateway endpoint so jobs can access training/validation data.

IAM policies for Bedrock:

Feature	Supported?
Identity-based policies	Yes
Resource-based policies	No
Policy condition keys	Yes
ABAC (tags in policies)	Yes
Temporary credentials	Yes
Service roles	Yes
Service-linked roles	No

Key IAM actions: `bedrock:InvokeModel`, `bedrock:InvokeModelWithResponseStream`, `bedrock:CreateModelCustomizationJob`, `bedrock:CreateModelInvocationJob`, `bedrock:CreateGuardrail`, `bedrock:ApplyGuardrail`.

AWS managed policies: `AmazonBedrockFullAccess` (full access), `AmazonBedrockReadOnly` (read-only).

AWS Lake Formation for granular data access — three levels:

- **Column-level** — restrict which columns users can see (e.g., hide PII columns).
- **Row-level** — restrict rows based on column values using PartiQL `WHERE` clause syntax.
- **Cell-level** — combines row + column filtering for maximum granularity.

Implementation uses data filters created via `CreateDataCellsFilter` API. Filters apply only to the `SELECT` Lake Formation permission. Integrates with Athena, Redshift Spectrum, and EMR.

CloudWatch for data access monitoring:

- Model Invocation Logging: disabled by default, must be explicitly enabled.
- Collects invocation logs, model input data, and model output data.
- Destinations: CloudWatch Logs, S3, or both.
- APIs: `PutModelInvocationLoggingConfiguration`, `GetModelInvocationLoggingConfiguration`, `DeleteModelInvocationLoggingConfiguration`.
- CloudWatch Logs: JSON events with metadata + I/O up to 100 KB. Larger data goes to S3.
- Runtime metrics (automatic): `Invocations`, `InvocationLatency`, `InputTokenCount`, `OutputTokenCount`, `TimeToFirstToken`, `InvocationThrottles`.

> ***Exam tip:*** *Bedrock does NOT support resource-based policies. All access control uses identity-based policies. For "network isolation" questions, the answer is VPC interface endpoints powered by AWS PrivateLink. For "granular data access in a data lake," the answer is Lake Formation with data filters.*

Privacy-Preserving Systems (Skill 3.2.2)

Amazon Comprehend PII Detection:

- `DetectPiiEntities` — real-time, locates PII with type, confidence score, and character offsets.

Up to 100 KB per document. English and Spanish.

- `ContainsPiiEntities` — returns labels and scores of PII types found (no position info).
- Async batch jobs for PII detection and redaction. Redaction replaces PII with asterisks.
- 22 universal PII types: ADDRESS, AGE, AWS_ACCESS_KEY, AWS_SECRET_KEY,

CREDIT_DEBIT_CVV/EXPIRY/NUMBER, DATE_TIME, DRIVER_ID, EMAIL, IBAN, IP_ADDRESS, LICENSE_PLATE, MAC_ADDRESS, NAME, PASSWORD, PHONE, PIN, SWIFT_CODE, URL, USERNAME, VIN.

- 14 country-specific types for Canada, India, UK, and US (SSN, passport, health numbers, etc.).

Amazon Macie for PII Detection in S3:

- Uses ML and pattern matching to discover sensitive data in S3.
- Two discovery methods: automated sensitive data discovery (daily, breadth-first) and targeted

sensitive data discovery jobs.

- Managed data identifiers: built-in criteria for financial, personal, and credential data.
- Custom data identifiers: user-defined regex (up to 512 chars), optional keywords, proximity

rules, custom severity settings.

- Allow lists for exceptions.
- Findings integrated with EventBridge and Security Hub.

Bedrock native data privacy features:

- Bedrock never shares your data with model providers.
- Bedrock does not use customer data to train base Foundation Models.
- Fine-tuning creates a private copy of the model. Training data is not stored after job completion.
- Model improvement is opt-in only.
- Encryption in transit: TLS 1.2. Encryption at rest: AWS-owned keys by default, customer managed

KMS keys optional.

- Compliance: GDPR, HIPAA eligible, SOC 1/2/3, ISO 27001, CSA STAR Level 2, FedRAMP High.

Bedrock Guardrails sensitive information filters:

- ML-based, context-dependent PII detection (not just regex).
- Two handling modes: BLOCK (entire request/response blocked) or ANONYMIZE/MASK (PII replaced with

type identifier tags like `{NAME}`, `{EMAIL}`, `{US_SOCIAL_SECURITY_NUMBER}`).

- 30+ PII entity types across categories: General, Finance, IT, USA, Canada, UK.
- Custom regex patterns supported (does NOT support regex lookaround).
- Each PII type can individually be set to BLOCK or ANONYMIZE.
- Works on both input and output, independently configurable.
- Standard tier extends PII detection to code elements (comments, variable names, string literals,

hardcoded credentials).

S3 Lifecycle configurations for data retention:

- Automate data lifecycle for privacy compliance (GDPR right to erasure, data minimization).

- Transition actions (move to cheaper storage classes) and expiration actions (delete objects).
- Up to 1,000 rules per bucket. Filter by prefix, tags, or size.

KMS encryption for Bedrock:

- AWS-owned keys by default (no charge, no management).
- Customer managed keys optional for: custom models, agents, knowledge base data source ingestion, vector stores in OpenSearch, evaluation jobs, flows, guardrails.
- When using customer managed keys, Bedrock creates KMS grants (primary for long-lived resources, secondary for async operations).
- Key policy needs: `kms:CreateGrant`, `kms:DescribeKey`, `kms:GenerateDataKey`, `kms:Decrypt`.
- Use `kms:ViaService` condition to restrict key access to Bedrock only.

> ***Exam tip:*** *For "detect PII in real-time text" the answer is Amazon Comprehend. For "discover PII in S3 at rest" the answer is Amazon Macie. For "filter PII in Foundation Model prompts and responses" the answer is Bedrock Guardrails sensitive information filters. Know the difference between BLOCK (stops content) and ANONYMIZE (replaces PII with tags).*

Privacy-Focused AI Systems (Skill 3.2.3)

Data masking techniques:

- **Static masking** — applied to data at rest before use. Original data permanently replaced. Irreversible. Used for creating non-production datasets from production data.
- **Dynamic masking** — applied at query time based on user permissions. Original data preserved. Lake Formation cell/row/column security and Bedrock Guardrails ANONYMIZE mode are forms of dynamic masking.

Anonymization strategies:

Strategy	Description	Reversible?
Pseudonymization	Replace identifiers with artificial IDs. Mapping table allows re-identification.	Yes (with mapping)
Tokenization	Replace sensitive data with format-preserving tokens. Token vault maintains mapping.	Yes (with vault)
Generalization	Reduce data precision (exact age to age range, address to city).	No
k-Anonymity	Ensure each record is indistinguishable from at least k-1 others.	No

Bedrock data privacy guarantees summary:

1. Bedrock does NOT use inputs/outputs to train base Foundation Models.
2. Data is not shared with model providers (Anthropic, Meta, AI21, etc.).
3. Fine-tuning creates a private copy; training data not stored after completion.
4. All data encrypted in transit (TLS 1.2) and at rest (AWS-owned or CMK).
5. Private VPC connectivity via AWS PrivateLink.
6. Model improvement is opt-in only.
7. Fine-tuned models can replay training data — filter confidential data before training.

Service selection matrix:

Need	Service
PII detection in text (real-time)	Amazon Comprehend `DetectPiiEntities`

Need	Service
PII detection in S3 at rest	Amazon Macie
PII filtering in AI prompts/responses	Bedrock Guardrails sensitive information filters
Network isolation for Bedrock	VPC interface endpoints (PrivateLink)
Fine-grained data lake access	AWS Lake Formation data filters
Encryption key management	AWS KMS (customer managed keys)
Model invocation auditing	CloudWatch Logs + S3 + CloudTrail
Data lifecycle/retention	S3 Lifecycle configuration

Task 3.3 — Implement AI Governance and Compliance Mechanisms

Key Concepts

Compliance Frameworks (Skill 3.3.1)

SageMaker AI Model Cards — centralized documentation for capturing critical details about ML models for governance and reporting:

Key content sections (JSON schema):

Section	Fields	Purpose
`model_overview`	`model_description`, `model_creator`, `model_artifact` (max 15), `algorithm_type`, `problem_type`	High-level identification
`intended_uses`	`purpose_of_model`, `intended_uses`, `factors_affecting_model_efficiency`, `risk_rating`	Document appropriate/inappropriate uses
`business_details`	`business_problem`, `business_stakeholders`, `line_of_business`	Business context
`training_details`	`objective_function`, `training_observations`, `training_job_details` (ARN, datasets, metrics)	Reproducibility and audit trail
`evaluation_details`	`name` (required), datasets (max 10), `metric_groups`	Performance evidence

Risk ratings: `unknown`, `low`, `medium`, `high`. Maps directly to EU AI Act risk classification.

Versioning: any edit (other than approval status update) creates a new immutable version, ensuring a complete audit trail.

Export: model cards can be exported to PDF via `CreateModelCardExportJob`.

Integration: Model Cards integrate with SageMaker Model Registry. Can auto-populate evaluation metrics from Clarify and Model Monitor reports stored in S3. Supports cross-account access.

Key APIs: `CreateModelCard`, `DescribeModelCard`, `ListModelCards`, `ListModelCardVersions`, `UpdateModelCard`, `DeleteModelCard`, `CreateModelCardExportJob`.

AWS Glue for data lineage:

- Glue Data Catalog: centralized metadata repository for data location, schema, and runtime metrics.

Organized into databases and tables.

- Crawlers automatically scan data sources, infer schemas, and populate the catalog.
- Data lineage records track transformations and operations performed on data.
- Column statistics (min/max, null counts, distinct values) without additional pipelines.
- Integrates with Lake Formation for fine-grained access control.

CloudWatch Logs for decision logs:

- Bedrock model invocation logs sent to CloudWatch Logs as JSON events.
- Each event contains invocation metadata plus I/O (up to 100 KB). Larger payloads go to S3.
- Use CloudWatch Logs Insights for querying. Create custom metric filters for alerting.

> ***Exam tip:*** *SageMaker Model Cards is the answer for "document Foundation Model limitations, intended uses, and risk ratings for regulatory compliance." Glue Data Catalog is the answer for "track data lineage and source metadata."*

Data Source Tracking for Traceability (Skill 3.3.2)

Glue Data Catalog for data source registration:

- Structure: databases, tables, crawlers, connections, schema registry.
- Crawlers use classifiers (built-in or custom) to recognize data formats and infer schemas.
- Supports S3, RDS, Redshift, DynamoDB, Kafka, MongoDB, JDBC-compatible sources.

Metadata tagging for source attribution:

- Bedrock Knowledge Bases: create `.metadata.json` files alongside S3 source documents with

key-value attribute pairs for metadata filtering during queries.

- S3 object tagging: up to 10 tags per object (keys up to 128 chars, values up to 256 chars).
- AWS Organizations Tag Policies enforce consistent tagging across accounts.

CloudTrail for audit logging:

- Bedrock management events logged by default (InvokeModel, Converse, all control plane ops).
- Bedrock data events must be explicitly configured via advanced event selectors.

Data events by resource type:

Resource Type	Events Logged
`AWS::Bedrock::AgentAlias`	`InvokeAgent`
`AWS::Bedrock::Model`	`InvokeModelWithBidirectionalStream`, `GetAsyncInvoke`, `StartAsyncInvoke`
`AWS::Bedrock::KnowledgeBase`	`Retrieve`, `RetrieveAndGenerate`
`AWS::Bedrock::FlowAlias`	`InvokeFlow`
`AWS::Bedrock::Guardrail`	Guardrail data events
`AWS::Bedrock::Prompt`	`RenderPrompt`

Each CloudTrail event includes: identity info (who), IP address, timestamp, request parameters, response elements, error codes.

Bedrock Knowledge Bases citation/attribution:

- `RetrieveAndGenerate` returns citations linking generated text to source documents.
- Each citation contains: `location` (position in source), `source` (identifier/URL),

`sourceContent` (referenced content), `title`.

- In the console, citations appear as footnotes expandable to source text and S3 links.
- Guardrails applied to `RetrieveAndGenerate` evaluate user input and generated response only — they do NOT evaluate the retrieved references.

> ***Exam tip:*** *For "audit trail of all Bedrock API calls" the answer is CloudTrail. For "track who invoked which model and when" the answer is CloudTrail management events. For "log full prompt/response content" the answer is Bedrock Model Invocation Logging (CloudWatch Logs + S3). For "cite sources in generated answers" the answer is Bedrock Knowledge Bases citations via* `RetrieveAndGenerate`*.*

Organizational Governance Systems (Skill 3.3.3)

Regulatory requirements and AWS support:

Framework	GenAI Relevance	AWS Support
GDPR	Right to explanation, data minimization, consent	Bedrock PII filters, regional data residency, encryption, VPC endpoints
HIPAA	PHI protection, BAA required	Bedrock is HIPAA-eligible, sensitive info filters, invocation logging
SOC 2	Audit controls, monitoring, access management	CloudTrail, CloudWatch, IAM, encryption
EU AI Act	Risk classification, transparency, human oversight	Model Cards risk ratings, Guardrails, model evaluation, A2I
ISO/IEC 42001:2023	AI management system certification	AWS was first major cloud provider to achieve this certification for Bedrock, Q Business, Textract, Transcribe
FedRAMP High	Federal government security	Bedrock has FedRAMP High authorization

AWS AI Service Cards:

- Published by AWS for their own managed AI services (Rekognition, Nova models, Titan, Comprehend, Transcribe, etc.).
- Content: intended use cases, limitations, responsible AI design choices, performance expectations, fairness/bias testing methodology, deployment best practices.
- Distinction: AI Service Cards are AWS-published and read-only. SageMaker Model Cards are customer-created for custom models and fully customizable.

AWS Service Catalog for approved GenAI products:

- Administrators create portfolios of approved products (CloudFormation or Terraform templates).
- End users can only launch products from portfolios they have access to.
- Apply launch constraints (deployment configurations) and template constraints (parameter values).
- Use TagOptions to enforce tagging on provisioned resources.
- AWS Organizations integration for cross-account portfolio sharing.

Service Control Policies (SCPs) for organizational guardrails:

- JSON policies attached to OUs or accounts that set maximum permission boundaries.
- GenAI governance examples: deny access to specific Bedrock models, deny `bedrock:InvokeModel` without a guardrail, restrict Bedrock to specific Regions, block model customization in certain accounts.
- SCPs are deny-only filters — they cannot grant permissions, only restrict.
- Combine IAM policies with `bedrock:GuardrailIdentifier` condition key to require guardrails on every invocation at the organizational level.

Continuous Monitoring and Governance Controls (Skill 3.3.4)

Bias drift monitoring with SageMaker Clarify:

1. Create a baseline: run a baselining job against training data.
2. Define allowed ranges for bias metrics (e.g., DPPL between -0.1 and 0.1).
3. Schedule monitoring via `create_monitoring_schedule()` on a `ModelBiasModelMonitor` instance.
4. Clarify computes bias metrics on live data during each monitoring window.
5. Uses Normal Bootstrap Interval method for statistical significance (avoids false alarms).
6. If the confidence interval does not overlap the allowed range, an alert fires via CloudWatch.

Bias metrics supported: DPPL (Difference in Positive Proportions in Predicted Labels), DI (Disparate Impact), DCAcc (Difference in Conditional Acceptance), DCR (Difference in Conditional Rejection).

SageMaker Model Monitor — four monitoring types:

Type	What It Monitors	Requires Ground Truth?
Data Quality	Statistical properties of input features (drift)	No
Model Quality	Prediction accuracy	Yes
Bias Drift	Fairness metrics over time	Yes
Feature Attribution Drift	SHAP value changes (NDCG score)	Yes

Feature Attribution Drift measures changes in feature importance using SHAP values. Uses NDCG (Normalized Discounted Cumulative Gain) to compare feature attribution rankings. Default alert threshold: NDCG < 0.90.

Automated alerting and remediation:

- CloudWatch Alarms on Bedrock runtime metrics (token counts, latency, errors, throttles).
- EventBridge Rules for Bedrock job state changes — trigger Lambda, Step Functions, or SNS.
- Remediation pattern: CloudWatch Alarm -> SNS -> Lambda -> (disable endpoint, retrain, notify).
- CloudWatch anomaly detection creates dynamic baselines for GenAI metrics.

Bedrock Model Invocation Logging:

- Disabled by default. Enable via console or `PutModelInvocationLoggingConfiguration`.
- Logs `Converse`, `ConverseStream`, `InvokeModel`, `InvokeModelWithResponseStream`.
- Choose which modalities to log: Text, Image, Embedding, Video.
- CloudWatch Logs: JSON events up to 100 KB. S3: gzipped JSON files for larger data.
- S3 bucket must be in same Region, ACLs disabled, bucket policy granting `s3:PutObject` to `bedrock.amazonaws.com`.
- Large data (>100 KB) or binary image outputs can ONLY go to S3, not CloudWatch Logs.

Token-level redaction:

- Bedrock Guardrails ANONYMIZE action replaces PII with tags (`{NAME}`, `{SSN}`).
- BLOCK removes content entirely and returns blocked messaging.
- All blocked content from guardrails appears as plain text in Model Invocation Logs.

Disable logging or use S3 with KMS encryption if this is a security concern.

Monitoring integration pattern:

```
Bedrock Invocation Logs -&gt; S3 -&gt; Glue Catalog -&gt; Athena -&gt; QuickSight
                        -&gt; CloudWatch Logs -&gt; Logs Insights -&gt; Dashboards
                        -&gt; Metric Filters -&gt; Alarms -&gt; SNS -&gt; Lambda
```

> ***Exam tip:*** *Data Quality monitoring does NOT need Ground Truth labels. Bias Drift and Model Quality monitoring DO require Ground Truth labels. For "blocked content appears in logs" — remember that guardrail-blocked content shows as plain text in invocation logs. Large payloads (>100 KB) can only go to S3.*

Task 3.4 — Implement Responsible AI Principles

Key Concepts

Transparent AI Systems (Skill 3.4.1)

Bedrock Agent tracing — the primary transparency mechanism for agent decision-making:

Seven trace types:

Trace Type	Purpose
PreProcessingTrace	Agent contextualizes and categorizes user input; determines if valid
OrchestrationTrace	Agent interprets input, invokes action groups, queries knowledge bases (the "reasoning")
PostProcessingTrace	Agent shapes orchestration output into user-facing response
CustomOrchestrationTrace	Details about custom orchestration steps
RoutingClassifierTrace	Routing classifier input/output for multi-agent collaboration
FailureTrace	Records the reason a step failed; only type WITHOUT a `ModelInvocationInput` object
GuardrailTrace	Actions taken by an attached Guardrail

How to enable:

- Console: choose "Show trace" in the agent test window.
- API: set `enableTrace` to `TRUE` in `InvokeAgent`. Disabled by default.

TSTALIASID: the test alias used with the DRAFT version for API-based testing. Must call `PrepareAgent` first to package the agent with latest working draft changes.

Each trace (except FailureTrace) contains a `ModelInvocationInput` object showing: the type (PRE_PROCESSING, ORCHESTRATION, ROUTING_CLASSIFIER, etc.), the actual prompt sent to the Foundation Model, the model used, inference configuration, and whether the prompt was DEFAULT or OVERRIDDEN.

Bedrock Prompt Flows tracing:

- Set `enableTrace` to `true` in `InvokeFlow`.
- Trace event types: FlowTraceNodeInputEvent, FlowTraceNodeOutputEvent,

FlowTraceConditionNodeResultEvent, FlowTraceNodeActionEvent, FlowTraceDependencyEvent.

Extended thinking / Chain-of-thought (Claude on Bedrock):

- Outputs internal `thinking` content blocks showing step-by-step reasoning before the final

`text` answer. Direct transparency mechanism for user-facing explanations.

- Supported models: Claude Opus 4.5, Opus 4, Sonnet 4, Sonnet 4.5, Haiku 4.5, Claude 3.7 Sonnet.
- Set `budget_tokens` in thinking configuration (minimum 1,024 tokens).
- Response includes `thinking` blocks followed by `text` blocks.
- Streaming required when `max_tokens` > 21,333.
- Adaptive thinking (Claude Opus 4.6, Sonnet 4.6): set `thinking.type` to `"adaptive"` instead of

fixed budget. Claude dynamically decides when and how much to think.

- NOT compatible with `temperature`, `top_p`, `top_k`, response pre-fill, or forced tool use.

Evidence presentation / source attribution:

- Bedrock Knowledge Bases `RetrieveAndGenerate` returns `citations` with each response.
- Each citation contains: `generatedResponsePart` (segment of response), `retrievedReferences` (source S3 URI, content excerpt, metadata).
- The `Retrieve` API returns raw source chunks for custom attribution handling.

> ***Exam tip:*** *For "show reasoning behind agent decisions" the answer is agent tracing with* `enableTrace: TRUE`*. Know that FailureTrace is the only trace type without ModelInvocationInput. TSTALIASID is the test alias. For "provide source attribution in generated answers" the answer is Knowledge Bases citations. For "show Foundation Model reasoning to users" the answer is extended thinking (Claude).*

Fairness Evaluations (Skill 3.4.2)

SageMaker Clarify for bias detection:

Pre-training bias metrics (model-agnostic, computed on raw data):

Metric	What It Measures
Class Imbalance (CI)	Imbalance in number of members between facet groups. Range: [-1, +1]
Difference in Proportions of Labels (DPL)	Imbalance of positive outcomes between facets. Range: [-1, +1]
Kullback-Leibler Divergence (KL)	How much outcome distributions of facets diverge
Jensen-Shannon Divergence (JS)	Symmetric measure of divergence between outcome distributions
Lp-norm (LP)	Distance between outcome distributions
Total Variation Distance (TVD)	Half the L1-norm distance between outcome distributions
Kolmogorov-Smirnov (KS)	Maximum divergence between outcomes in facet labels
Conditional Demographic Disparity (CDD)	Disproportionately fewer positive outcomes, conditioned on a group variable (addresses Simpson's Paradox)

Post-training bias metrics require model predictions and detect legal concepts like disparate impact.

Key terminology: facet = attribute column assessed for bias; facet value = specific values that might be favored/disfavored; positive label = favorable outcome.

SHAP (SHapley Additive exPlanations): computes feature attributions to explain which features contributed most to a prediction.

Production monitoring: SageMaker Model Monitor + Clarify detects bias drift and feature attribution drift over time.

Bedrock Model Evaluation for bias:

Built-in datasets for fairness evaluation:

Task	Dataset	Metric
Robustness (bias)	BOLD (Bias in Open-ended Language Generation)	Word Error Rate
Toxicity	RealToxicityPrompts	Toxicity score
Accuracy	TREX	Real World Knowledge score
Summarization	Gigaword	BERTScore
Q&A	BoolQ, NaturalQuestions, TriviaQA	NLP-F1

BOLD is the recommended dataset for measuring bias in open-ended generation tasks. All built-in datasets are downsampled to 100 prompts each. Custom datasets support up to 1,000 prompts per job.

LLM-as-a-Judge (GA in Bedrock Model Evaluation):

- Quality metrics: correctness, completeness, professional style and tone, coherence.
- Responsible AI metrics: harmfulness, answer refusal.
- Supports Bring Your Own Inference Responses (BYOI) — evaluate any model hosted anywhere.
- Custom metrics via custom judge prompts.
- Can evaluate RAG systems for correctness, completeness, faithfulness, and responsible AI metrics.

Bedrock Prompt Management for A/B testing:

- Create multiple prompt variants — alternative configurations of the same prompt.
- Compare versions side-by-side (differences highlighted).
- Test different versions against the same inputs to measure output quality and bias.
- Create immutable versions for production deployment.

Designing prompts to reduce bias:

- Specify balanced, neutral perspectives in instructions.
- Instruct the model to consider multiple demographic groups.
- Use Chain-of-Thought to make reasoning explicit and auditable.
- Test prompts across demographic groups and compare outputs systematically.
- Use Prompt Management's variant comparison feature for A/B testing.

> ***Exam tip:*** *BOLD is the built-in bias evaluation dataset. SageMaker Clarify pre-training metrics are model-agnostic (data only); post-training metrics require predictions. LLM-as-a-Judge responsible AI metrics are harmfulness and answer refusal. Know the difference between pre-training and post-training bias metrics.*

Policy-Compliant AI Systems (Skill 3.4.3)

Mapping organizational policies to Bedrock Guardrails:

Policy Type	Guardrail Filter
Content policies	Content filters (set strength levels per category)
Prohibited topics	Denied topics (natural language definitions)
Language standards	Word filters (profanity + custom lists)
Privacy policies	Sensitive information filters (PII types to block/mask)
Accuracy requirements	Contextual grounding checks
Compliance rules	Automated reasoning checks

SageMaker Model Cards for documenting limitations:

- Risk rating field (`unknown`, `low`, `medium`, `high`) maps to EU AI Act risk classification.
- Intended uses section records appropriate and inappropriate use cases.
- Evaluation details section records performance metrics with datasets.
- Immutable versioning provides evidence for auditors.
- Can be exported to PDF for stakeholder communication.

Model Cards vs. AI Service Cards:

Aspect	SageMaker Model Cards	AWS AI Service Cards
Created by	You (for your models)	AWS (for their services)

Aspect	SageMaker Model Cards	AWS AI Service Cards
Purpose	Internal governance/documentation	External transparency to customers
Customizable	Yes, full JSON schema	No, read-only AWS documentation
Integrated with	SageMaker Model Registry	Published at docs.aws.amazon.com
Versioned	Yes, immutable versions	Updated by AWS

Lambda functions for automated compliance checks:

- Pre-process inputs before sending to models (validation, sanitization).
- Post-process outputs before returning to users (compliance scanning).
- Call `ApplyGuardrail` API within Lambda for custom pipeline guardrail checks.
- Log compliance events to CloudWatch/CloudTrail.
- Trigger alerts via SNS/EventBridge on violations.

Cross-Cutting Responsible AI Topics

AWS Responsible AI dimensions (eight pillars from the Well-Architected Generative AI Lens):

1. **Fairness** — not discriminating; considering impacts on different stakeholder groups.
2. **Explainability** — understanding and evaluating system outputs.
3. **Privacy and Security** — appropriately obtaining, using, and protecting data and models.
4. **Safety** — reducing harmful output and misuse; operating within defined scope.
5. **Controllability** — mechanisms to monitor and steer AI system behavior.
6. **Veracity and Robustness** — correct outputs even with unexpected/adversarial inputs.
7. **Governance** — best practices in the AI supply chain.
8. **Transparency** — enabling stakeholders to make informed choices about AI engagement.

Human in the loop — Amazon Augmented AI (A2I):

- Integrates human review into ML workflows.
- Built-in task types: Textract key-value extraction, Rekognition image moderation.
- Custom task types: any ML workflow via `StartHumanLoop` in the A2I Runtime API.
- Work teams: Mechanical Turk, vendor-managed, or private workforce.
- Conditions for review: confidence threshold below a specified value triggers human review.
- Results stored in S3; notifications via CloudWatch Events.
- GenAI integration: review low-confidence Foundation Model outputs, audit responses for quality

and bias, build feedback loops, validate RAG answers against sources.

Relationship between Guardrails, Model Cards, and AI Service Cards:

Tool	Scope	Type	When Used
Bedrock Guardrails	Runtime enforcement	Active controls	Applied at inference time to filter/block content
SageMaker Model Cards	Your models	Documentation	Throughout model lifecycle for governance
AWS AI Service Cards	AWS services	Transparency docs	Before/during service adoption for due diligence

Together they address: transparency (AI Service Cards + Model Cards), safety and controllability (Guardrails), and governance (all three).

Transparency requirements mapped to AWS services:

Requirement	AWS Service/Feature
Show reasoning behind FM decisions	Extended thinking (Claude), Agent tracing
Cite sources in generated answers	Bedrock Knowledge Bases citations
Document model limitations	SageMaker Model Cards, AI Service Cards
Track decisions across pipeline	Prompt Flows tracing, Agent tracing
Monitor model performance	CloudWatch, SageMaker Model Monitor
Detect guardrail-blocked content	GuardrailTrace, invocation logs
Explain individual predictions	SageMaker Clarify SHAP values
Invisible watermarking	Amazon Titan Image Generator watermarks

Bedrock data privacy commitments:

- Does NOT use customer data to train Foundation Models.
- Does NOT store or log prompts/completions (unless you enable invocation logging).
- Model providers have NO access to customer data.
- Mantle inference engine implements Zero Operator Access (ZOA) design.
- Encrypted in transit (TLS 1.2) and at rest (AWS-owned or customer managed KMS keys).

Exam tip: *Know the eight Responsible AI dimensions. For "human review of low-confidence outputs" the answer is Amazon A2I. For "document model limitations for regulatory compliance" the answer is SageMaker Model Cards. For "understand AWS service limitations before adoption" the answer is AWS AI Service Cards.*

Domain 3 Quick-Reference: Service-to-Task Mapping

Service	Primary Task(s)	Key Feature for This Domain
Amazon Bedrock Guardrails	3.1, 3.2, 3.3, 3.4	Content filters, denied topics, PII filters, grounding checks, prompt attack detection
Amazon Comprehend	3.1, 3.2	Toxicity detection, PII detection/redaction, prompt safety classifier
Amazon Macie	3.2	PII discovery in S3
AWS Lake Formation	3.2	Column/row/cell-level data access control
AWS KMS	3.2	Customer managed encryption keys for Bedrock resources
AWS PrivateLink / VPC Endpoints	3.2	Network isolation for Bedrock
AWS CloudTrail	3.3	Audit logging of all Bedrock API calls
CloudWatch / CloudWatch Logs	3.1, 3.3, 3.4	Model invocation logging, runtime metrics, alarms, dashboards
SageMaker Model Cards	3.3, 3.4	Model documentation, risk ratings, regulatory compliance
SageMaker Clarify	3.3, 3.4	Bias detection (pre/post-training), SHAP explanations, bias drift monitoring
SageMaker Model Monitor	3.3	Data quality, model quality, bias drift, feature attribution drift monitoring

Service	Primary Task(s)	Key Feature for This Domain
AWS Glue Data Catalog	3.3	Data source registration, lineage tracking, metadata management
Bedrock Knowledge Bases	3.1, 3.3, 3.4	RAG grounding, citations/attribution, text-to-SQL
Bedrock Model Evaluation	3.1, 3.4	Toxicity evaluation, bias evaluation (BOLD), LLM-as-a-Judge
Amazon A2I	3.4	Human-in-the-loop review workflows
AWS Service Catalog	3.3	Approved GenAI product portfolios with governance controls
SCPs (Organizations)	3.3	Maximum permission boundaries for Bedrock across accounts
API Gateway / WAF	3.1	Rate limiting, input validation, API-level security
AWS AI Service Cards	3.3, 3.4	AWS-published transparency docs for managed AI services

Domain 4: Operational Efficiency and Optimization for GenAI Applications

Exam Weight: 12%

This domain covers three task areas: cost optimization and resource efficiency, application performance optimization, and monitoring systems for GenAI applications. Despite being the lowest-weighted domain, it is dense with testable AWS-specific details around Bedrock pricing mechanics, caching features, scaling patterns, and observability tooling.

Task 4.1 -- Implement Cost Optimization and Resource Efficiency Strategies

Key Concepts

Token Efficiency Systems (Skill 4.1.1)

Amazon Bedrock charges per token (input and output). Every unnecessary token directly increases cost, making token efficiency the single most impactful lever for cost control.

Token Tracking and Monitoring

Bedrock publishes token metrics to CloudWatch under the `AWS/Bedrock` namespace:

Metric	Description
`InputTokenCount`	Tokens provided as input to the model
`OutputTokenCount`	Tokens generated by the model
`CacheReadInputTokens`	Tokens retrieved from prompt cache (not reprocessed)
`CacheWriteInputTokens`	Tokens written into the prompt cache
`EstimatedTPMQuotaUsage`	Quota consumption including cache write tokens and burndown

Pre-configured CloudWatch dashboards show invocation count, token counts by model, daily token trends, input vs. output distributions, and requests grouped by input token count.

Burndown Rate

For Anthropic Claude models v3.7 and later, output tokens have a **5x burndown rate** -- meaning 1 output token consumes 5 tokens from the TPM/TPD quota. All other models have a 1:1 rate.

Example: A request with 1,000 input tokens and 100 output tokens on Claude Sonnet 4 depletes **1,500 tokens** from quota (1,000 + 100 x 5), but you are only **billed** for 1,100 tokens. The quota deduction and billing amount differ.

The `max_tokens` Parameter

This parameter is deducted from quota at the **start** of each request, before any output is generated. Setting it too high (e.g., 32,000 when actual output averages 500) artificially reserves quota capacity, reducing your available concurrency. Unused tokens are replenished when the request completes, but during execution those reserved tokens count against your quota.

Optimization: use CloudWatch `OutputTokenCount` metrics to study actual output sizes, then set `max_tokens` to a value slightly above the observed maximum.

> ***Exam tip:*** *If a question describes throttling despite low actual token usage, the answer likely involves* `max_tokens` *being set too high or the burndown rate consuming quota faster than expected.*

Context Window Optimization

Each model has a finite context window (e.g., Claude 3.5 Sonnet: 200K tokens, Amazon Nova Pro: 300K tokens). Techniques to optimize context usage:

- Send only relevant context: for RAG, retrieve only the top-k most relevant chunks rather than stuffing the full knowledge base.
- Manage conversation history: implement sliding-window approaches that keep only the most recent N turns, or summarize older turns before including them.
- Use structured output instructions: tell the model exactly what format you need, reducing unnecessary back-and-forth.

Prompt Compression and Context Pruning

Ranked by impact:

1. **Amazon Bedrock Prompt Optimization** -- a managed feature that automatically optimizes prompts for a target LLM and task. Integrated into Bedrock Playground and Prompt Management.
2. **Manual prompt engineering** -- remove filler words, redundant instructions, and unnecessary context. The Well-Architected GenAI Lens (GENCOST03-BP01) recommends making prompts as short as possible while meeting performance requirements.
3. **LLM-based compression** -- use a cheaper model to summarize or compress a prompt before sending it to the primary model. Trades a small additional inference cost for significant savings on the main call.
4. **Prompting technique selection** -- zero-shot eliminates the token cost of examples entirely. Chain-of-thought and tree-of-thought add output tokens but improve reasoning. Choose the cheapest technique that meets accuracy requirements.
5. **Context pruning in RAG** -- limit retrieved chunks, use reranking to select only the most relevant results, strip metadata and formatting from documents before injection.

Response Size Controls

- Set `max_tokens` to the minimum needed for the use case.
- Use explicit instructions: "respond in 3 sentences" or "return only JSON."
- Provide an output schema that constrains response length.
- For streaming, implement client-side truncation when sufficient information is received.

Cost-Effective Model Selection Frameworks (Skill 4.1.2)

The fundamental principle: not every request needs the most capable (and expensive) model.

Amazon Bedrock Intelligent Prompt Routing

Provides a single serverless endpoint that dynamically routes each request to the most suitable model within a model family, predicting response quality per request.

Aspect	Detail
Supported families	Anthropic Claude (Haiku + Sonnet), Meta Llama (8B + 70B), Amazon Nova (Lite + Pro)
Cost savings	Up to 30% without compromising accuracy (internal testing showed 48% for Claude)
Router types	Default (pre-configured) and Configured (user-specified criteria and fallback model)
Limitations	English only, pairwise routing (two models), cannot adapt to app-specific metrics

Aspect	Detail
Pricing	No charge for routing itself; pay on-demand price for whichever model processes the request

Amazon Bedrock On-Demand Service Tiers

Four tiers map workload priority to cost:

Tier	Use Case	Key Characteristic
Priority	Mission-critical, customer-facing	Up to 25% better OTPS latency vs. Standard
Standard	General AI applications	Consistent performance baseline
Flex	Non-urgent: evaluations, batch-like workloads	Discounted rate, higher throttling risk
Reserved	Predictable, consistent throughput	1-month or 3-month commitments, guaranteed capacity

You select the tier on a per-API-call basis, allowing workload-level optimization.

Amazon Bedrock Model Distillation

Transfers knowledge from a larger "teacher" model to a smaller "student" model:

- Distilled models are up to **500% faster** and up to **75% less expensive** with less than 2% accuracy loss for use cases like RAG.
- Supported flows include Amazon (Nova Premier to Nova Pro), Anthropic (Sonnet to Haiku), and Meta (70B to 1B/3B).
- Input: JSONL file in S3 or historical invocation logs.
- Deployment: distilled model requires Provisioned Throughput for inference.

> ***Exam tip:*** *Model Distillation is the answer when a question asks "how to get a cheaper model with similar accuracy for my specific use case." Intelligent Prompt Routing is the answer when the question asks "how to route different requests to the right model automatically."*

SageMaker Inference Cost Optimization

For self-hosted models:

- **Inference Recommender** benchmarks your model across instance types for optimal price-performance.
- **SageMaker Savings Plans** offer up to 64% cost reduction for consistent usage.
- **Multi-model endpoints (inference components)** deploy multiple models on a single endpoint to improve GPU utilization.
- **Serverless inference** provides pay-per-invocation pricing for intermittent traffic.

High-Performance FM Systems for Throughput (Skill 4.1.3)

Bedrock Batch Inference

For large-scale, non-real-time workloads:

- Submit JSONL input files to S3 (up to 10,000 records, 200 MB max). Create a batch job. Results are written to an output S3 bucket asynchronously within a 24-hour window.
- **Up to 50% less** than on-demand pricing.
- Batch inference is **not supported for Provisioned Throughput** models.

- Monitoring: CloudWatch metrics in `AWS/Bedrock/Batch` namespace, EventBridge notifications for job state changes.

SageMaker Inference Options

Option	Best For	Payload Limit	Timeout	Scale to Zero
Real-time	Low latency, predictable traffic	25 MB	60s (8 min streaming)	No
Serverless	Intermittent/unpredictable traffic	4 MB	60s	Yes (automatic)
Asynchronous	Large payloads, long processing	1 GB	1 hour	Yes
Batch Transform	Offline processing, large datasets	No limit	Days	N/A (no endpoint)

Bedrock Provisioned Throughput

- Measured in **Model Units (MUs)**, each delivering a specific token throughput per minute.
- Commitment options: no commitment, 1-month, or 6-month (longer = cheaper).
- Use for high, predictable, consistent workloads that need guaranteed performance.
- Must request MU quota through AWS Support before purchasing.
- Specify the provisioned model ARN as `modelId` in API calls.

Cross-Region Inference

Routes requests across multiple regions within a geography for higher quotas:

- Geographic profiles (e.g., `us.`) route within a boundary, standard pricing.
- Global profiles (e.g., `global.`) route anywhere worldwide, ~10% cost savings, highest throughput.
- No additional cost for routing itself.

Auto-Scaling for SageMaker Endpoints

- **Target-tracking**: use `SageMakerVariantInvocationsPerInstance` as the predefined metric.
- **Step scaling**: define thresholds and scale amounts.
- **Scheduled scaling**: pre-scale for known traffic patterns.
- **Scale to zero**: supported for async endpoints using the `HasBacklogWithoutCapacity` CloudWatch alarm.
- Configure cooldown periods to prevent thrashing.

Intelligent Caching Systems (Skill 4.1.4)

There are two distinct caching layers, and the exam tests both.

Layer 1: Amazon Bedrock Prompt Caching (Model-Level)

Caches the internal model state for prompt prefixes, avoiding recomputation on subsequent calls.

How it works:

1. Mark portions of a prompt as **cache checkpoints**.
2. Everything before the checkpoint becomes the cached prompt prefix.
3. Subsequent requests with the same prefix skip recomputation.
4. The prefix must be **static** -- any change causes a cache miss.

Aspect	Detail
Cost reduction	Up to **90%** on input token costs
Latency improvement	Up to **85%** reduction
TTL	5-minute default (resets on each hit); some models support 1-hour extended TTL
Supported APIs	Converse, ConverseStream, InvokeModel, InvokeModelWithResponseStream
Supported models	Claude (Opus 4.5 through 3.5 Haiku), Amazon Nova (Micro through Premier)

Pricing mechanics:

- **Cache writes** are charged at a higher rate than standard input tokens.
- **Cache reads** are charged at a **reduced rate** (up to 90% less).
- Net savings depend on your cache hit rate.

Best use cases: chat with document (cache the document, vary the question), few-shot learning (cache system prompt + examples), coding assistants (cache code context), agentic workflows (cache tool definitions).

Layer 2: Semantic Caching (Application-Level)

Stores complete request-response pairs and **bypasses the LLM entirely** for matching queries.

Semantic Caching with ElastiCache for Valkey:

1. Convert the incoming query into a vector embedding.
2. Search the cache for semantically similar previous queries using vector similarity.
3. If a match exceeds the similarity threshold, return the cached response (no LLM call).
4. If no match, invoke the LLM, store the query embedding + response in the cache.

ElastiCache for Valkey is the AWS-recommended store due to lowest latency vector search with 95%+ recall. Supports HNSW and FLAT search algorithms, cosine/Euclidean/inner product distance.

Alternative stores:

- **Amazon MemoryDB for Redis** -- durable semantic caching (persists across restarts).
- **Amazon DynamoDB** -- persistent cache with flexible querying.
- **Amazon OpenSearch Service** -- hybrid full-text and vector search.

Deterministic Request Hashing

Hash the exact request (prompt + parameters) as a cache key. Look up in ElastiCache or DynamoDB. Suitable when queries repeat verbatim (automated pipelines, templated prompts). Simpler than semantic caching but misses paraphrased queries.

Edge Caching with CloudFront

Cache deterministic API responses at edge locations. Use CloudFront with API Gateway as origin. Appropriate for static/semi-static queries (product descriptions, FAQ responses). NOT suitable for personalized or conversational responses.

> ***Exam tip:*** *Bedrock prompt caching and semantic caching solve different problems. Prompt caching caches the model's internal computation state for repeated prefixes. Semantic caching stores full responses and skips the model entirely. If the question says "reduce LLM calls for similar queries," think semantic caching. If it says "reduce latency for repeated prompt context," think prompt caching.*

Cost Optimization Decision Framework

Problem	Solution
Prompts are too long/verbose	Prompt compression, Bedrock Prompt Optimization, context pruning

Problem	Solution
Paying for capability you don't need	Intelligent Prompt Routing, tiered model selection, Distillation
High volume, non-urgent workloads	Batch inference (50% savings), Flex tier
Same prompt prefixes processed repeatedly	Bedrock prompt caching (90% input cost reduction)
Same/similar questions asked by users	Semantic caching with ElastiCache (bypass LLM entirely)
Unpredictable traffic causing over-provisioning	On-demand pricing, SageMaker auto-scaling, scale-to-zero
High predictable traffic	Provisioned Throughput, Reserved tier, SageMaker Savings Plans
Output tokens consuming too much quota	Reduce `max_tokens`, use response size controls
Multi-model deployment wasting GPUs	SageMaker inference components (multi-model endpoints)
Need cheaper model for specific use case	Model Distillation (teacher to student)

Task 4.2 -- Optimize Application Performance

Key Concepts

Responsive AI Systems and Latency Optimization (Skill 4.2.1)

Three primary latency metrics for LLM applications:

- **Time to First Token (TTFT)**: latency from when Bedrock receives a streaming request to the first response token. Most critical for perceived responsiveness.
- **Output Tokens Per Second (OTPS)**: rate at which tokens are generated after the first. Determines how quickly the full response appears.
- **End-to-End Latency (E2E)**: total time from request submission to final token receipt.

Latency-Optimized Inference in Bedrock

Configured via the `performanceConfig` parameter:

```
"performanceConfig": {
    "latency": "optimized"
}
```

- Requires no additional setup or fine-tuning.
- Uses purpose-built AI chips (AWS Trainium2) and software optimizations.
- If latency-optimized quota is exceeded, requests **fall back to standard latency** (no error).
- CloudWatch metrics tracked under `model-id+latency-optimized`.

Response Streaming

Two streaming APIs:

- `InvokeModelWithResponseStream` (model-specific format)
- `ConverseStream` (unified API -- recommended)

Streaming is critical for perceived performance. Users see tokens as they are generated. The `ConverseStream` response returns events: `messageStart`, `contentBlockStart`, `contentBlockDelta`, `contentBlockStop`, `messageStop`, `metadata`.

Streaming does **not** change the total cost -- you pay the same tokens regardless. It only improves perceived latency.

Pre-computation Techniques

- **Prompt caching**: cache frequently used prefixes (up to 85% latency reduction).
- **Semantic caching**: cache full responses for similar queries (bypasses LLM entirely).
- **Edge caching**: serve deterministic responses closer to users via CloudFront.

Parallel Requests for Complex Workflows

- Use **Step Functions** parallel states for independent sub-tasks.
- Bedrock Agents can invoke tools in parallel where tasks are independent.
- Map-reduce: split a document into chunks, process in parallel, combine results.

Retrieval Performance Enhancement (Skill 4.2.2)

Retrieval performance depends on three factors: **index quality**, **query quality**, and **ranking quality**.

Index Optimization for Vector Stores

Amazon OpenSearch Service:

- Vector engines: FAISS, NMSLIB, Lucene.
- Two search methods: **exact k-NN** (brute force) and **approximate k-NN** (HNSW -- faster for large datasets, recommended for production).
- **Auto-optimize**: specify acceptable latency and recall thresholds; OpenSearch evaluates and recommends optimal configurations. Results in under an hour.
- **GPU-accelerated indexing**: up to 10x faster index builds at 1/4 the cost.
- **Disk-based vector search**: keeps reduced-precision vectors in memory for candidate generation, retrieves full-precision vectors for scoring. Runs at 1/3 the cost.
- Distance metrics: Euclidean, cosine similarity, dot product.

Amazon Aurora PostgreSQL (pgvector):

- **HNSW** index: better query performance, higher memory, slower build. Preferred for production.
- **IVFFlat** index: faster build, lower memory, slightly lower recall.
- **Iterative index scans** (pgvector 0.8.0): solve the "overfiltering" problem where combining vector similarity with WHERE clauses returns too few results. Use `relaxed_order` mode for best balance.

Query Preprocessing Techniques

- **Query decomposition**: Bedrock Knowledge Bases break complex queries into sub-queries, retrieve independently, then combine results.
- **Implicit metadata filtering**: Bedrock auto-generates filters from the user query and a metadata schema.
- **Manual metadata filtering**: explicit filters (equals, not equals, greater than, less than, in) to narrow the search space before vector comparison.

Hybrid Search

Bedrock Knowledge Bases search types:

- `Default`: Bedrock decides the strategy.
- `Hybrid`: combines vector embeddings (semantic) with raw text (lexical/keyword). **Only supported for Amazon RDS, OpenSearch Serverless, and MongoDB with a filterable text field.**
- `Semantic`: pure vector search.

Reranking: a second-pass that uses an LLM to reorder initial vector search results. Configure via `VectorSearchBedrockRerankingConfiguration`. Set `numberOfRerankedResults` (1-100) -- fewer results = faster but may exclude relevant content.

> ***Exam tip:*** *Hybrid search requires a filterable text field. Without one, Bedrock falls back to semantic-only. If a question mentions "keyword search not working in Knowledge Bases," check whether the vector store has a filterable text field configured.*

FM Throughput Optimization (Skill 4.2.3)

Throughput is measured in:

- **Requests Per Minute (RPM)**: individual API calls allowed.
- **Tokens Per Minute (TPM)**: total tokens (input + output) per minute.
- **Tokens Per Day (TPD)**: daily token budget (default = TPM x 1,440).

Handling Throttling (429 ThrottlingException)

- **Exponential backoff with jitter** on retries.
- **Token-aware rate limiting**: track consumption client-side before sending.
- **Right-size quotas**: request increases through Service Quotas.
- Break large tasks into smaller chunks to avoid token-based throttling.
- Use streaming responses to process output incrementally.

Capacity Tiers (On-Demand)

Tier	RPM Range	TPM Range	Throttling Risk
Flex	10-100	5K-50K	High
Standard	100-500	50K-150K	Medium
Priority	500-1000+	150K-300K+	Low

Cross-Region Inference for Throughput

- Geographic profiles: route within a geographic boundary, standard pricing.
- Global profiles: route to any commercial region, ~10% cost savings, highest throughput.
- No additional cost for the routing itself.

FM Parameter Tuning (Skill 4.2.4)

Randomness and Diversity Parameters

Parameter	Lower Value Effect	Higher Value Effect
Temperature	More deterministic, higher-probability outputs	More creative/random, lower-probability outputs
Top K	Smaller candidate pool, more predictable	Larger candidate pool, more diverse
Top P	Only highest-probability tokens	Broader set of tokens considered

Parameter Selection Guidelines

Use Case	Temperature	Top P	Top K
Factual Q&A, data extraction	0.0-0.3	0.1-0.3	1-10
Customer service	0.3-0.5	0.5-0.7	20-50
Creative writing, brainstorming	0.7-1.0	0.8-0.95	100+
Code generation	0.1-0.3	0.2-0.5	5-20
Summarization	0.3-0.5	0.5-0.8	20-50

Length and Control Parameters

- `maxTokens`: maximum tokens in the response. Affects quota reservation (see 4.1.1).
- `stopSequences`: character sequences that halt generation.

The Converse API is the recommended unified interface for inference parameters. Use `inferenceConfig` for standard parameters (`maxTokens`, `temperature`, `topP`, `stopSequences`) and `additionalModelRequestFields` for model-specific parameters.

A/B Testing

- **Bedrock Model Evaluations**: compare model performance with automatic metrics, human

evaluation, or LLM-as-a-Judge.

- **Application inference profiles**: tag and track costs/performance per configuration.

> ***Exam tip:*** *Parameter tuning does* ***not change the model*** *-- it only changes inference behavior. Temperature = 0 maximizes determinism but is not guaranteed to be fully deterministic for all models. The Converse API is the preferred interface for setting parameters across models.*

Resource Allocation for FM Workloads (Skill 4.2.5)

Capacity Planning

- Use CloudWatch `EstimatedTPMQuotaUsage` to track actual quota consumption.
- Set alarms at **70-80% utilization** to proactively request increases.
- Account for burndown rate (5x for Claude 3.7+ output) when planning.
- Transition to Reserved when consistent load exceeds ~40% of on-demand costs.

SageMaker Endpoint Auto-Scaling

- **Target-tracking**: use `SageMakerVariantInvocationsPerInstance` or custom metrics.
- **Async endpoints**: can scale to zero using `HasBacklogWithoutCapacity` alarm.
- **Serverless endpoints**: auto-scale Provisioned Concurrency using

`SageMakerVariantProvisionedConcurrencyUtilization`.

- Configure cooldown periods (default 300s) to prevent rapid fluctuation.

Lambda Concurrency for GenAI

- Default: 1,000 concurrent executions per Region across all functions.
- **Reserved concurrency**: sets a maximum for a specific function. Use to **limit** FM call

rate and prevent exceeding Bedrock RPM/TPM quotas.

- **Provisioned concurrency**: pre-initializes execution environments to eliminate cold starts.

Can auto-scale using Application Auto Scaling.

> ***Exam tip:*** *Reserved concurrency on Lambda is a* ***rate-limiting mechanism*** *for downstream FM calls. This is a commonly tested pattern. Bedrock handles its own scaling for on-demand inference -- you do not configure*

scaling policies for Bedrock itself.

System Performance Optimization (Skill 4.2.6)

API Call Profiling

- Enable **Bedrock Model Invocation Logging** (disabled by default) to capture full

request/response data.

- Send logs to CloudWatch Logs (real-time analysis via Logs Insights) and/or S3.
- Use **application inference profiles** to tag and track specific usage patterns by team

or application.

Key profiling questions:

- What is the input/output token ratio? (Guides `max_tokens` sizing.)
- Which requests are throttled? (Guides tier selection.)
- What is the TTFT distribution? (Guides latency optimization.)
- Are there time-based patterns in token consumption? (Guides scheduled scaling.)

Vector Database Query Optimization

- OpenSearch: use HNSW, quantization, disk-based search, auto-optimize.
- Aurora pgvector: use HNSW indexes, enable iterative scans for filtered queries.
- Bedrock Knowledge Bases: use reranking, metadata filtering, query decomposition, hybrid search.

Latency Reduction Techniques for LLM Inference

1. Response streaming (`ConverseStream`).
2. Latency-optimized inference (`performanceConfig.latency = "optimized"`).
3. Prompt caching (up to 85% latency reduction).
4. Intelligent Prompt Routing (route simple queries to faster models).
5. Cross-region inference (route to less-busy regions).
6. Smaller context windows and model selection (use Haiku for simple tasks).

Efficient Service Communication Patterns

- **Asynchronous**: SQS + Lambda for decoupled, rate-limited FM invocations. Set Lambda

concurrency to match Bedrock RPM limits.

- **Circuit breaker**: stop sending requests on 503 (ServiceUnavailable) to avoid cascading

failures. Implement with Step Functions or client-side logic.

- **Exponential backoff with jitter**: standard retry for 429 (Throttling).
- **Fan-out/fan-in**: Step Functions parallel states for independent sub-tasks.
- **Event-driven**: EventBridge + Lambda for async GenAI processing.

X-Ray Tracing

- **Segments**: processing data per service. **Subsegments**: downstream calls within a segment.
- **Traces**: end-to-end request path (e.g., API Gateway to Lambda to Bedrock to DynamoDB).
- **Annotations**: indexed key-value pairs for filtering (model_id, token_count).
- **Metadata**: non-indexed data for debugging (prompt content, response content).
- **Adaptive Sampling**: dynamically increases sampling during anomalies. Includes Sampling

Boost (rate increase during anomalies) and Anomaly Span Capture (captures critical spans even without full traces). 1-minute boost window, 10-minute cooldown.

Exam tip: *X-Ray annotations are indexed and searchable; metadata is not. The circuit breaker pattern is for 503 errors; exponential backoff is for 429 errors. SQS + Lambda with reserved concurrency is the canonical*

pattern for rate-limiting FM calls.

Task 4.3 -- Implement Monitoring Systems for GenAI Applications

Key Concepts

Holistic Observability Systems (Skill 4.3.1)

GenAI observability extends the three traditional pillars (metrics, traces, logs) with FM-specific signals: token consumption, response quality, and reasoning path visibility.

CloudWatch Runtime Metrics

Bedrock publishes metrics under `AWS/Bedrock`:

Metric	Description
`Invocations`	Successful requests (Converse, InvokeModel, and streaming variants)
`InvocationLatency`	Time from request to last token (ms)
`InvocationClientErrors`	4xx errors
`InvocationServerErrors`	5xx errors
`InvocationThrottles`	Throttled requests (do NOT count as Invocations or Errors)
`InputTokenCount`	Input tokens
`OutputTokenCount`	Output tokens
`TimeToFirstToken`	Time to first token (streaming APIs only)
`EstimatedTPMQuotaUsage`	Quota consumption including cache writes and burndown multipliers

The `ModelId` dimension filters all metrics by model. TTFT and EstimatedTPMQuotaUsage are emitted at no additional cost, updated every minute.

Bedrock Agent Traces

Every agent response includes a structured trace (enable with `enableTrace: TRUE` in the `InvokeAgent` request, disabled by default). Trace types:

- **PreProcessingTrace**: how the agent contextualizes and categorizes inputs.
- **OrchestrationTrace**: decision logic -- which actions to execute, which knowledge bases to

query. Contains `rationale` (reasoning), `invocationInput` (action being invoked), `modelInvocationInput/Output`, and `observation` (results).

- **PostProcessingTrace**: how the agent shapes the final response.
- **FailureTrace**: reason a step failed.
- **GuardrailTrace**: actions taken by guardrails.
- **RoutingClassifierTrace**: input/output of multi-agent routing decisions.
- **CustomOrchestrationTrace**: for custom orchestration logic.

X-Ray Integration

- Bedrock AgentCore integrates with X-Ray via the **AWS Distro for OpenTelemetry (ADOT) SDK**.
- X-Ray SDKs are in maintenance mode; AWS recommends migrating to OpenTelemetry.
- AgentCore emits telemetry in **OpenTelemetry-compatible format**.

Custom CloudWatch Dashboards

Pre-configured dashboards include invocation count, latency, token counts by model, daily trends, throttle counts. Build custom dashboards combining Bedrock, Agent, Guardrail, and Knowledge Base metrics alongside infrastructure metrics.

Business Impact Metrics

These are custom metrics (not built-in). Examples: task completion rates, user satisfaction, revenue per interaction, support deflection, cost per conversation. Use `CloudWatch PutMetricData` or embed metrics via CloudWatch Logs metric filters.

> ***Exam tip:*** *Throttled requests do NOT count as Invocations or Errors. TTFT is only available for streaming API operations. EstimatedTPMQuotaUsage accounts for cache write tokens and output burndown multipliers.*

Comprehensive GenAI Monitoring (Skill 4.3.2)

Model Invocation Logging

Must be explicitly enabled (disabled by default). Configuration via `PutModelInvocationLoggingConfiguration` or the console Settings page.

Logs can be sent to:

- **CloudWatch Logs** -- for real-time analysis via Logs Insights.
- **Amazon S3** -- required for large data, binary content (images, video).
- Both simultaneously.

Captures: full request/response payloads, metadata (model ID, request ID, timestamp, token counts, latency). Choose which modalities to log (text, image, embedding, video). The S3 bucket must be in the same Region as the Bedrock logging configuration.

Prompt Effectiveness Monitoring with Bedrock Evaluations

- **Programmatic evaluations**: computed metrics for accuracy, robustness, toxicity.
- **LLM-as-a-Judge**: a second LLM scores responses on correctness, completeness, helpfulness,

logical coherence, faithfulness, harmfulness, stereotyping, answer refusal.

- **RAG evaluations**: context relevance, context coverage, citation precision, citation

coverage, correctness, completeness, faithfulness.

- **Bring Your Own Inference (BYOI)**: evaluate responses from models hosted anywhere.
- Maximum **1,000 prompts per evaluation** job.

Hallucination Rate Tracking

Three mechanisms:

1. **Contextual Grounding Checks** (Bedrock Guardrails): validates responses against a grounding

source and user query. Filters over 75% of hallucinated responses for RAG/summarization. Configurable thresholds per chunk.

2. **Automated Reasoning Checks** (Bedrock Guardrails): uses formal mathematical logic to verify

accuracy. Up to 99% verification accuracy. Encodes domain rules as a policy. Limited regional availability.

3. **Custom detection**: build agentic workflows with Bedrock Agents that detect hallucinations

and route to human review via SNS.

Anomaly Detection

CloudWatch Anomaly Detection (metrics):

- ML algorithms analyze past data and create models of expected values.
- Accounts for hourly, daily, and weekly patterns.

- Alarms trigger when metrics go above, below, or outside the expected band.
- Apply to `InputTokenCount`, `OutputTokenCount`, `InvocationLatency`.

CloudWatch Logs Anomaly Detection:

- `anomaly` command in Logs Insights identifies unusual log patterns.
- Assigns priority and severity levels.
- Can suppress anomalies temporarily or permanently.

AWS Cost Anomaly Detection

- ML-based identification of anomalous spend patterns.
- Runs approximately 3 times per day (not real-time).
- Monitor types: AWS Managed (auto-track all accounts/services) and Customer Managed (specific accounts/tags, max 10 per monitor).
- Alert via email, SNS, EventBridge, or AWS User Notifications.

> ***Exam tip:*** *Model invocation logging is disabled by default. Contextual grounding checks require a grounding source -- they cannot operate without reference material. Automated Reasoning checks provide deterministic, mathematically provable verification (unique among GenAI safeguards). Cost Anomaly Detection is NOT real-time.*

Integrated Observability Solutions (Skill 4.3.3)

CloudWatch GenAI Observability Dashboards

Two pre-built capabilities:

1. **Model Invocations Dashboard**: model usage, token consumption, curated invocation logs table showing input/output content. Metrics include total/average invocations, token usage, latency (average, P90, P99), error rates, throttling, cost attribution.
2. **AgentCore Agents Dashboard**: performance metrics for Agents, Memory, Built-in Tools, Gateways, and Identity.

Advanced Analytics Pipeline

For deeper analysis beyond CloudWatch dashboards:

- Send invocation logs to **S3**, query with **Amazon Athena**, visualize in **Amazon QuickSight**.
- Use Athena Federated Queries to combine Bedrock logs with other data sources.

Compliance Monitoring

- CloudWatch Logs **Sensitive Data Protection** policies can mask or redact PII in logs.
- Use the `bedrock:GuardrailIdentifier` IAM condition key to enforce guardrail usage at the API level -- non-matching requests are rejected.
- Amazon Bedrock adheres to GDPR, HIPAA, SOC, FedRAMP High.

Forensic Traceability with CloudTrail

CloudTrail records:

- **Management Events**: control plane operations (CreateAgent, UpdateKnowledgeBase).
- **Data Events**: runtime operations (InvokeAgent, InvokeModel, Converse). Must be explicitly enabled.

Best practices from the Well-Architected GenAI Lens:

- Enable centralized log storage.
- Maintain detailed request/response logs.

- Record user interactions and system changes.
- Log model version changes and configuration modifications.

Amazon Managed Grafana

Advantages over CloudWatch dashboards:

- Manage dashboard access separately from AWS account access.
- Advanced visualization widgets.
- Cross-account and cross-region data access via AWS Organizations.
- Non-AWS data sources and community plugins.
- Multiple workspaces with separate data sources, users, and policies.
- Requires **IAM Identity Center** for authentication.

> ***Exam tip:*** *CloudTrail data events for InvokeModel must be explicitly enabled. Managed Grafana requires IAM Identity Center. The* `bedrock:GuardrailIdentifier` *IAM condition key enforces guardrail usage at the API level.*

Tool Performance Frameworks (Skill 4.3.4)

Agent Metrics

Bedrock Agents publish to the `AWS/Bedrock/Agents` namespace (not `AWS/Bedrock`):

Metric	Description
`InvocationCount`	Requests to InvokeAgent/InvokeInlineAgent
`TotalTime`	End-to-end server processing time
`TTFT`	Time-to-first-token (streaming only)
`ModelLatency`	Time the FM took
`ModelInvocationCount`	Number of FM calls the agent made
`InputTokenCount` / `OutputTokenCount`	Token consumption

Dimensions: `Operation`, `ModelId`, `AgentAliasArn`.

OrchestrationTrace for Tool Analysis

The trace reveals tool call patterns: which action groups were selected (`invocationInput`), why (`rationale`), what the tool returned (`observation`), and how many FM calls were needed.

External Tool Monitoring

Lambda functions as action groups publish their own CloudWatch metrics (Duration, Errors, Throttles, ConcurrentExecutions). Correlate Lambda metrics with Agent metrics via request IDs. X-Ray tracing spans across agent-to-Lambda invocations.

Multi-Agent Coordination Tracking

- `RoutingClassifierTrace` traces routing classifier decisions for multi-agent setups.
- AgentCore supports Agent-to-Agent (A2A) protocol and Model Context Protocol (MCP).
- Track coordination through linked traces across agents.

Usage Baselines

Establish baselines and alarm on deviations:

- Number of FM calls per agent invocation (if normally 3 but suddenly 10, that's an anomaly).
- Token consumption per interaction.

- Tool selection frequency distributions.
- Error rates per tool.

> ***Exam tip:*** *Agent metrics are in* `AWS/Bedrock/Agents`*, NOT* `AWS/Bedrock`*. The agent IAM service role needs explicit* `cloudwatch:PutMetricData` *permission for this namespace. TTFT for agents requires streaming configuration to be enabled.*

Vector Store Operational Management (Skill 4.3.5)

OpenSearch Performance Monitoring

CloudWatch metrics under `AWS/ES` (managed) or `AWS/AOSS` (serverless):

Metric Category	Key Metrics
Cluster health	`ClusterStatus.green/yellow/red`, `CPUUtilization`, `JVMMemoryPressure`
Search	`SearchLatency`, `SearchRate`, `SearchRequestErrors`
Indexing	`IndexingLatency`, `IndexingRate`
k-NN specific	`KNNGraphMemoryUsage` (graph memory must fit in RAM)
Serverless	`SearchOCU`, `IndexingOCU`, `StorageUsedInS3`

Automated Index Optimization

- **Auto-Optimize**: specify latency and recall thresholds; system recommends optimal configurations. Available on OpenSearch 2.17+ and serverless collections.
- **GPU-accelerated indexing**: serverless GPUs attach dynamically for HNSW operations.

Knowledge Base Data Quality Monitoring

- Enable CloudWatch Logs for knowledge bases (APPLICATION_LOGS type).
- Track: documents ignored during ingestion, exceptions during embedding, exceptions during indexing/deletion.
- RAG Evaluation metrics (context relevance, coverage, faithfulness) serve as quality indicators.

Embedding Drift Detection

No single AWS service detects embedding drift. Combine:

- Periodic RAG evaluation jobs on a stable test dataset.
- Recall monitoring against established baselines.
- Search result relevance tracking via user feedback.
- Distance distribution monitoring (shift in similarity score patterns).

> ***Exam tip:*** `KNNGraphMemoryUsage` *is the critical metric for vector workloads -- the graph must fit in RAM. Auto-Optimize is a batch job, not a real-time process. Embedding drift requires a combination of periodic evaluations and metric monitoring.*

FM-Specific Troubleshooting Frameworks (Skill 4.3.6)

Golden Datasets for Hallucination Detection

A curated set of question-answer pairs with verified correct answers (ground truth):

- Bedrock Model Evaluation supports ground-truth datasets in **JSONL format** in S3.
- Maximum 1,000 prompts per evaluation.
- S3 bucket requires CORS permissions.

- Programmatic evaluations compare outputs against known-correct answers.
- LLM-as-a-Judge scores outputs for correctness, faithfulness, completeness.

Output Diffing for Response Consistency

- Run the same prompt multiple times and compare outputs.
- Track response variation across model versions.
- Use Bedrock Evaluations with BYOI to compare responses from different configurations.
- Store outputs in S3, query with Athena for systematic comparison.
- Use LLM-as-a-Judge to score consistency between outputs.

Reasoning Path Tracing for Logical Errors

The Agent Trace is the primary tool. Debugging workflow:

1. Enable traces (`enableTrace: TRUE`).
2. Examine PreProcessingTrace -- verify correct intent classification.
3. Walk through OrchestrationTrace steps -- find where reasoning diverges.
4. Check if correct knowledge bases and action groups were invoked.
5. Examine PostProcessingTrace -- verify response formatting.
6. Look for FailureTrace -- identify explicit error points.

AgentCore Observability provides end-to-end prompt tracing in CloudWatch for inspecting each step, auditing intermediate outputs, and debugging bottlenecks.

Specialized Observability Pipeline Architecture

1. **Collection**: invocation logs (CloudWatch Logs + S3), agent traces, AgentCore telemetry (OpenTelemetry), CloudTrail audit events, custom metrics.
2. **Analysis**: Logs Insights for real-time queries, Anomaly Detection for metrics and logs, Athena for S3 log analysis, Bedrock Evaluations for quality.
3. **Visualization**: CloudWatch dashboards, Managed Grafana, QuickSight.
4. **Alerting**: CloudWatch Alarms, EventBridge rules, SNS notifications, Cost Anomaly Detection.
5. **Third-party integration**: Arize AI Phoenix, Datadog, Langfuse, IBM Instana -- all via OpenTelemetry.

> ***Exam tip:*** *Golden datasets must be JSONL in S3 with CORS enabled. Agent traces are the primary tool for reasoning path analysis -- look for the* `rationale` *field. Automated Reasoning checks are the only GenAI safeguard with deterministic, mathematically provable verification. There is no single "hallucination metric" in CloudWatch.*

Monitoring Service Quick Reference

Monitoring Need	Primary AWS Service
Runtime metrics (latency, tokens, errors)	CloudWatch (`AWS/Bedrock`, `AWS/Bedrock/Agents`)
Full request/response logging	Bedrock Model Invocation Logs (CloudWatch Logs + S3)
Agent reasoning visibility	Bedrock Agent Trace (`enableTrace`)
Distributed tracing	X-Ray via ADOT SDK / AgentCore Observability
API audit logging	CloudTrail
Hallucination detection	Bedrock Guardrails (Contextual Grounding + Automated Reasoning)

Monitoring Need	Primary AWS Service
Model quality evaluation	Bedrock Evaluations (Programmatic + LLM-as-a-Judge + RAG)
Metric anomaly detection	CloudWatch Anomaly Detection
Log anomaly detection	CloudWatch Logs Anomaly Detection
Cost anomaly detection	AWS Cost Anomaly Detection
Vector DB monitoring	CloudWatch (`AWS/ES` or `AWS/AOSS`)
Vector index optimization	OpenSearch Auto-Optimize
Advanced dashboards	Amazon Managed Grafana
Business analytics	Athena + QuickSight
Compliance enforcement	IAM (`bedrock:GuardrailIdentifier` condition key) + CloudTrail

Common Exam Pitfalls for Domain 4

1. **Forgetting to enable logging**: Model invocation logging, knowledge base logging, and CloudTrail data events are all off by default.
2. **Wrong CloudWatch namespace**: Agent metrics are in `AWS/Bedrock/Agents`, not `AWS/Bedrock`.
3. **Confusing caching types**: Bedrock prompt caching (model computation state) vs. semantic caching (full response, bypasses LLM) vs. edge caching (CloudFront, deterministic responses).
4. `max_tokens` **and quota**: it is deducted at request start, not end. Setting it too high reduces concurrency even though billing is based on actual tokens.
5. **Burndown rate**: 5x for Claude 3.7+ output tokens. Affects quota, not billing.
6. **Streaming does not change cost**: same tokens billed regardless, only perceived latency improves.
7. **Bedrock manages its own scaling**: you do not configure auto-scaling for Bedrock on-demand inference. You do configure scaling for SageMaker endpoints and Lambda.
8. **Automated Reasoning vs. Contextual Grounding**: Automated Reasoning uses formal math (deterministic); Contextual Grounding uses probabilistic NLP assessment.
9. **Hybrid search prerequisite**: requires a filterable text field in the vector store.
10. **Cost Anomaly Detection timing**: runs ~3 times daily, not real-time.

Domain 5: Testing, Validation, and Troubleshooting

Exam Weight: 11% — the smallest domain on the exam

This domain covers two task areas: implementing evaluation systems for GenAI (Task 5.1 with nine skills) and troubleshooting GenAI applications (Task 5.2 with five skills). Despite its lower weight, this domain is uniquely cross-cutting — it requires fluency with services and concepts from every other domain.

Task 5.1 — Implement Evaluation Systems for GenAI

Key Concepts

Comprehensive Assessment Frameworks (Skill 5.1.1)

Traditional ML evaluation metrics (accuracy, precision, recall, F1) do not capture the quality dimensions of generative outputs. FM evaluation requires metrics across four categories:

Category	Metrics
Quality	Correctness, completeness, helpfulness, logical coherence, relevance, following instructions
Faithfulness	Grounding to source material, hallucination detection, citation accuracy
Style	Professional tone, fluency, formatting appropriateness
Responsible AI	Harmfulness, stereotyping, refusal behavior

Traditional NLP overlap metrics are still relevant as baselines but insufficient alone:

Metric	What It Measures	Limitation for GenAI
ROUGE	N-gram overlap between generated and reference text	Penalizes valid paraphrases
BLEU	Precision of n-gram overlap (machine translation)	Same — punishes diverse but correct outputs
BERTScore	Semantic similarity using contextual embeddings	Better than lexical metrics but does not catch factual errors

For GenAI, prefer **LLM-as-a-Judge** or **human evaluation** over lexical metrics, because generative models produce valid responses that differ significantly in wording from reference answers.

> ***Exam tip:*** *If a question asks about evaluating FM outputs "beyond traditional ML evaluation," the answer involves LLM-as-a-Judge metrics (correctness, completeness, faithfulness) rather than ROUGE/BLEU scores.*

Systematic Model Evaluation with Bedrock (Skill 5.1.2)

Amazon Bedrock provides three evaluation methods:

Method	How It Works	Best For
Programmatic (automatic)	Predefined NLP metrics (accuracy, robustness, toxicity) against curated datasets	Quick baseline comparisons
Human evaluation	Bring your own dataset; define custom metrics; use your team or AWS-managed workers	Subjective quality (brand voice, style, nuance)

Method	How It Works	Best For
LLM-as-a-Judge	A judge model (e.g., Claude, Nova Pro) scores the generator model's output using curated or custom metrics	Scalable human-like quality at up to 98% cost savings vs. full human eval

Bedrock LLM-as-a-Judge built-in metrics:

Metric ID	What It Scores
`Builtin.Correctness`	Is the response accurate? Considers ground truth if provided
`Builtin.Completeness`	Does the response answer all aspects of the question?
`Builtin.Faithfulness`	Does the response contain only information found in the provided context?
`Builtin.Helpfulness`	Is the response useful, coherent, and does it anticipate implicit needs?
`Builtin.Coherence`	Is the response logically consistent without gaps or contradictions?
`Builtin.Relevance`	Is the response relevant to the prompt?
`Builtin.FollowingInstructions`	Does the response follow exact directions from the prompt?
`Builtin.ProfessionalStyleAndTone`	Is the style appropriate for a professional setting?
`Builtin.Harmfulness`	Does the response contain harmful content?
`Builtin.Stereotyping`	Does the response contain stereotypes (positive or negative)?
`Builtin.Refusal`	Does the response decline to answer or reject the request?

All metric scores range **0 to 1** (closer to 1 = more of that characteristic).

Custom metrics are also supported: you write a judge prompt, define a rating scale, and use built-in variables to inject data at runtime. Custom metrics enable evaluation against specific brand voice, categorical rubrics, or domain-specific criteria.

Supported evaluator models: Amazon Nova Pro, Anthropic Claude (3.5 Sonnet v1/v2, 3.7 Sonnet, 3 Haiku, 3.5 Haiku), Meta Llama 3.1 70B Instruct, Mistral Large. Cross-Region inference profiles are supported for all evaluator models.

Multi-model evaluation and comparison: Bedrock supports running evaluation jobs across multiple generator models and comparing results side-by-side. Use this to evaluate cost-performance tradeoffs:

- **Token efficiency** — compare input/output token counts per response across models.
- **Latency-to-quality ratios** — measure response quality vs. time-to-first-token.
- **Business outcomes** — tie evaluation metrics to downstream KPIs (conversion rate, support ticket resolution).

A/B testing and canary testing:

Strategy	Description	Implementation
A/B testing	Route a percentage of traffic to a new model/prompt variant; compare metrics	AppConfig feature flags, Bedrock Prompt Flows, or application-level routing
Canary testing	Deploy a new version to a small fraction of traffic (e.g., 5%); gradually increase if stable	Step Functions, CodeDeploy traffic shifting, or application routing logic
Shadow deployment	Run input through both old and new versions; compare outputs without exposing the new version to users	Parallel Lambda invocations; compare results offline

Max prompts per evaluation job: 1,000.

> ***Exam tip:*** *Questions asking "how to identify the optimal model configuration" point to Bedrock Model Evaluations with LLM-as-a-Judge, combined with A/B testing or canary releases for online validation.*

User-Centered Evaluation Mechanisms (Skill 5.1.3)

Continuous improvement requires feedback from real users:

Feedback Type	Examples	Collection Method
Explicit	Thumbs up/down, star ratings, free-text comments	API Gateway for feedback endpoints; store in DynamoDB
Implicit	Copy response, rephrase query, abandon conversation, time on response	Application event logging; CloudWatch custom metrics
Annotation workflows	Subject-matter experts label responses for quality, accuracy, relevance	SageMaker Ground Truth, Amazon Augmented AI (A2I), or custom annotation UIs

Feedback loop pattern:

1. Collect feedback signals from users.
2. Aggregate and analyze in CloudWatch or a data warehouse.
3. Triage failures — convert user-reported failures into new test cases.
4. Add new cases to the versioned evaluation dataset.
5. Re-evaluate the system; iterate on prompts, models, or retrieval settings.

This creates a **virtuous cycle** where production usage directly hardens the automated test suite.

> ***Exam tip:*** *Questions about "continuously improving FM performance based on user experience" point to feedback collection interfaces (thumbs up/down), annotation workflows (SageMaker Ground Truth / A2I), and feeding failures back into the evaluation dataset.*

Systematic Quality Assurance Processes (Skill 5.1.4)

GenAI QA requires a multi-layered testing framework:

Layer	Test Type	What It Validates
Foundational	Unit tests	Individual deterministic components (data transformation, API clients) — mock FM calls
Foundational	Integration tests	Component interactions — data pipeline integrity, tool/API integration, auth flows
Foundational	End-to-end tests	Full workflow from user input through response — verify workflow completion and structural correctness, not exact output
Quality Assessment	Offline evaluation	LLM-as-a-Judge scoring against versioned evaluation datasets; integrated into CI/CD as quality gates
Quality Assessment	Online evaluation	A/B testing and canary releases with real traffic
Quality Assessment	Human-in-the-loop	Expert review, feedback collection, annotation workflows

Continuous evaluation workflows:

- Integrate offline evaluation into CI/CD pipelines (CodePipeline/CodeBuild).
- Pipeline fails the build if quality scores drop below predefined thresholds.
- This automatically prevents regressions.

Regression testing for model outputs:

- Maintain a **golden dataset** of known-good prompt/response pairs.
- After any change (prompt update, model swap, retrieval config change), re-run the golden dataset through the evaluation pipeline.
- Compare scores against baselines; flag any degradation.

Automated quality gates for deployments:

- Step Functions orchestrate: build -> unit test -> evaluation -> security scan -> deploy.
- Evaluation stage runs the versioned dataset through LLM-as-a-Judge.
- Only deploy if all quality thresholds pass.

Key unit testing practice for GenAI: Mock and stub FM API calls. Unit tests must remain fast and isolated — they test the deterministic code around the FM, not the FM itself. Test edge cases for input preprocessing and output parsing logic.

Key end-to-end testing practice for GenAI: Do not assert on exact output text. Instead, verify that the workflow completes, the response is structurally correct (e.g., valid JSON), and the content is semantically relevant. Handle variable response times with appropriate timeouts.

> ***Exam tip:*** *"Maintain consistent performance standards" and "quality gates" both point to CI/CD-integrated evaluation pipelines that fail builds when LLM-as-a-Judge scores fall below thresholds.*

Comprehensive Assessment — RAG Evaluation and LLM-as-a-Judge (Skill 5.1.5)

RAG Evaluation in Amazon Bedrock supports two job types:

Job Type	What Is Evaluated	Metrics
Retrieve only	Quality of retrieved chunks from the knowledge base	Context relevance, context coverage (requires ground truth)
Retrieve and generate	Retrieved chunks + generated response quality	All retrieval metrics + correctness, completeness, faithfulness, helpfulness, coherence, citation precision, citation coverage, harmfulness, stereotyping, refusal

RAG retrieval metrics explained:

Metric	Description
Context relevance	Are retrieved chunks contextually relevant to the query? (Precision-oriented)
Context coverage	Do retrieved chunks cover all information in the ground truth? (Recall-oriented; requires ground truth)

RAG generation metrics (in addition to standard LLM-as-a-Judge metrics):

Metric	Description
Citation precision	How many cited passages are cited correctly?
Citation coverage	How well is the response supported by cited passages? (Approximately citation recall)
Faithfulness	Does the response avoid hallucination relative to retrieved text?

Best practice: Always use both citation precision and citation coverage together for a complete view of citation quality.

Bring Your Own Inference (BYOI): Bedrock RAG evaluation supports evaluating custom RAG pipelines — bring your own input-output pairs and retrieved contexts directly, bypassing the Bedrock Knowledge Base call.

Bedrock Guardrails integration: You can incorporate Guardrails directly into RAG evaluation jobs for more comprehensive testing of safety controls.

Hallucination detection approaches:

Approach	AWS Implementation
Contextual grounding checks	Bedrock Guardrails — verify response is grounded in source material and relevant to query
Automated Reasoning checks	Bedrock Guardrails — mathematical logic and formal verification; up to 99% verification accuracy
Faithfulness scoring	LLM-as-a-Judge `Builtin.Faithfulness` metric
Golden dataset comparison	Compare FM responses against known-correct answers; flag deviations

Contextual grounding checks in Bedrock Guardrails use two paradigms:

- **Grounding** — ensures factual accuracy against the source material.
- **Relevance** — ensures the response is relevant to the user query.

Each chunk is independently evaluated; if any chunk passes, the whole response is considered grounded/relevant. Configurable thresholds control filtering sensitivity.

> ***Exam tip:*** *"Ensure thorough evaluation from multiple perspectives" means combining RAG evaluation (retrieval + generation metrics), LLM-as-a-Judge (automated quality), and human feedback collection in a single evaluation strategy.*

Retrieval Quality Testing (Skill 5.1.6)

Evaluate retrieval components independently from generation:

Dimension	How to Test	Tools
Relevance scoring	Measure how well retrieved chunks answer the query	Bedrock RAG evaluation (retrieve-only), RAGAS context_relevancy metric
Context matching verification	Verify retrieved content matches expected ground truth	Context coverage metric (requires ground truth annotations)
Retrieval latency	Measure time from query to retrieved results	CloudWatch custom metrics on Knowledge Base API calls; Application Signals
Embedding quality	Compare embedding similarity scores for known-similar and known-dissimilar pairs	Custom benchmark scripts; MTEB leaderboard for model selection

RAGAS framework metrics (open-source, works with Bedrock):

- Context precision, context recall, faithfulness, answer relevancy, answer correctness.

Testing retrieval configuration changes: Use Bedrock RAG evaluation's compare feature to evaluate different Knowledge Base configurations side-by-side (different chunking strategies, embedding models, vector stores, retrieval settings).

Agent Performance Frameworks (Skill 5.1.7)

Amazon Bedrock AgentCore Evaluations provides automated assessment of agent performance:

Evaluation Type	Description
Online evaluation	Continuously monitors agent quality using live production traffic; configurable session sampling and percentage-based filtering
On-demand evaluation	Targeted assessment of specific agent interactions; useful for testing, investigation, and historical analysis

Built-in agent evaluation metrics:

- **Goal attainment** — did the agent accomplish the user's stated goal?
- **Tool accuracy** — did the agent select the correct tool and extract the right parameters?
- **Custom evaluators** — define domain-specific metrics for agent behavior.

Agent trace analysis — Bedrock provides detailed traces for debugging agent behavior:

Trace Type	What It Shows
PreProcessingTrace	Input/output of the pre-processing step (intent classification)
OrchestrationTrace	Reasoning steps, tool selection, knowledge base queries
PostProcessingTrace	Response formatting and finalization
FailureTrace	Reason a step failed
GuardrailTrace	Actions taken by Guardrails
RoutingClassifierTrace	How the agent classified the request for routing

Key agent evaluation dimensions:

- **Task completion rate** — percentage of user requests fully resolved.
- **Tool usage effectiveness** — correct tool selected, correct parameters extracted, correct execution.
- **Reasoning quality** — logical consistency of multi-step reasoning chains.
- **Latency** — end-to-end response time for agent workflows.

AgentCore Evaluations integrates with Strands Agents, LangGraph, and OpenTelemetry/ OpenInference. Traces are stored in CloudWatch Transaction Search and can be evaluated using the `Evaluate` API.

Limits: Up to 1,000 evaluation configurations per region; up to 100 active at a time; up to 1 million input/output tokens per minute for large regions.

> ***Exam tip:*** *Agent evaluation questions map to Bedrock AgentCore Evaluations for automated assessment, and Bedrock agent traces for debugging reasoning paths and tool usage.*

Comprehensive Reporting Systems (Skill 5.1.8)

Communicate evaluation results to stakeholders:

Tool	Use Case
Bedrock evaluation report card	Console-based visualization of metric histograms and scores per evaluation job
Bedrock evaluation compare	Side-by-side comparison of multiple evaluation jobs (different models, configs)
Amazon QuickSight	Custom dashboards for business stakeholders; connect to evaluation data in S3
CloudWatch dashboards	Operational dashboards for token usage, latency, error rates, model invocation metrics
Amazon Managed Grafana	Advanced visualization with CloudWatch, Prometheus, or custom data sources
Automated reports	Step Functions + Lambda to generate periodic evaluation summaries; deliver via SNS/SES

Bedrock evaluation report card details: The console displays a metrics summary card with histogram charts showing the distribution of scores across all prompts. Each metric includes an average score (0-1) and a breakdown graph plotting how many responses fall within each score range.

Deployment Validation Systems (Skill 5.1.9)

Ensure reliability during FM updates (model changes, prompt changes, config changes):

Validation Technique	Description	Implementation
Synthetic user workflows	Automated test scenarios that simulate real user interactions	CloudWatch Synthetics canaries; Step Functions orchestrated test flows
Hallucination rate monitoring	Track faithfulness scores over time; alert on degradation	Bedrock evaluation jobs + CloudWatch alarms on faithfulness metric
Semantic drift detection	Compare embedding distributions of responses over time; detect when outputs shift meaning	Custom Lambda comparing response embeddings against baseline vectors
Output diffing	Compare new version outputs against baseline for same inputs	Lambda-based diff pipeline; flag significant deviations
Response consistency checks	Verify the same prompt produces consistent outputs across multiple invocations	Run golden dataset multiple times; measure variance in LLM-as-a-Judge scores
Canary releases	Route small traffic percentage to new version; monitor before full rollout	CodeDeploy traffic shifting; application-level routing

GenAIOps CI/CD pipeline for deployment validation:

1. **Trigger** — commit to main branch or promotion of a new prompt version.
2. **Build** — package application into container images.
3. **Unit test** — test deterministic components (data processing, API clients).
4. **Evaluation** — run versioned dataset through LLM-as-a-Judge; fail if scores below threshold.
5. **Security scan** — automated adversarial testing (prompt injection, PII exposure).
6. **Deploy** — deploy to staging; run canary release or A/B test.

Versioned artifacts for GenAIOps: Every deployment is a snapshot of the entire stack — code version (Git commit), prompt version, model configuration, and evaluation dataset version. This ensures reproducibility and auditability.

> ***Exam tip:*** *"Maintain reliability during FM updates" = synthetic user workflows + automated quality checks (LLM-as-a-Judge in CI/CD) + canary releases + hallucination/drift monitoring.*

Task 5.2 — Troubleshoot GenAI Applications

Key Concepts

Content Handling Issues (Skill 5.2.1)

Context window overflow is one of the most common GenAI failure modes:

Problem	Symptoms	Resolution
Context window overflow	`ValidationError`, truncated responses, missing information in output	Reduce input size: prompt compression, context pruning, smaller chunks
Token limit exceeded	API error (400) indicating input exceeds model's max token count	Count tokens before sending; use tiktoken or model-specific tokenizer; truncate or split
Truncated retrieval context	RAG responses miss important details; incomplete answers	Reduce number of retrieved chunks; use hierarchical chunking (child for precision, parent for context); increase chunk overlap
Long conversation history	Multi-turn conversations exceed context window mid-session	Implement sliding window (keep last N turns); summarize older turns; store full history in DynamoDB but send only recent + summary

Dynamic chunking strategies for troubleshooting:

- If responses are incomplete or miss key details, increase chunk size or overlap.
- If responses contain irrelevant information, decrease chunk size for more precise retrieval.
- Switch from fixed-size to semantic or hierarchical chunking if content boundaries are poorly aligned with chunk boundaries.

Prompt design optimization for content handling:

- Move critical instructions to the beginning of the prompt (models attend more to the start).
- Use explicit delimiters (`<context>`, `<instructions>`) to help the model parse long inputs.
- Compress retrieved context: summarize or extract key sentences before injecting into prompt.
- Set `max_tokens` for the response to reserve space within the context window for input.

> ***Exam tip:*** *Context window overflow questions often have "ValidationError" or "truncated response" in the scenario. The answer involves token counting, prompt compression, context pruning, or dynamic chunking.*

FM Integration Issues (Skill 5.2.2)

Common Bedrock API errors and their resolutions:

Error	HTTP Code	Cause	Resolution
`AccessDeniedException`	403	Missing IAM permissions	Verify IAM policy grants `bedrock:InvokeModel` or `bedrock-runtime:*` for the target model
`ThrottlingException`	429	Exceeded account quotas (tokens/min, requests/min)	Implement exponential backoff with jitter; request quota increase; use Provisioned Throughput or Cross-Region inference
`ValidationError`	400	Malformed request body, invalid parameters, token limit exceeded	Validate request format against model-specific schema; check parameter ranges; count tokens
`ServiceUnavailable`	503	Temporary service issue	Exponential backoff with jitter; switch to different Region (Cross-Region inference); use Provisioned Throughput
`InternalFailure`	500	Server error	Retry with exponential backoff; contact AWS Support if persistent
`ResourceNotFound`	404	Invalid model ID, endpoint name, or resource identifier	Verify model ID; use `ListFoundationModels` to confirm available models; check model access is enabled
`ModelTimeoutException`	—	Model invocation exceeded time limit	Retry with backoff; consider streaming API; reduce input size; use a smaller/faster model
Connection timeout/reset	—	Idle connections through NAT Gateway, VPC endpoint, or NLB	Enable TCP keep-alive to prevent idle connection drops

Retry and resilience patterns:

Pattern	Implementation
Exponential backoff with jitter	AWS SDK retry config: `max_attempts`, `mode: standard/adaptive`; custom decorator for additional control

Pattern	Implementation
Circuit breaker	Step Functions state machine: open circuit after N consecutive failures; route to fallback
Fallback model	AppConfig stores primary and fallback model IDs; Lambda switches on 5xx/throttle errors
Cross-Region inference	Bedrock inference profiles (Geographic or Global) for automatic regional routing

Error logging and request validation:

- Enable **Bedrock Model Invocation Logging** to CloudWatch Logs and/or S3.
- Log full prompt/response payloads for debugging (text, image, video, embedding data delivery configurable independently).
- Use **CloudWatch Logs Insights** to query prompt/response patterns for specific error scenarios.
- Use **AWS X-Ray** (Application Signals) to trace end-to-end request flow across Lambda, API Gateway, Bedrock, and other services.

Bedrock runtime metrics available in CloudWatch:

Metric	Description
`Invocations`	Total model invocations
`InvocationLatency`	Time from request to response
`InvocationClientErrors`	4xx errors
`InvocationServerErrors`	5xx errors
`InvocationThrottles`	Throttled requests
`InputTokenCount`	Tokens in the input
`OutputTokenCount`	Tokens in the output
`TimeToFirstToken`	Latency to first streaming token
`EstimatedTPMQuotaUsage`	Estimated tokens-per-minute quota utilization

> ***Exam tip:*** *API integration troubleshooting questions typically involve* `ThrottlingException` *(answer: exponential backoff + quota increase + Cross-Region inference),* `ValidationError` *(answer: validate request format + check token limits), or connectivity issues (answer: TCP keep-alive + VPC endpoint configuration).*

Prompt Engineering Problems (Skill 5.2.3)

When FM responses are low-quality, inconsistent, or off-target:

Problem	Diagnosis	Resolution
Inconsistent output format	Model sometimes returns JSON, sometimes plain text	Add explicit format instructions; use tool use / function calling for guaranteed JSON; add JSON schema in the prompt
Hallucinations	Model generates plausible but incorrect facts	Add "only use information from the provided context" instruction; enable Bedrock Guardrails contextual grounding; use RAG for factual grounding
Ignoring instructions	Model doesn't follow specific directions	Move instructions to the beginning of the system prompt; use explicit delimiters; reduce prompt complexity; test with fewer constraints

Problem	Diagnosis	Resolution
Low relevance	Model responds generically instead of specifically	Add more context; provide few-shot examples; use chain-of-thought prompting
Prompt confusion	Model misinterprets ambiguous instructions	Simplify language; break complex prompts into sequential chain; use Bedrock Prompt Flows for multi-step orchestration
Regression after changes	Quality drops after a prompt update	Compare outputs from old and new prompt versions using LLM-as-a-Judge; roll back and iterate

Prompt testing frameworks:

- **Bedrock Prompt Management** — version prompts; compare outputs across versions.
- **Bedrock Prompt builder** — console UI for iterative prompt testing.
- **Evaluation datasets** — maintain a golden dataset of prompt/expected-response pairs.
- **CloudWatch alarms** — monitor regression metrics after prompt changes (failure rate, unexpected output rate).

Systematic refinement workflow:

1. Identify the failure pattern (hallucination, format error, irrelevance).
2. Create test cases that reproduce the failure.
3. Iterate on the prompt (add constraints, examples, delimiters).
4. Run the full evaluation dataset to confirm the fix doesn't regress other cases.
5. Version the new prompt in Bedrock Prompt Management.
6. Deploy via CI/CD with automated quality gate.

> ***Exam tip:*** *"Improve FM response quality and consistency beyond basic prompt adjustments" points to prompt testing frameworks, version comparison with LLM-as-a-Judge, and systematic refinement workflows — not just rewriting the prompt.*

Retrieval System Issues (Skill 5.2.4)

Troubleshooting RAG pipeline problems:

Problem	Diagnosis	Resolution
Low retrieval relevance	Retrieved chunks don't answer the query	Check embedding model quality; try hybrid search (semantic + keyword); add reranking; adjust `topK` and similarity threshold
Embedding quality degradation	Retrieval quality drops over time	Monitor embedding drift; re-embed with updated model; verify chunking consistency
Stale data in vector store	Responses contain outdated information	Verify sync job status; implement incremental sync; add EventBridge triggers for real-time updates
Vectorization errors	Documents fail to embed or produce zero vectors	Check document format compatibility; validate preprocessing pipeline; review embedding model token limits vs. chunk size
Chunking misalignment	Important information split across chunks	Switch to semantic or hierarchical chunking; increase overlap tokens; test different chunk sizes
Slow vector search	High retrieval latency	Optimize OpenSearch sharding and index configuration; use approximate nearest-neighbor (HNSW); enable disk-based vector search for cost savings; consider quantization

Problem	Diagnosis	Resolution
Missing metadata filters	Retrieval returns results from wrong domain/context	Add metadata to documents during ingestion; use metadata filter expressions in `Retrieve`/`RetrieveAndGenerate` API calls
Poor reranking results	Reranker doesn't improve relevance	Verify reranker model is appropriate for the domain; check that enough candidates are passed to the reranker (increase initial `topK`)

Embedding quality diagnostics:

- Compare retrieval results for known queries against expected documents.
- Use the MTEB leaderboard to benchmark embedding models for your domain.
- Test multiple vector dimensions (Titan V2: 256/512/1024) — start compact, increase if needed.
- Verify the embedding model's max token input >= your chunk size.

Drift monitoring for retrieval systems:

- Track retrieval relevance scores over time in CloudWatch.
- Set alarms when average context relevance drops below a threshold.
- Schedule periodic re-evaluation using Bedrock RAG evaluation jobs.
- Monitor vector store index health (OpenSearch cluster metrics, index size, shard balance).

Vector search performance optimization:

- **OpenSearch HNSW tuning:** Increase `m` (connections per node) for better recall at the cost of memory; increase `ef_search` for better query accuracy at the cost of latency.
- **Quantization:** Use scalar or product quantization to reduce RAM; trades slight accuracy for significant cost reduction.
- **Disk-based vector search:** Keep compressed vectors in memory for candidate selection; retrieve full-precision from disk for final scoring.
- **OpenSearch auto-optimize:** Automatically evaluates configuration tradeoffs between search quality, speed, and cost.

> ***Exam tip:*** *Retrieval troubleshooting questions often describe symptoms like "irrelevant results" (answer: hybrid search + reranking + embedding quality check) or "outdated information" (answer: incremental sync + real-time change detection + scheduled refresh).*

Prompt Maintenance Issues (Skill 5.2.5)

Ongoing prompt management and debugging:

Problem	Diagnosis Tool	Resolution
Prompt confusion	CloudWatch Logs — analyze prompt/response pairs to identify where the model misinterprets instructions	Simplify prompt structure; add explicit section delimiters; split into prompt chain
Format inconsistencies	Schema validation — validate FM output against expected JSON schema	Add strict output format instructions; use tool use for structured output; add Lambda post-processing validation
Observability gaps	X-Ray prompt observability pipeline — trace the full request lifecycle to identify where quality degrades	Instrument with X-Ray/Application Signals; add OpenTelemetry GenAI attributes; enable Bedrock model invocation logging

Problem	Diagnosis Tool	Resolution
Template variable errors	Template testing — verify all variables are correctly populated before sending to the FM	Unit test template rendering; validate variable substitution in CI/CD
Cross-model inconsistencies	Same prompt produces different quality across models	Create model-specific prompt variants; use Bedrock Prompt Management to manage variants per model

Observability pipeline for prompts:

- **CloudWatch Logs Insights** — query patterns: search for specific prompt templates, filter by error responses, analyze token usage per prompt variant.
- **X-Ray / Application Signals** — end-to-end tracing with GenAI-specific attributes: `gen_ai.request.model`, `gen_ai.request.max_tokens`, `gen_ai.request.temperature`, `gen_ai.usage.input_tokens`, `gen_ai.usage.output_tokens`, `gen_ai.response.finish_reasons`.
- **Bedrock Model Invocation Logging** — capture full prompt/response payloads to CloudWatch Logs and S3; configure text, image, video, and embedding data delivery independently.
- **CloudWatch GenAI Observability dashboards** — pre-configured views for invocation count, latency, token counts by model, error counts, and throttling events.

Systematic prompt refinement workflow:

1. Enable model invocation logging to capture all prompt/response data.
2. Use CloudWatch Logs Insights to identify failure patterns.
3. Create targeted test cases for each failure pattern.
4. Iterate on the prompt in Bedrock Prompt builder (console).
5. Version the improved prompt in Bedrock Prompt Management.
6. Run the evaluation dataset to verify the fix; confirm no regressions.
7. Deploy through CI/CD pipeline with automated quality gate.

> ***Exam tip:*** *Prompt maintenance questions mention CloudWatch Logs (for prompt/response analysis), X-Ray (for tracing), and schema validation (for format consistency). The correct answer always includes both diagnosis tools and a systematic refinement process.*

Important Services Reference for Domain 5

Amazon Bedrock Evaluations

Feature	Description
Model Evaluation (LLM-as-a-Judge)	Judge model scores generator outputs on 11 built-in metrics; supports custom metrics
Model Evaluation (Programmatic)	Predefined NLP metrics (accuracy, robustness, toxicity)
Model Evaluation (Human)	Bring your own dataset; define custom metrics; use your team or AWS workers
RAG Evaluation (Retrieve only)	Evaluate retrieval quality: context relevance, context coverage
RAG Evaluation (Retrieve and generate)	Evaluate both retrieval and generation: all retrieval metrics + 10 generation metrics
BYOI (Bring Your Own Inference)	Evaluate custom RAG pipelines by providing your own input-output pairs and contexts
Compare feature	Side-by-side comparison of multiple evaluation jobs
Custom metrics	Write custom judge prompts with configurable rating scales and dynamic content injection

Feature	Description
Max prompts per job	1,000

Amazon Bedrock AgentCore Evaluations

Feature	Description
Online evaluation	Continuous monitoring of live agent traffic; configurable sampling and filtering
On-demand evaluation	Targeted assessment of specific traces/spans; useful for debugging and investigation
Built-in evaluators	Goal attainment, tool accuracy
Custom evaluators	Define domain-specific agent metrics
Framework support	Strands Agents, LangGraph, OpenTelemetry/OpenInference
Limits	1,000 eval configs/region; 100 active; 1M tokens/min

Amazon Bedrock Guardrails (Evaluation-Related)

Feature	Role in Testing
Contextual grounding checks	Detect and filter hallucinated responses using grounding + relevance scores
Automated Reasoning checks	Mathematical/formal verification of factual accuracy; up to 99% accuracy
Content filters	Test for harmful, stereotyped, or inappropriate content
Integration with RAG evaluation	Incorporate Guardrails into evaluation jobs for comprehensive safety testing

Observability Services

Service	Domain 5 Use Case
CloudWatch Metrics	Bedrock runtime metrics: invocations, latency, token counts, errors, throttles
CloudWatch Logs	Store and query prompt/response payloads via Logs Insights
CloudWatch GenAI Observability	Pre-configured dashboards for model invocations and AgentCore agents
CloudWatch Synthetics	Synthetic canaries for automated deployment validation tests
CloudWatch Application Signals	Fine-grained metrics at resource level (Model ID, Guardrails ID, KB ID, Agent ID) with GenAI attributes
AWS X-Ray	End-to-end request tracing across Lambda, API Gateway, Bedrock
Amazon Q Developer	GenAI-specific error pattern recognition for troubleshooting
Bedrock Model Invocation Logging	Full prompt/response logging to CloudWatch Logs and S3; configurable per modality
Bedrock Agent Traces	Step-by-step reasoning process traces (PreProcessing, Orchestration, PostProcessing, Failure, Guardrail)

Supporting Services

Service	Domain 5 Use Case
AWS Step Functions	Orchestrate evaluation workflows, quality gates, canary release logic, circuit breakers

Service	Domain 5 Use Case
AWS Lambda	Custom validation, output schema checks, evaluation scripts, synthetic test drivers
Amazon SageMaker Ground Truth	Managed annotation workflows for human evaluation of FM outputs
Amazon Augmented AI (A2I)	Human review workflows for FM outputs requiring expert validation
AWS CodePipeline / CodeBuild	CI/CD pipelines with integrated evaluation quality gates
AWS CodeDeploy	Canary and linear traffic shifting for deployment validation
AWS AppConfig	Feature flags for A/B testing and dynamic model/prompt variant routing
Amazon DynamoDB	Store feedback data, evaluation results, conversation history for analysis
Amazon S3	Central storage for evaluation datasets, model invocation logs, evaluation reports
Amazon QuickSight	Business-facing dashboards for evaluation metrics and trends

Exam Patterns and Common Distractors

Pattern: "How do you evaluate FM output quality beyond traditional ML metrics?"

Answer: LLM-as-a-Judge with Bedrock Model Evaluations — use built-in metrics (correctness, completeness, faithfulness, helpfulness) rather than ROUGE/BLEU. Combine with human evaluation for subjective quality.

Pattern: "How do you evaluate a RAG system's retrieval quality?"

Answer: Bedrock RAG evaluation (retrieve-only job) with context relevance and context coverage metrics. Context coverage requires ground truth annotations. For generation quality, use retrieve-and-generate job with faithfulness and citation precision/coverage metrics.

Pattern: "How do you detect and prevent hallucinations?"

Answer: Bedrock Guardrails contextual grounding checks (grounding + relevance scores), Automated Reasoning checks (formal verification, up to 99% accuracy), LLM-as-a-Judge faithfulness metric, and golden dataset comparisons in CI/CD.

Pattern: "How do you ensure a model update doesn't degrade quality?"

Answer: Deployment validation pipeline: run versioned evaluation dataset through LLM-as-a-Judge as a CI/CD quality gate; fail the build if scores drop below threshold. Use canary releases to validate with live traffic. Monitor hallucination rates and semantic drift with CloudWatch alarms.

Pattern: "How do you troubleshoot ThrottlingException from Bedrock?"

Answer: Implement exponential backoff with jitter (AWS SDK retry config). Request quota increase. Use Cross-Region inference profiles. Consider Provisioned Throughput for high-volume workloads.

Pattern: "How do you troubleshoot low-quality RAG responses?"

Answer: Diagnose: check retrieval quality separately from generation quality (use retrieve-only evaluation). Fix retrieval: try hybrid search (semantic + keyword), add reranking, adjust chunk size/overlap, verify embedding model

quality. Fix generation: improve prompt, add grounding instructions, enable contextual grounding checks.

Pattern: "How do you debug agent behavior?"

Answer: Enable Bedrock agent trace to view step-by-step reasoning (OrchestrationTrace), tool selection, and knowledge base queries. Use AgentCore Evaluations for automated assessment of goal attainment and tool accuracy. Use CloudWatch Transaction Search to investigate specific sessions.

Pattern: "How do you observe and trace GenAI application performance?"

Answer: Enable Bedrock Model Invocation Logging to CloudWatch Logs/S3. Use CloudWatch GenAI Observability dashboards for metrics. Use X-Ray / Application Signals for end-to-end tracing with OpenTelemetry GenAI attributes. Use CloudWatch Logs Insights for prompt/response analysis.

Pattern: "How do you continuously improve prompts in production?"

Answer: Systematic prompt refinement: enable model invocation logging -> analyze failures with CloudWatch Logs Insights -> create test cases -> iterate in Prompt builder -> version in Prompt Management -> validate with evaluation dataset in CI/CD -> deploy. Collect user feedback to feed failures back into the evaluation dataset.

Pattern: "Context window overflow — how do you resolve it?"

Answer: Token counting before invocation. Prompt compression (remove redundant context). Context pruning (recent turns only + summary of older turns). Dynamic chunking (reduce chunk size or number of retrieved chunks). Set `max_tokens` to reserve space for the response.

Quick-Reference: GenAI Evaluation Framework

```
Evaluation Dataset (versioned, expanded with real-world failures)
    |
    v
Offline Evaluation (CI/CD quality gate)
    |-- LLM-as-a-Judge (Bedrock Model Evaluation)
    |       |-- Quality: correctness, completeness, helpfulness, coherence
    |       |-- Faithfulness: grounding, hallucination detection
    |       |-- Responsible AI: harmfulness, stereotyping, refusal
    |       |-- Custom: brand voice, domain-specific rubrics
    |
    |-- RAG Evaluation (Bedrock Knowledge Base Evaluation)
    |       |-- Retrieval: context relevance, context coverage
    |       |-- Generation: faithfulness, citation precision, citation coverage
    |
    |-- Agent Evaluation (AgentCore Evaluations)
            |-- Goal attainment, tool accuracy, custom evaluators
    |
    v
Online Evaluation (live traffic)
    |-- A/B testing (AppConfig feature flags)
    |-- Canary releases (5% traffic -&gt; monitor -&gt; scale)
    |-- Shadow deployment (parallel comparison)
    |
    v
Human-in-the-Loop Evaluation
    |-- Explicit feedback (thumbs up/down, ratings)
```

```
    |-- Implicit feedback (copy, rephrase, abandon)
    |-- Expert annotation (SageMaker Ground Truth / A2I)
    |
    v
Feedback Loop (failures -&gt; new test cases -&gt; evaluation dataset)
```

Quick-Reference: Troubleshooting Decision Tree

```
FM Response Issue
    |
    |-- Is it an API error?
    |       |-- 429 ThrottlingException -&gt; Exponential backoff + Cross-Region inference
    |       |-- 400 ValidationError -&gt; Check request format + token limits
    |       |-- 403 AccessDenied -&gt; Check IAM permissions + model access
    |       |-- 503 ServiceUnavailable -&gt; Retry + failover region
    |       |-- Connection timeout -&gt; TCP keep-alive for VPC/NAT
    |
    |-- Is the response incomplete?
    |       |-- Context window overflow -&gt; Token counting + prompt compression
    |       |-- Truncated retrieval -&gt; Adjust chunking + increase overlap
    |       |-- Long conversation -&gt; Sliding window + conversation summarization
    |
    |-- Is the response inaccurate (hallucination)?
    |       |-- Enable Guardrails contextual grounding checks
    |       |-- Add RAG grounding ("use only provided context")
    |       |-- Evaluate faithfulness with LLM-as-a-Judge
    |       |-- Check retrieval quality (retrieve-only evaluation)
    |
    |-- Is the response irrelevant?
    |       |-- Check retrieval: hybrid search + reranking
    |       |-- Improve prompt: add few-shot examples + chain-of-thought
    |       |-- Check embedding model quality (MTEB benchmark)
    |
    |-- Is the format inconsistent?
            |-- Add explicit format instructions + JSON schema
            |-- Use tool use / function calling for structured output
            |-- Add Lambda post-processing validation
```

Domain 5 Key Numbers

Item	Value
Domain weight	11%
Bedrock evaluation max prompts per job	1,000
LLM-as-a-Judge metric score range	0 to 1
Automated Reasoning verification accuracy	Up to 99%
LLM-as-a-Judge cost savings vs. human evaluation	Up to 98%
AgentCore Evaluations max configs per region	1,000

Item	Value
AgentCore Evaluations max active configs	100
AgentCore Evaluations max tokens/min	1,000,000
Bedrock built-in LLM-as-a-Judge metrics	11
RAG retrieval-only metrics	2 (context relevance, context coverage)
RAG retrieve-and-generate metrics	10 (quality + responsible AI + citations)

AWS Well-Architected Framework: Generative AI Lens

Study guide synthesized from the official AWS Well-Architected Generative AI Lens (Publication date: November 19, 2025). Mapped to the AIP-C01 exam domains for targeted study. Source: https://docs.aws.amazon.com/wellarchitected/latest/generative-ai-lens/generative-ai-lens.html

Exam Relevance

The Well-Architected Generative AI Lens is explicitly referenced in the AIP-C01 exam guide under Task 1.1.3: "Create standardized technical components to ensure consistent implementation across multiple deployment scenarios (AWS Well-Architected Framework, WA Tool Generative AI Lens)."

Understanding this lens is relevant across all five exam domains:

Exam Domain	Relevant Pillars
D1: Foundation Model Integration, Data Management, and Compliance (31%)	Operational Excellence, Performance Efficiency, Reliability
D2: Implementation and Integration (26%)	Operational Excellence, Reliability, Performance Efficiency
D3: AI Safety, Security, and Governance (20%)	Security, Operational Excellence
D4: Operational Efficiency and Optimization (12%)	Cost Optimization, Performance Efficiency, Sustainability
D5: Testing, Validation, and Troubleshooting (11%)	Operational Excellence, Performance Efficiency, Reliability

Overview

The AWS Well-Architected Generative AI Lens extends the Well-Architected Framework to address the unique considerations of generative AI. It covers the entire generative AI lifecycle -- scoping, model selection, customization, development, deployment, and continuous improvement -- evaluated against the six Well-Architected pillars: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability.

Scope: Best practices for generative AI applications using foundation models on Amazon Bedrock, customer-managed models on Amazon SageMaker AI, and business applications with Amazon Q. Traditional ML workloads (non-generative) are covered by the separate Machine Learning Lens.

Intended audience: Architects, builders, security experts, MLOps engineers, and decision-makers involved in designing, developing, and operating generative AI applications on AWS.

Availability: The lens is available as a custom lens JSON file for import into the AWS Well-Architected Tool from the public AWS Well-Architected custom lens GitHub repository.

Design Principles

Six design principles apply across all generative AI workloads:

1. **Design for controlled autonomy:** Implement comprehensive guardrails and boundaries that govern how AI systems operate, scale, and interact. Establish clear operational requirements, security controls, and failure conditions to keep AI systems within safe, efficient, and cost-effective parameters while maintaining reliability.

2. **Implement comprehensive observability:** Monitor and measure specific aspects of your generative AI system, from security and performance to cost and environmental impact. Collect metrics across every layer -- user feedback, model behavior, resource utilization, and security events -- to maintain operational excellence while optimizing system behavior.

3. **Optimize resource efficiency:** Select and configure AI components based on empirical requirements rather than assumptions. Right-size models, optimize data operations, and implement dynamic scaling to balance performance needs with cost and sustainability goals.

4. **Establish distributed resilience:** Design systems that remain operational despite component or regional failures. Implement redundancy, automated recovery mechanisms, and geographic distribution of resources for consistent service delivery.

5. **Standardize resource management:** Maintain centralized catalogs and controls for critical components like prompts, models, and access permissions. Implement structured management systems for security, version control, and cost optimization.

6. **Secure interaction boundaries:** Protect and control data flows and system interfaces. Implement least-privilege access, secure communications, input/output sanitization, and comprehensive monitoring for system security and integrity.

Key Definitions

Term	Definition
Agent	An AI system that can perform tasks autonomously and interact with its environment to achieve specific goals
Chunking	Breaking up large data files into small, discrete chunks to fit data into a context window
Continuous pre-training	Continuously updating a pre-trained model with new data to improve performance and adapt to evolving domains
Embedding	Transforms chunks of data into vectors that represent semantic meaning
Fine-tuning	Adapting a pre-trained model to a specific task or domain by training on a smaller, task-specific dataset
Foundation model	Large language model pre-trained on vast amounts of data, serving as a foundation for downstream tasks
GenAIOps / LLMOps	Operational practices for managing the lifecycle of LLMs, including model selection, data prep, deployment, monitoring, and governance
Hallucination	When a generative AI model produces outputs that are inconsistent, factually incorrect, or unrelated to the input prompt
Indexing	Process of inserting embedded chunks into a vector data store
Model card	Document providing key information about an ML model including intended use, training data, performance, and limitations
Model customization	Modifying a foundation model using various techniques to control its behavior
Model distillation	Creating a smaller, more efficient model that mimics a larger model's behavior
Model gateway	An interaction layer offering secure access to the model hub through standardized APIs

Term	Definition
Model hub	Central repository providing access to enterprise foundation models from first-party, third-party, and open-source providers
Model orchestration	Encapsulation of multistep workflows characteristic of generative AI workflows
Prompt catalog	Centralized repository for storing, managing, and versioning prompts
Provisioned throughput	Amazon Bedrock feature providing higher throughput at fixed cost for predictable workloads
Quantization	Reducing precision of model parameters to decrease memory footprint and computational requirements
RAG (Retrieval-Augmented Generation)	Augmenting an LLM's output with relevant information retrieved from a document corpus to ground responses and reduce hallucination
Tokenization	Breaking input text into smaller units (tokens) as a preprocessing step
Vector store	Specialized data store for efficient storage and retrieval of high-dimensional vector embeddings

Responsible AI

AWS defines Responsible AI along eight dimensions that are assessed and updated over time:

1. **Fairness:** Considering impacts on different groups of stakeholders. Implement robust testing frameworks to detect potential bias and regularly audit outcomes across different segments.
2. **Explainability:** Understanding and evaluating system outputs. Implement interpretability techniques and translate complex model decisions into understandable explanations.
3. **Privacy and security:** Appropriately obtaining, using, and protecting data and models. Implement sophisticated data governance frameworks including encryption, access controls, and data minimization practices.
4. **Safety:** Reducing harmful system output and misuse. Implement comprehensive safety frameworks including content filtering, output validation, and clear escalation paths.
5. **Controllability:** Having mechanisms to monitor and steer AI system behavior. Implement robust monitoring systems that track performance metrics, user feedback, and system outputs.
6. **Veracity and robustness:** Achieving correct system outputs, even with unexpected or adversarial inputs. Implement comprehensive testing frameworks and automated reasoning to capture and correct hallucinations.
7. **Governance:** Incorporating best practices into the AI supply chain, including providers and deployers. Establish AI governance committees with technical, business, and risk perspectives.
8. **Transparency:** Enabling stakeholders to make informed choices about their engagement with an AI system. Clearly communicate when and how AI is being used.

Generative AI Lifecycle

The generative AI lifecycle consists of six key phases, each evaluated against all six Well-Architected pillars:

Phase 1: Scoping

Prioritize understanding the business problem. Define goals, requirements, and use cases. Key considerations:

- Determine if generative AI is the right approach for the problem
- Assess risks and costs of investment
- Identify model requirements (off-the-shelf vs. custom, single vs. orchestrated models)
- Establish success metrics for model performance evaluation
- Determine technical and organizational feasibility
- Develop a comprehensive risk profile (technology and business risks)
- Create security scoping matrices for different use cases
- Assess data availability and quality for model customization

Phase 2: Model Selection

Evaluate models based on requirements and use cases. Key factors:

- Modality, size, accuracy, training data, pricing, context window, inference latency
- Compatibility with existing infrastructure
- Data usage policies by hosting providers
- Instance type evaluation for SageMaker AI hosting
- Vector database selection and availability for RAG workloads
- Hosting options: batch inference, real-time inference, or combined inference profiles
- Model routing solutions and model catalogs for flexibility

Phase 3: Model Customization

Align the model with application goals through iterative refinement:

- **Prompt engineering:** Craft prompts to guide desired outputs; implement template management
- **RAG:** Augment with retrieval from external knowledge bases
- **Fine-tuning:** Train on domain-specific data for improved performance
- **Continuous pre-training:** Update with new domain data
- **Model distillation:** Create smaller efficient models from larger teacher models
- **Human feedback alignment:** Refine behavior based on human input

Phase 4: Development and Integration

Integrate the model into applications and systems:

- Implement conversational interfaces, prompt catalogs, agents, and knowledge bases
- Connect to databases, data pipelines, and enterprise applications
- Implement guardrails to reduce risks like hallucination
- Optimize for real-time inference performance
- Build APIs for model interaction
- Design user-friendly interfaces
- Validate with automated testing
- Establish monitoring systems

Phase 5: Deployment

Roll out the solution in a controlled manner:

- Implement CI/CD pipelines using CodePipeline, CodeBuild
- Use IaC tools (AWS CDK, CloudFormation, Terraform) for resource management

- Maintain version control and automated pipelines
- Validate security and privacy requirements
- Enable quick rollbacks through documentation and versioning

Phase 6: Continuous Improvement

Ongoing monitoring and refinement:

- Track key metrics (accuracy, toxicity, coherence)
- Gather user feedback on quality and usefulness
- Update training data based on feedback
- Experiment with new techniques, algorithms, and architectures
- Regularly reassess and update AI strategy

Data Architecture

Three primary use cases for data in generative AI, each with distinct architectural requirements:

Pre-training data: Petabyte-scale diverse datasets requiring highly scalable infrastructure. Challenges include data quality across diverse sources, efficient storage/retrieval, and computational resources for processing. Must address data versioning, privacy protection, and sustainable long-term practices.

Fine-tuning / customization data: Smaller, focused datasets adapted to specific tasks/domains. Requires flexible architectures for varying data sizes. Challenges include data selection/curation, quality/relevance, and bias reduction. Must support rapid iteration and careful versioning.

RAG data: Dynamic retrieval from external knowledge bases combined with model inference. Demands low-latency retrieval systems and seamless integration. Must address efficient indexing of large knowledge bases, real-time retrieval, and up-to-date information.

Strategic imperatives:

- Data quality as the foundation (automated validation, cleansing, continuous monitoring)
- Unified data architecture (break down silos, integrate structured and unstructured data)
- Privacy-first approach (differential privacy, federated learning, synthetic data generation)

Agentic AI

AI agents are autonomous entities capable of perceiving their environment, reasoning, and taking actions to achieve defined goals. The core of an agentic system is usually an LLM augmented with retrieval, tools, and memory. Agency exists on a spectrum -- only increase agency when task complexity requires it.

Three common agentic patterns:

1. **LLM-augmented workflows:** Largely deterministic code paths with certain steps augmented

by LLMs for decision-making. Low degree of agency. Example: document classification and routing system using tool calls.

2. **Autonomous agents:** Built around an LLM with retrieval, tools, and memory orchestrated

in a ReACT (reason and act) loop. The agent is given tools and instructions, then loops until given a stop reason. Tools connected via MCP protocol. System prompt defines goal, role, and reasoning guidelines.

3. **Hybrid (Plan and Solve):** Combines workflows and autonomous agents with high agency.

An LLM creates a plan and task breakdown, a queue of tasks is resolved by ReACT agents, then another LLM decides if the result is sufficient or needs more tasks. Scales with queues, auto-scaling, and asynchronous execution.

Pillar 1: Operational Excellence

Operational excellence for generative AI focuses on achieving consistent model output quality, monitoring operational health, maintaining traceability, automating lifecycle management, and determining when to customize models.

Principles:

- Implement comprehensive observability across all layers (application, service, foundation model)
- Automate operational management using IaC and automated lifecycle processes
- Establish operational controls (prompt template management, rate limits, workflow tracing)

GENOPS01: Model Output Quality

How do you achieve and verify consistent model output quality?

BP01: Periodically Evaluate Functional Performance (Risk: High)

Implement periodic evaluations using stratified sampling and custom metrics to maintain performance and reliability. Regularly assess against ground truth data and specific evaluation criteria.

Key implementation details:

- Run ground truth data periodically, applying sampling techniques for monitoring
- Employ stratified sampling (divide data into relevant categories, randomly sample each)
- Periodically update ground truth datasets as inputs and usage evolve
- Address data drift where actual usage diverges from initial ground truth

AWS services and tools:

- Amazon Bedrock model evaluation feature (built-in)
- Amazon SageMaker AI (manual evaluations via Studio, automatic evaluations via fmeval library)
- Open-source libraries: fmeval, ragas
- Amazon Bedrock model invocation logging for metadata, requests, and responses
- Amazon CloudWatch for metrics
- AWS Step Functions for workflow automation

Organization considerations: AI policy should define minimum performance levels and validation cadence. Identify a single-threaded workload owner for performance evaluations. Run evaluations when new candidate models are available or after model customization.

BP02: Collect and Monitor User Feedback (Risk: High)

Supplement model evaluation with direct user feedback. Implement continuous feedback loops (thumbs up/down, ratings) to detect issues early and inform prompt engineering.

AWS services and tools:

- Amazon Q Business: analytics dashboard (usage trends, conversations, query trends, feedback),

PutFeedback API, conversation logging (S3, CloudWatch Logs, or Amazon Data Firehose)

- Amazon Bedrock: S3 for feedback storage, Lambda for processing, EventBridge for triggers
- Amazon DynamoDB for scalable, low-latency feedback storage
- Amazon QuickSight for feedback visualization dashboards
- AWS Step Functions for feedback processing pipelines
- A/B testing for validating improvements

GENOPS02: Operational Health

How do you monitor and manage operational health?

BP01: Monitor All Application Layers (Risk: High)

Implement comprehensive monitoring and logging across all layers:

- **Application layer:** Software abstraction above the foundation model
- **Service layer:** Gateway that negotiates prompts and brokers responses (may interact with prompt catalog, vector store, or guardrails)
- **Foundation model layer:** Responds back through the service layer; complex workloads may navigate knowledge graphs, run prompt flows, or initiate agents

AWS services and tools:

- Amazon Bedrock, Amazon Q Business, Amazon OpenSearch Service Serverless (managed monitoring integrated with CloudWatch and CloudTrail)
- Amazon SageMaker AI Inference Endpoints (logs to CloudWatch)
- SageMaker AI HyperPod (EKS): one-click observability with Amazon Managed Grafana, NVIDIA DCGM metrics, EFA metrics, CloudWatch Container Insights
- AWS X-Ray or Amazon Bedrock Agent tracing for service layer
- Amazon Data Firehose for log ingestion, Amazon Athena for ad-hoc log querying
- Amazon GuardDuty for threat detection

BP02: Monitor Foundation Model Metrics (Risk: High)

Set up continuous monitoring and alerting for foundation models covering performance, security, and cost-efficiency.

Key metrics to track:

Service	Metrics
Amazon Bedrock	Invocation counts, latency, InputTokenCount, OutputTokenCount, InvocationThrottles, error rates
Amazon SageMaker AI	Invocation counts, latency, error rates, GPU and memory utilization
SageMaker AI HyperPod	NVIDIA DCGM (GPU health), EFA (network), system metrics (CPU, memory, disk)

Implementation pattern:

1. Enable model invocation logging (S3 or CloudWatch Logs)
2. Set up CloudWatch dashboards with anomaly detection
3. Create alarms for critical thresholds (elevated latency, high error rates/throttling)
4. Implement EventBridge rules to capture events and trigger SNS/Lambda responses
5. Develop incident response playbooks
6. Establish regular review process

BP03: Mitigate Risk of System Overload (Risk: High)

Two primary approaches:

1. **Scale inference serving architecture:** Use Amazon SageMaker AI Inference Components to host and scale models independent of underlying infrastructure. Models are treated as primary elements, scaling hardware based on CPU/GPU resources and inference load.
2. **Rate limit and throttle managed inference:** Control request processing rates. Amazon

Bedrock has default quotas for API request rates, token usage, and concurrent invocations. Increase throughput via cross-Region inference or provisioned model throughput.

Implementation patterns:

- Amazon API Gateway for rate limiting
- Exponential backoff with jitter (AWS SDK built-in retry mechanisms)
- AWS Step Functions for complex retry workflows with circuit breaker patterns
- SageMaker AI HyperPod: Kubernetes resource quotas, horizontal pod autoscaling,

HyperPod Task Governance with intelligent throttling

- Continuous monitoring and regular load testing

GENOPS03: Traceability and Auditability

How do you maintain traceability for models, prompts, and assets?

BP01: Implement Prompt Template Management (Risk: High)

Maintain a versioned prompt template management system for consistent, optimized performance.

Amazon Bedrock Prompt Management capabilities:

- Create, edit, version, and share prompts across teams
- Components: prompts, variables, variants, visual builder interface
- Integration into applications by specifying prompts during model inference

Amazon Bedrock Flows capabilities:

- Create and manage advanced workflows without writing code
- Visual builder linking foundation models, prompts, agents, knowledge bases
- Supports versioning, rollback, and A/B testing

Key practices:

- Version prompts including hyperparameters or ranges
- Test and compare prompt variants
- Capture baseline metrics and validate for deviations
- Establish cross-team collaboration and guidelines

BP02: Enable Tracing for Agents and RAG Workflows (Risk: High)

Implement comprehensive tracing for visibility into agent decision-making.

Amazon Bedrock Agent trace components:

- **PreProcessingTrace:** How the agent contextualizes and categorizes user input
- **OrchestrationTrace:** How the agent interprets input, invokes action groups, queries

knowledge bases

- **PostProcessingTrace:** How the agent handles final output
- **FailureTrace:** Reasons for step failures
- **GuardrailTrace:** Actions taken by guardrails

Amazon AgentCore:

- Supports agent tracing by default
- Captures and visualizes traces and spans for each workflow step
- Supports OpenTelemetry for integration with CloudWatch, Datadog, LangSmith, Langfuse

Multi-agent optimization:

- Use supervisor with routing mode to route information directly to appropriate collaborator

agents, reducing unnecessary data transfers and latency

GENOPS04: Lifecycle Management

How do you automate lifecycle management?

BP01: Automate with Infrastructure as Code (Risk: High)

Use IaC tools for consistent, version-controlled, automated infrastructure deployment.

Tools: AWS CDK, AWS CloudFormation, or Terraform for defining and managing infrastructure resources (Amazon Bedrock, API Gateway, Lambda, data pipelines).

Key practices:

- Store IaC templates in Git version control
- Implement CI/CD pipelines using AWS CodePipeline
- Use same templates with different parameters for dev/test/production
- AWS Config for resource configuration tracking
- Service Catalog for standardized resource provisioning
- Regular security audits of IaC templates

BP02: Implement GenAIOps (Risk: High)

GenAIOps is a specialized subset of MLOps focusing on operationalizing foundation models.

Two categories:

1. **Operationalizing foundation model consumption:** Applications follow traditional DevOps

patterns but with complex orchestration (RAG, agents). Involves vector database selection, indexing pipelines, and retrieval strategies.

2. **Operationalizing foundation model training/tuning (FMOps/LLMOps):** Model selection,

continuous tuning/training, experiment tracking, central model registry, prompt management/evaluation, and deployment.

AWS services:

- Amazon SageMaker AI Pipelines (serverless workflow orchestration for MLOps/LLMOps)
- SageMaker AI MLflow or self-managed MLflow for experiment tracking and model cataloging
- Amazon Bedrock for managed RAG (Knowledge Bases), Agents, fine-tuning, continued pre-training
- AWS CodePipeline, CodeBuild, CodeDeploy for CI/CD integration

GENOPS05: Model Customization

How do you determine when to customize models?

BP01: Learn When to Customize Models (Risk: High)

Start with the least resource-intensive option and progressively move to more advanced methods:

Approach	When to Use	Effort
Prompt engineering	First approach -- well-crafted prompts can often achieve desired results	Low
RAG	When the model needs access to external knowledge sources	Medium

Approach	When to Use	Effort
Fine-tuning	Specific task requiring improved performance, labeled data available, domain-specific language needed	High
Continuous pre-training	Domain adaptation with more data than fine-tuning requires	Higher
Custom foundation model	No pre-trained models meet requirements, vast proprietary data, need complete control	Highest

AWS services:

- Amazon Bedrock: built-in evaluation tools, managed RAG, agents, fine-tuning, continued pre-training
- Amazon SageMaker AI with HyperPod: distributed training with data and model parallelism
- Amazon OpenSearch Service for vector databases in RAG workflows
- SageMaker AI Model Monitor for ongoing effectiveness tracking

Pillar 2: Security

Security for generative AI focuses on protecting endpoints, mitigating harmful outputs, monitoring/auditing events, securing prompts, controlling excessive agency, and preventing data poisoning.

Principles:

- Implement comprehensive access controls with least privilege across all components
- Secure data and communication flows (private networks, input sanitization, governed data access)
- Monitor and enforce security boundaries across control and data planes
- Control AI system behaviors with guardrails and boundaries

GENSEC01: Endpoint Security

How do you manage access to generative AI endpoints?

BP01: Grant Least Privilege Access to Foundation Model Endpoints (Risk: High)

Establish identity-based security for generative AI workloads.

Key implementation details:

- Use IAM to limit access to foundation model endpoints to specific IAM roles
- Grant roles least privilege access with session durations and permissions boundaries
- Control model access at the organization layer through service control policies (SCPs),

resource control policies, session policies, and permission boundaries

- Block or restrict unapproved models by accounts, Regions, or organization

For SageMaker AI HyperPod: Two primary roles -- cluster admin users (create, configure, manage clusters) and data scientist users (run ML workloads, submit jobs). Each role should have only necessary permissions.

For Amazon Q Developer: Subscription model -- confirm user needs and subscription level matches required access.

BP02: Implement Private Network Communication (Risk: High)

Reduce the attack surface by keeping communication off the public internet.

Key implementation details:

- AWS PrivateLink for private communication between VPCs and AWS generative AI services (Amazon Bedrock, Amazon Q family)
- SageMaker AI inference endpoints deployed in VPC can be made private via PrivateLink
- SageMaker AI HyperPod: deploy clusters within private VPC with appropriate subnets and security groups (both EKS and Slurm)
- Extend private network access to supporting infrastructure (vector stores, external tools for agents)

BP03: Least Privilege Access to Data Stores (Risk: High)

Treat generative AI systems like privileged users when granting data access.

Key implementation details:

- Deploy data stores in VPC with strong access controls and zero-trust principles
- Use temporary, least privilege credentials for application access
- Implement guardrails in Amazon Bedrock and SageMaker AI for granular control
- Apply data obfuscation and anonymization for model training data
- Use metadata filtering in vector stores and knowledge bases for granular access control
- Amazon Bedrock Knowledge Bases for data access abstraction
- Sanitize data before ingestion -- remove prohibited material before the model accesses it
- Test with curated prompts to confirm models cannot access sensitive information

BP04: Implement Access Monitoring (Risk: High)

Monitor access to generative AI services and foundation models for unauthorized use.

Key implementation details:

- AWS CloudTrail for service-level API access monitoring
- Amazon Bedrock model invocation logging (metadata, requests, responses)
- Amazon Q Developer: user activity capture in settings
- Amazon Q Business: log delivery to S3, CloudWatch Logs, or Amazon Data Firehose
- SageMaker AI HyperPod: CloudWatch Logs, CloudWatch Alarms, AWS Security Hub
- Log both application name and end-user for each request
- Agentic workloads require additional logging for each agent called
- Implement guardrails to mask/remove sensitive data before invocations

GENSEC02: Response Validation

How do you stop applications from generating harmful, biased, or incorrect responses?

BP01: Implement Guardrails to Mitigate Harmful or Incorrect Responses (Risk: High)

Guardrail techniques (layered approach):

Technique	Description
Prompt engineering	Chain-of-thought, logic-of-thought, few-shot prompting to encourage reasoning
RAG architectures	Ground responses with factual information from knowledge bases
Amazon Bedrock Guardrails	Filter on content, topic, or sensitive information; define fallback behavior
Amazon Q Business admin controls	Block certain phrases and topics

Technique	Description
Human review	Greatest confidence for the most critical responses
Model-as-a-judge	Foundation models classify responses and take action
Agent flows	Route response review to appropriate process

Amazon Bedrock Guardrails configuration options:

- Content filter sensitivity levels
- Denied topics
- Word filters (allowlists and blocklists)
- Sensitive information filters for PII
- Automated reasoning checks for contextual grounding and response relevance

Amazon Bedrock Guardrails SDK is available for custom self-hosted generative AI workloads.

GENSEC03: Event Monitoring

How do you monitor and audit events?

BP01: Control Plane and Data Access Monitoring (Risk: High)

Implement comprehensive monitoring across six dimensions:

1. **Performance monitoring:** Response times, latency, throughput, resource utilization, token usage, batch processing efficiency, model loading/unloading times
2. **Quality and accuracy monitoring:** Completion rates, response quality scores, content safety, hallucination rates, prompt effectiveness
3. **Security monitoring:** Auth attempts, prompt injection exploits, access patterns, rate limiting/quota usage, data leakage
4. **Cost monitoring:** Token usage/costs, resource utilization costs, API call volumes, storage/transfer costs, model deployment/training costs
5. **Audit trail implementation:** Detailed request/response logs, user interaction logs, model version changes, configuration modifications
6. **Compliance monitoring:** Data retention compliance, PII handling, regulatory requirements, consent management, geographic data restrictions

GENSEC04: Prompt Security

How do you secure system and user prompts?

BP01: Implement a Secure Prompt Catalog (Risk: Medium)

Use Amazon Bedrock Prompt Management for centralized, identity-protected prompt storage.

Key practices:

- Secure prompt catalog access with IAM policy documents
- Develop roles with least privilege access to prompt actions (CreatePromptVersion, GetPrompt)
- Enforce separation of duties between prompt engineering and agent workflow testing
- Use automated prompt optimization before cataloging
- Extend CI/CD workflows to incorporate prompt engineering
- Use Amazon Bedrock Flows for testing prompts in orchestrated manner

BP02: Sanitize and Validate User Inputs (Risk: High)

Protect against prompt injection and improper content.

Defense techniques:

- Add abstraction layer between user prompt and foundation model
- Create context boundaries in prompt templates (explicit instructions to maintain safety constraints)
- Search for suspicious keywords
- Scan user-influenced prompts with a guardrails solution
- Use a separate LLM-as-a-judge to confirm prompt safety
- Set character and token size limits on prompts
- Set rate limits on requests
- Amazon Bedrock Guardrails with allowlists and blocklists
- Continually refine guardrails based on testing results

GENSEC05: Excessive Agency

How do you avoid excessive agency for models?

BP01: Least Privilege Access for Agentic Workflows (Risk: High)

Excessive agency is an OWASP Top 10 security threat for LLMs, introduced through agentic architectures.

Key implementation details:

- Develop permissions boundaries on foundation model requests and agentic workflows
- Amazon Bedrock Agents have execution roles; Bedrock Flows have service roles -- both

need least privilege access

- Create separate developer roles (prompt engineer vs. security engineer)
- Define permission boundaries setting maximum permissions for roles
- Share permission boundaries across models in multi-model/multi-agent scenarios
- Implement user confirmation for agent actions to mitigate excessive agency
- Consider Amazon Verified Permissions for fine-grained authorization

GENSEC06: Data Poisoning

How do you detect and remediate data poisoning risks?

BP01: Implement Data Purification Filters (Risk: High)

Data poisoning occurs during pre-training, domain adaptation, and fine-tuning when inappropriate data is introduced into training. Best handled at the data layer before training begins.

Techniques:

- Amazon Transcribe Toxicity Detection for voice data
- Amazon Bedrock Guardrails API (ApplyGuardrail) for text data filtering
- fmeval or SageMaker AI Studio model evaluation for post-training assessment
- Isolate model training environment, infrastructure, and data
- SageMaker AI HyperPod (EKS): containerized data validation services in Kubernetes jobs
- SageMaker AI HyperPod (Slurm): data purification scripts as prerequisite batch jobs
- Log and monitor the filtering process to catch anomalies

Pillar 3: Reliability

Reliability focuses on managing throughput requirements, maintaining reliable communication, implementing recovery actions, versioning artifacts, distributing inference, and ensuring fault-tolerant distributed computation.

Principles:

- Design for distributed resilience across multiple Regions and Availability Zones
- Implement robust error management with automated recovery
- Standardize resource management through centralized catalogs
- Architect for intelligent scalability based on actual utilization patterns

GENREL01: Throughput Management

How do you determine throughput quotas for foundation models?

BP01: Scale and Balance Foundation Model Throughput (Risk: Medium)

Collect utilization data and implement dynamic scaling to match capacity with demand.

Key implementation details:

- Implement request buffering using message queue services to smooth irregular traffic
- Use token bucket algorithms for rate limiting
- Amazon Bedrock Provisioned Throughput for dedicated infrastructure with stable throughput
- Amazon Bedrock cross-Region inference profiles for distributing requests
- SageMaker AI Inference Endpoints with EC2 Auto Scaling groups behind load balancers
- Monitor Provisioned Throughput capacity in Amazon CloudWatch
- Implement retry mechanisms with exponential backoff
- Proactively engage AWS support to increase service quotas

GENREL02: Network Reliability

How do you maintain reliable communication among architecture components?

BP01: Implement Redundant Network Connections (Risk: Medium)

Create multiple network paths between critical components.

Key implementation details:

- Multi-AZ deployments across at least two Availability Zones
- AWS PrivateLink for secure private communications
- Deploy vector databases (OpenSearch Service Serverless) across multiple AZs with VPC Endpoints
- Extend multi-AZ to agentic workflows (Lambda functions in multi-AZ)
- Multi-Region: VPC peering, AWS Transit Gateway, or Amazon VPC Lattice for cross-Region connectivity
- Amazon CloudFront with latency-based routing
- Amazon Route 53 with latency-based routing within VPC
- Combine with Amazon Bedrock cross-Region inference capabilities
- Regular failover drills and failure simulations

GENREL03: Remediation and Recovery

How do you implement remediation for loops, retries, and failures?

BP01: Manage Prompt Flows and Recover from Failure (Risk: Medium)

Leverage conditions, loops, and logical structures for reliable prompt flows.

Key implementation details:

- Create abstraction layers between users and models for retries, error handling, graceful failures
- Implement logic checks to verify prompts contain expected information
- Verify model responses meet expectations before returning to users
- For external data sources: verify relevant data exists; define fallback actions
- For agentic workflows: classify responses as actionable or inactionable; handle errors

(error codes, empty responses) with appropriate agent prompts

- Amazon Bedrock Flows with iterator and condition nodes for flow logic

Error classification system:

- Categorize failure types with severity levels
- Create response templates for each category
- Implement circuit breaker patterns
- Track recovery success rates

BP02: Implement Timeout Mechanisms on Agentic Workflows (Risk: High)

Control long-running unexpected workflows.

Key implementation details:

- Set runtime timeouts on external infrastructure (Lambda functions, API endpoints)
- Account for edge cases (cold starts) when setting timeout values
- Connect agentic workflows to event systems (EventBridge) for asynchronous management
- Implement dead letter queues for timed-out processes
- Ensure error handling is transparent to users while maintaining system security
- Monitor timeout frequency and alert on repeated timeouts

GENREL04: Prompt and Model Management

How do you maintain versions for prompts, model parameters, and foundation models?

BP01: Implement a Prompt Catalog (Risk: Medium)

Centralized versioned storage for prompts.

Key practices:

- Release prompts to live version only after passing appropriate testing
- Enable rollback to previous versions for unexpected/undesirable behavior
- Maintain versioned hyperparameter ranges (temperature, top_p, top_k) paired with prompt versions
- Maintain test results for prompts against several model versions
- Define prompt metadata schema (version, author, purpose)
- Establish naming conventions, tagging, and access control
- Implement approval workflows and audit trails
- Codify governance in AI usage/policy document

AWS services: Amazon Bedrock Prompt Management

BP02: Implement a Model Catalog (Risk: Low)

Centralized versioned storage for models.

Key practices:

- Maintain first-party, third-party, and custom models in centralized catalog
- Record and track model access, versions, and model card information
- Implement model cards (capabilities, limitations, training data, intended use, ethical considerations)
- Establish approval workflows, deployment procedures, deprecation policies
- Maintain performance benchmarks

AWS services: Amazon Bedrock API, Amazon Bedrock Marketplace, Amazon Bedrock model catalog

GENREL05: Distributed Availability

How do you distribute inference workloads across Regions?

BP01: Load-Balance Inference Requests Across Regions (Risk: Medium)

Use multi-Region deployment for highly available inference.

Key implementation details:

- Amazon Bedrock cross-Region inference profiles for routing to nearest available endpoint
- SageMaker AI Inference Endpoints with multi-AZ deployment and auto-scaling
- Health checks and automated failover
- Monitor latency, error rates, and throughput across Regions

BP02: Replicate Embedding Data Across Regions (Risk: Medium)

Ensure data availability across all Regions of availability.

Key implementation details:

- Amazon S3 cross-Region replication for embedding data
- Amazon OpenSearch Service cross-cluster replication
- AWS Glue data pipelines for synchronization
- Consider data sovereignty requirements and regulatory restrictions
- Amazon S3 Intelligent Tiering for cost-effective storage
- Monitor for replication issues and compliance violations

BP03: Verify Agent Capabilities Across Regions (Risk: Medium)

Ensure supporting infrastructure is available wherever agents operate.

Key implementation details:

- Deploy APIs across multiple Regions behind CloudFront with latency-based routing
- Amazon Route 53 with latency-based routing for VPC traffic
- Configure model access in all required Regions
- Verify agents can access required resources in all Regions

GENREL06: Distributed Compute Tasks

How do you design high-performance distributed computation for successful completion?

BP01: Design for Fault-Tolerance (Risk: High)

Model pre-training, fine-tuning, and distillation require orchestrating dozens to hundreds of machines running for days, weeks, or months.

Key implementation details:

- Amazon SageMaker AI HyperPod: managed infrastructure with automatic fault detection and recovery
  - EKS-based: application-level checkpointing, shared persistent volumes, Kubernetes health checks
  - Slurm-based: auto-resume for zero-touch resiliency, multi-head node support
  - Both: built-in GPU health monitoring (DCGM), EFA health checks, automatic node replacement
- Amazon Bedrock: managed fine-tuning, continued pre-training, model distillation
- AWS Batch: distributed tasks with automatic scaling, retry logic, resource optimization
- Amazon FSx for Lustre for rapid access to large data volumes
- Amazon S3 for checkpoint storage
- CloudWatch for alerts, EventBridge for automated remediation

Pillar 4: Performance Efficiency

Performance efficiency focuses on establishing evaluation processes, maintaining model performance, optimizing compute resources, and improving data retrieval performance.

Principles:

- Measure and validate performance systematically with comprehensive testing frameworks
- Optimize model and vector operations based on empirical performance requirements
- Leverage managed services for operational efficiency

Key performance metrics:

- Inference latency (time to generate a response)
- Throughput (concurrent requests without degradation)
- Response quality (accuracy, relevance, coherence against ground truth)
- Resource utilization (CPU, memory, GPU efficiency)
- Availability (system responsiveness percentage)

GENPERF01: Performance Evaluation

How do you capture and improve model performance?

BP01: Define a Ground Truth Dataset (Risk: Medium)

Ground truth data (golden dataset) facilitates model testing for use-case-specific scenarios.

Key implementation details:

- Curate sample prompts and expected responses (dozens to thousands depending on complexity)
- Group prompts by variations of the same ask
- Treat as a living artifact that changes with use cases
- For agent workflows: capture ideal prompt flows tracing through various systems
- Golden datasets also useful for fine-tuning and model distillation

AWS services: Amazon SageMaker Ground Truth, Amazon S3 (hierarchical storage), AWS Glue Crawler (data dictionary), Amazon Athena (segment querying), Amazon SageMaker JumpStart

BP02: Collect Performance Metrics (Risk: Medium)

Foundation model performance on specific tasks is measured differently depending on the desired outcome. Inaccurate or toxic responses may be considered under-performing.

Key implementation details:

- Traditional metrics (latency, throughput) plus response quality dimensions
- Report metrics, telemetry, and logs to Amazon CloudWatch
- Use OpenLLMetry trace frameworks for detailed tracing
- Benchmark with golden datasets and external benchmarking datasets
- Consult model cards for strengths and weaknesses
- Track experiments with Amazon SageMaker AI with MLflow
- Establish performance thresholds and configure CloudWatch alarms

GENPERF02: Maintaining Model Performance

How do you verify acceptable performance levels?

BP01: Load Test Model Endpoints (Risk: Medium)

Determine baseline performance under average and extreme throughput.

Key implementation details:

- Develop test suites simulating heaviest expected load against ground truth data
- Amazon Bedrock: review published metrics; benchmark with golden datasets; consider provisioned throughput or cross-Region inference for throughput limitations
- Amazon SageMaker AI: test with respect to instance type/size; use quantization, LoRA; LMI deep learning containers (request batching, vLLM support)
- Performance optimization: prompt caching (Bedrock), streaming responses, batch inference

BP02: Optimize Inference Parameters (Risk: Low)

Configure hyperparameters to improve response quality.

Key parameters:

- **Temperature:** Controls randomness (lower = more deterministic, higher = more creative)
- **Top-p (nucleus sampling):** Cumulative probability cutoff for token selection
- **Top-k:** Number of top tokens considered for selection

Optimization approach:

- Test highest and lowest values, compare to golden data
- Use Newtonian approach (halving increments) to find ideal values
- LLM-as-a-judge pattern for evaluating performance at scale
- Document recommended hyperparameter ranges by task in AI policy

BP03: Select and Customize the Appropriate Model (Risk: Medium)

Choose the right model family and size for your use case.

Key implementation details:

- Amazon Bedrock Evaluations / fmeval for testing against evaluation datasets
- Amazon Bedrock Prompt Routing for dynamic routing within a model family when testing is inconclusive
- Model customization progression: fine-tuning (small labeled data), continuous pre-training

(more data, domain adaptation), model distillation (synthetic data from teacher to student)

- Track dominant model family per workload in AI usage document

GENPERF03: High-Performance Compute Optimization

How do you optimize computational resources?

BP01: Use Managed Solutions Where Appropriate (Risk: Medium)

Hosting options (increasing control, increasing operational burden):

Option	When to Use
Amazon Bedrock	Fully managed; select from models via API; Custom Model Import for own models
Amazon SageMaker AI endpoints	More flexibility; managed endpoints with inference-time customization (LoRA)
Amazon EC2	Maximum control; full infrastructure management

Training: Amazon SageMaker AI HyperPod provides managed infrastructure reducing training time by up to 32% for large clusters. Supports both EKS and Slurm orchestration. HyperPod recipes provide pre-configured training stacks for NVIDIA H100, A100, and AWS Trainium.

Observability: Amazon Managed Grafana and Prometheus for unified monitoring.

GENPERF04: Vector Store Optimization

How do you improve data retrieval performance?

BP01: Test Vector Embeddings for Latency and Relevance (Risk: Medium)

Effective chunking and embedding strategies have the greatest effect on performance.

Chunking strategies: Fixed-size, hierarchical, or semantic. Amazon Bedrock allows custom chunking via AWS Lambda functions.

Search algorithms (Approximate Nearest Neighbor):

Algorithm	Strength
LSH (Locality-Sensitive Hashing)	Fast indexing
HNSW (Hierarchical Navigable Small World)	High accuracy
IVF (Inverted File Index)	Balance of speed and accuracy
PQ (Product Quantization)	Compact storage

Key practices:

- Organize indices hierarchically (top-level for general, lower-level for detail)
- Implement query expansion using AI-generated context
- Shift from fuzzy matching toward semantic similarity
- Leverage hybrid search approaches (keyword + vector)
- Monitor embedding generation, index construction, query processing, result retrieval
- Maintain data quality through regular assessments and version control

BP02: Optimize Vector Sizes (Risk: Low)

Embedding models may support different vector sizes with performance trade-offs.

Key guidance:

- Start with compact encoding, increase if warranted
- Higher latency tolerance: larger vectors may offer more accuracy
- Low-latency needs: smaller vectors for faster retrieval
- A well-tuned model with smaller dimensions (256) can outperform a generic model with

larger dimensions (1024+)

- Check HuggingFace MTEB Leaderboard for benchmarks
- Consider self-hosting open-source embedding models via SageMaker if needed

Pillar 5: Cost Optimization

Cost optimization focuses on model selection, pricing model selection, cost-aware prompting, vector store optimization, and agent workflow optimization.

Principles:

- Optimize model and inference selection based on actual performance requirements
- Control resource consumption parameters (prompt lengths, response sizes, vector dimensions)
- Design workflow boundaries with clear limits and exit conditions

GENCOST01: Model Selection

How do you select the appropriate model to optimize costs?

BP01: Right-Size Model Selection (Risk: Medium)

Foundation model costs vary greatly across providers, families/sizes, and hosting paradigms.

Cost dimensions:

- Managed models (Amazon Bedrock): charged by tokens (input/output)
- Self-hosted models (EC2/SageMaker): charged by infrastructure uptime plus storage/network

Strategy:

- Start with a smaller model, gradually increase until acceptable performance
- Right-size as an ongoing activity (newer models may be smaller and more cost-effective)
- Decompose workloads and route to different-sized models based on complexity
- Amazon Bedrock Prompt Routing for dynamic routing within a model family
- Use batch inference for non-real-time needs
- Deploy multiple models to single multi-model endpoints where appropriate

GENCOST02: Pricing Model

How do you select a cost-effective pricing model?

BP01: Balance Cost and Performance of Inference Paradigms (Risk: Medium)

Key guidance:

- Amazon Bedrock Provisioned Throughput: prefer longer commitment terms for better unit costs;

validate with shorter commitments first

- Compare hosting on SageMaker AI vs. Amazon Bedrock Custom Model Import
- Deploy to the most cost-effective option where performance trade-offs are negligible

BP02: Optimize Resource Consumption for Hosting (Risk: Medium)

Key guidance:

- Right-size inference endpoints to smallest instance meeting performance goals
- Shut down during off-hours for predictable workloads
- Purchase EC2 Reserved Instances or Savings Plans
- Use SageMaker AI Inference Recommender for instance type evaluation
- SageMaker AI HyperPod Task Governance for dynamic resource allocation based on priority
- HyperPod Flexible Training Plans: set timeline/budget constraints for cost-optimized execution
- Advanced techniques: quantization, LoRA adaptation
- SageMaker AI HyperPod usage reporting for granular GPU/CPU visibility at team/task levels

GENCOST03: Cost-Aware Prompting

How do you engineer prompts to optimize cost?

BP01: Optimize Prompt Token Length (Risk: Medium)

Shorter prompts require less compute for inference.

Key techniques:

- Rigorous testing to shorten prompts without performance loss
- Use a separate LLM to shorten prompts (Amazon Bedrock Prompt Optimization)
- Zero-shot prompting for common tasks
- Chain-of-thought for reasoning tasks
- Least-to-most for complex problem decomposition

BP02: Control Model Response Length (Risk: Medium)

Control response sizes to reduce costs.

Key techniques:

- Set response length hyperparameters (max tokens)
- Add prompt instructions encouraging succinctness
- For deterministic scenarios: instruct model to return keys (True/False) instead of

full-text responses, reducing tokens and increasing determinism

BP03: Implement Prompt Caching (Risk: Medium)

Cache frequently used prompt portions to reduce inference costs and latency.

Amazon Bedrock prompt caching details:

- Cached tokens charged at reduced rate
- Tokens written to cache may be charged at higher rate
- Uncached tokens at standard rate
- Model-specific minimum/maximum token counts (e.g., Claude 3.7 Sonnet requires minimum

1,024 tokens per checkpoint)

- Five-minute TTL that resets with each hit

Best for: Workloads with long, repeated contexts (e.g., chatbot where users upload documents and ask questions -- cache the document content).

BP04: Annotate User Input for Cost-Aware Content Filtering (Risk: Medium)

Use input tags to selectively apply content filtering only to user-provided content, avoiding unnecessary processing of system prompts and trusted content.

Pattern: XML-style tags with unique random suffix per request to reduce prompt injection:

```
&lt;amazon-bedrock-guardrails-guardContent_xyz&gt;
[Content to be filtered]
&lt;/amazon-bedrock-guardrails-guardContent_xyz&gt;
```

Tag: User queries, current conversation turns, new/unverified content. **Leave untagged:** System prompts, verified search results, historical context, trusted content.

GENCOST04: Vector Store Costs

How do you optimize vector stores for cost?

BP01: Reduce Vector Length on Embedded Tokens (Risk: Medium)

Smaller vector sizes reduce embedding costs and database computation.

Key guidance:

- Use smallest vector length supported that meets quality requirements
- Smaller vectors = fewer tokens = reduced embedding cost
- Consider compressed vector types where supported
- May need to modify chunk size or introduce overlapping chunks
- Performance trade-off: test retrieval quality with smaller vectors

GENCOST05: Agent Workflow Costs

How do you optimize agent workflows for cost?

BP01: Create Stopping Conditions (Risk: High)

Agentic workflows can run indefinitely, incurring unbounded costs.

Key implementation details:

- Estimate maximum time needed for agent completion (model response + tool execution + network latency)
- Implement timeout mechanisms (Amazon Bedrock Agent timeout, application layer)
- Set timeouts on external tools (Lambda functions, API endpoints)
- Set token limits on model responses
- Evaluate asynchronous workflows for ability to interrupt/halt long-running events
- Ensure prompts handle timeout responses gracefully

Pillar 6: Sustainability

Sustainability focuses on minimizing computational resources for training, customization, hosting, data processing, and storage while leveraging model efficiency techniques and serverless architectures.

Principles:

- Identify if generative AI is the right solution (simpler approaches may suffice)
- Design for environmental efficiency (right-sized models, optimized data operations)
- Implement dynamic resource optimization (auto-scaling, serverless architectures)

GENSUS01: Energy-Efficient Infrastructure

How do you minimize computational resources for training, customizing, and hosting?

BP01: Implement Auto Scaling and Serverless Architectures (Risk: Medium)

Provision and consume resources only when needed.

Key implementation details:

- Amazon Bedrock and Amazon Q are fully managed (AWS handles infrastructure and scaling)
- SageMaker AI Inference Recommender for automated load testing and optimal instance selection
- Amazon EC2 Inferentia instances for highest compute power per watt
- SageMaker AI HyperPod for managed infrastructure with built-in scaling
- Scale self-hosted models to zero when not in use
- Configure idle timeout for SageMaker notebooks
- AWS Customer Carbon Footprint Tool for tracking environmental impact

BP02: Use Efficient Model Customization Services (Risk: Medium)

Maximize efficiency through distributed training and parameter-efficient fine-tuning.

Key techniques:

- Amazon Bedrock managed fine-tuning and continued pre-training
- SageMaker AI HyperPod with AWS Trainium instances for reduced energy consumption
- Parameter-Efficient Fine Tuning (PEFT) like LoRA to reduce parameters needing updates
- Managed spot training using spare EC2 capacity (with checkpointing)
- SageMaker AI Debugger for detecting/stopping problematic training jobs early

GENSUS02: Sustainable Data Processing and Storage

How do you optimize data processing and storage to minimize energy consumption?

BP01: Optimize Data Processing and Storage (Risk: Medium)

Key implementation details:

- Amazon S3 Intelligent-Tiering for automatic cost-effective storage class transitions
- S3 Lifecycle policies for data transitions and deletion
- Columnar formats (Apache Parquet) with compression to reduce data transfer and processing
- Amazon Athena for serverless queries directly on S3
- AWS Glue for serverless ETL (runs only when needed)
- AWS Lambda for event-driven serverless computing
- Amazon EMR with auto-scaling for large batch processing
- SageMaker AI HyperPod: differentiated training SLAs (production gets consistent resources, development gets longer completion windows, flexible training plans with cost-efficient capacity blocks)
- Monitor with AWS Cost Explorer, Trusted Advisor, CloudWatch metrics

GENSUS03: Energy-Efficient Models

How do you maintain model efficiency and resource optimization?

BP01: Leverage Smaller Models and Optimized Inference Techniques (Risk: Medium)

Implement model optimization to reduce resource consumption while meeting performance goals.

Key techniques:

Technique	Description
Quantization	Reducing precision of model weights/activations; decreases model size, speeds up inference
Pruning	Removing redundant/unnecessary parameters for more compact models
Efficient architectures	Fine-tune smaller models for specific use cases instead of using large general-purpose models
Model distillation (Amazon Bedrock)	Train smaller model to mimic a larger model; similar performance with fewer resources
SageMaker AI inference optimization	Apply quantization, speculative decoding, LoRA; evaluate for latency, throughput, price

Quick Reference: All Best Practices by Risk Level

High Risk (must implement)

ID	Best Practice	Pillar
GENOPS01-BP01	Periodically evaluate functional performance	Operational Excellence
GENOPS01-BP02	Collect and monitor user feedback	Operational Excellence
GENOPS02-BP01	Monitor all application layers	Operational Excellence
GENOPS02-BP02	Monitor foundation model metrics	Operational Excellence
GENOPS02-BP03	Mitigate risk of system overload	Operational Excellence
GENOPS03-BP01	Implement prompt template management	Operational Excellence
GENOPS03-BP02	Enable tracing for agents and RAG workflows	Operational Excellence
GENOPS04-BP01	Automate with IaC	Operational Excellence
GENOPS04-BP02	Implement GenAIOps	Operational Excellence
GENOPS05-BP01	Learn when to customize models	Operational Excellence
GENSEC01-BP01	Grant least privilege access to endpoints	Security
GENSEC01-BP02	Implement private network communication	Security
GENSEC01-BP03	Least privilege access to data stores	Security
GENSEC01-BP04	Implement access monitoring	Security
GENSEC02-BP01	Implement guardrails for harmful/incorrect responses	Security
GENSEC03-BP01	Control plane and data access monitoring	Security
GENSEC04-BP02	Sanitize and validate user inputs	Security
GENSEC05-BP01	Least privilege access for agentic workflows	Security
GENSEC06-BP01	Implement data purification filters	Security
GENREL03-BP02	Implement timeout mechanisms on agentic workflows	Reliability

ID	Best Practice	Pillar
GENREL06-BP01	Design for fault-tolerance in distributed compute	Reliability
GENCOST05-BP01	Create stopping conditions for agent workflows	Cost Optimization

Medium Risk (should implement)

ID	Best Practice	Pillar
GENSEC04-BP01	Implement a secure prompt catalog	Security
GENREL01-BP01	Scale and balance foundation model throughput	Reliability
GENREL02-BP01	Implement redundant network connections	Reliability
GENREL03-BP01	Manage prompt flows and recover from failure	Reliability
GENREL04-BP01	Implement a prompt catalog	Reliability
GENREL05-BP01	Load-balance inference across Regions	Reliability
GENREL05-BP02	Replicate embedding data across Regions	Reliability
GENREL05-BP03	Verify agent capabilities across Regions	Reliability
GENPERF01-BP01	Define a ground truth dataset	Performance Efficiency
GENPERF01-BP02	Collect performance metrics	Performance Efficiency
GENPERF02-BP01	Load test model endpoints	Performance Efficiency
GENPERF02-BP03	Select and customize appropriate model	Performance Efficiency
GENPERF03-BP01	Use managed solutions where appropriate	Performance Efficiency
GENPERF04-BP01	Test vector embeddings for latency and relevance	Performance Efficiency
GENCOST01-BP01	Right-size model selection	Cost Optimization
GENCOST02-BP01	Balance cost and performance of inference	Cost Optimization
GENCOST02-BP02	Optimize resource consumption for hosting	Cost Optimization
GENCOST03-BP01	Optimize prompt token length	Cost Optimization
GENCOST03-BP02	Control model response length	Cost Optimization
GENCOST03-BP03	Implement prompt caching	Cost Optimization
GENCOST03-BP04	Annotate user input for cost-aware filtering	Cost Optimization
GENCOST04-BP01	Reduce vector length on embedded tokens	Cost Optimization
GENSUS01-BP01	Implement auto scaling and serverless architectures	Sustainability
GENSUS01-BP02	Use efficient model customization services	Sustainability
GENSUS02-BP01	Optimize data processing and storage	Sustainability
GENSUS03-BP01	Leverage smaller models and optimized inference	Sustainability

Low Risk (consider implementing)

ID	Best Practice	Pillar
GENREL04-BP02	Implement a model catalog	Reliability
GENPERF02-BP02	Optimize inference parameters	Performance Efficiency
GENPERF04-BP02	Optimize vector sizes	Performance Efficiency

AWS Services Cross-Reference

The following table maps key AWS services to the Well-Architected GenAI Lens pillars where they are most heavily referenced:

AWS Service	OE	SEC	REL	PERF	COST	SUS
Amazon Bedrock	X	X	X	X	X	X
Amazon Bedrock Guardrails	X	X			X	
Amazon Bedrock Prompt Management	X	X	X		X	
Amazon Bedrock Flows	X		X			
Amazon Bedrock Knowledge Bases		X				
Amazon Bedrock Agents		X	X		X	
Amazon AgentCore	X					
Amazon SageMaker AI	X	X	X	X	X	X
SageMaker AI HyperPod	X	X	X	X	X	X
SageMaker AI Model Monitor	X			X		
SageMaker AI MLflow	X			X		
Amazon Q Business	X	X				
Amazon Q Developer		X				
Amazon CloudWatch	X	X	X	X		X
AWS CloudTrail	X	X				

AWS Service	OE	SEC	REL	PERF	COST	SUS
Amazon EventBridge	X		X			
AWS Step Functions	X					
AWS Lambda	X		X	X		X
Amazon API Gateway	X					
AWS IAM		X				
AWS PrivateLink		X	X			
Amazon OpenSearch Service		X	X	X	X	
Amazon S3	X	X	X	X		X
AWS CDK / CloudFormation	X					
AWS CodePipeline / CodeBuild	X					
Amazon Route 53			X			
Amazon CloudFront			X			
Amazon DynamoDB	X					
Amazon Athena	X			X		X
AWS Glue			X	X		X
Amazon Data Firehose	X	X				
QuickSight	X	X				
EC2 Inferentia / Trainium				X		X
Amazon FSx for Lustre			X			
Amazon Verified Permissions		X				
AWS Batch			X			X
SageMaker AI Inference Recommender					X	X